Code Crawler v4 — Complete Technical Study Guide

Audience: Embedded Linux C developers (drivers, HAL, CCSP, nl80211) who have not worked with databases, parsers, or parse trees before.

Purpose: After studying this guide, you will understand every component of Code Crawler deeply enough to design v5 and implement it yourself.

Table of Contents

1. Foundation Concepts
    - 1.1 What Is a Parser?
    - 1.2 What Is an Abstract Syntax Tree (AST)?
    - 1.3 What Is Tree-sitter? (And Why Not Just Regex?)
    - 1.4 What Is a Database? (DuckDB Specifically)
    - 1.5 What Is a Property Graph?
    - 1.6 What Is a Vector Embedding?
    - 1.7 What Is an Event Bus?
    - 1.8 What Is MCP (Model Context Protocol)?
2. The Core Engine — How It All Connects
    - 2.1 The Complete Indexing Pipeline (Step-by-Step)
    - 2.2 How the Engine Is Language-Agnostic
    - 2.3 The Universal Parse Contract (DTOs)
3. The C Parser — Deep Dive
    - 3.1 How Tree-sitter Parses C Code
    - 3.2 Walking the Parse Tree
    - 3.3 Extracting Functions, Structs, Variables, Calls
    - 3.4 The Regex Fallback
4. The Database Layer — Everything About Storage
    - 4.1 DuckDB Crash Course
    - 4.2 Our Complete Schema (Every Table Explained)
    - 4.3 Edge Tables — How Relationships Work
    - 4.4 The Property Graph (DuckPGQ)
    - 4.5 Vector Search (VSS)
    - 4.6 Pre-Materialized LLM Views
5. Data Flow — Source Code to Database
    - 5.1 The Complete Journey of a C File
    - 5.2 How ParseResult Maps to Database Tables
    - 5.3 How Relationship Chains Are Built
6. The Tiering System
    - 6.1 Why Tiers Exist
    - 6.2 The Four Tiers (i0–i3)
    - 6.3 Classification Pipeline
7. Priority Scoring — The Math
    - 7.1 The 6-Dimension Formula
    - 7.2 Each Dimension Explained With Math
    - 7.3 Worked Example

    - 7.4 Why These Weights? The Reasoning
    - 7.5 Self-Tuning Weights
8. The Event Bus Architecture
9. The Plugin System
10. MCP Integration
11. Build System Detection
12. Telemetry Correlation
13. Complete API Reference

---

1. Foundation Concepts

1.1 What Is a Parser?

You already know this concept — you just haven't called it "parsing" in the formal sense.

When the C compiler reads your driver code, the **first thing it does** is parse it. Parsing means reading raw text and understanding its structure. Think of it like this:

```
Your C file (text)  →  Parser  →  Structured representation (data you can
query)
```

Analogy from your world: When you write a Device Tree file (`.dts`), the DT compiler (`dtc`) parses your text into a structured binary blob (`.dtb`). The kernel then walks that structured blob to find `compatible` strings, register addresses, interrupt mappings, etc. The DT compiler is a parser. The `.dtb` is the structured output.

Code Crawler does the same thing but for a different purpose. Instead of compiling code into a binary, we parse code into a **database** so that AI models and developers can query it: "Show me every function that calls `ioctl()`" or "Which functions write to this global variable?"

What a parser produces

Given this C code:

```c
#include <stdio.h>
#define MAX_RETRIES 3

static int retry_count = 0;

int wifi_connect(const char *ssid, int timeout) {
    retry_count++;
    if (retry_count > MAX_RETRIES) {
        printf("Max retries exceeded for %s\n", ssid);
        return -1;
    }
    return hal_wifi_associate(ssid, timeout);
}
```

A parser extracts:

Element	What was extracted
Include	`stdio.h`
Macro	`MAX_RETRIES` = `3`
Variable	`retry_count`, type `int`, global, static
Function	`wifi_connect(const char *ssid, int timeout)`, lines 6–12
Call edges	`wifi_connect` → `printf` (line 9), `wifi_connect` → `hal_wifi_associate` (line 12)

That's what our parser does. It turns text into structured data.

1.2 What Is an Abstract Syntax Tree (AST)?

An AST is the **structured, tree-shaped representation** that the parser builds from your source code. "Abstract" means it drops unnecessary details (like whitespace and semicolons) and keeps only the meaningful structure.

Analogy: A Device Tree is literally a tree of nodes. `/soc/wifi@1800000/` is a node with children. An AST is the same concept but for source code.

Here's how the C code above becomes an AST:

```
translation_unit                        ← root (the whole file)
├── preproc_include                     ← #include <stdio.h>
│   └── system_lib_string: "stdio.h"
├── preproc_def                         ← #define MAX_RETRIES 3
│   ├── name: "MAX_RETRIES"
│   └── value: "3"
├── declaration                         ← static int retry_count = 0;
│   ├── storage_class: "static"
│   ├── type: "int"
│   └── declarator
│       ├── name: "retry_count"
│       └── initializer: 0
└── function_definition                 ← int wifi_connect(...)
    ├── type: "int"
    ├── declarator
    │   ├── name: "wifi_connect"
    │   └── parameters
    │       ├── parameter: "const char *ssid"
    │       └── parameter: "int timeout"
    └── body (compound_statement)
        ├── expression_statement        ← retry_count++;
        │   └── update_expression
        ├── if_statement                ← if (retry_count > MAX_RETRIES)
        │   ├── condition: binary_expression (>)
        │   └── consequence
        │       ├── call_expression     ← printf(...)
        │       └── return_statement    ← return -1;
        └── return_statement            ← return hal_wifi_associate(...)
            └── call_expression
                ├── function: "hal_wifi_associate"
                └── arguments: ssid, timeout
```

Why do we care about the tree structure?

Because it tells us **relationships**:

- `wifi_connect` **contains** the call to `printf` → we know wifi_connect calls printf
- `retry_count` is at the **top level** (child of `translation_unit`) → it's a global variable
- The `if_statement` adds 1 to the **cyclomatic complexity** of `wifi_connect`

Without the tree, we'd just have text. With the tree, we have **queryable structure**.

1.3 What Is Tree-sitter? (And Why Not Just Regex?)

The Regex Problem

You might think: "Why not just use regex to find functions?" Let's try:

```
# Naive regex: find C functions
pattern = r"(\w+)\s+(\w+)\s*\([^)]*\)\s*\{"
```

This breaks on:

```
// 1. Function pointers — regex thinks "callback" is a function definition
void (*callback)(int, int) = {NULL};

// 2. Multi-line signatures — regex misses this entirely
static inline int __attribute__((always_inline))
wifi_scan_handler(struct nl_msg *msg,
                  void *arg) {
    ...
}

// 3. #ifdef guards — regex can't know which branch is active
#ifdef CONFIG_WIFI_6E
int wifi_6e_init(void) { ... }
#else
int wifi_legacy_init(void) { ... }
#endif
```

Regex treats code as flat text. It has no concept of nesting, scope, or structure.

Tree-sitter: A Real Parser

Tree-sitter is a **parser generator library** originally built for code editors (syntax highlighting in VS Code, Neovim, etc.). It:

1. **Has grammar files** for each language (C, C++, Python, etc.) — these are the formal rules of the language
2. **Builds a real AST** — the tree structure from section 1.2

3. **Is fault-tolerant** — even if your code has syntax errors, it still produces a partial tree (crucial for indexing incomplete or broken code)
4. **Is incremental** — when you change one line, it doesn't re-parse the whole file

How Code Crawler uses Tree-sitter

```
import tree_sitter_c as tsc
from tree_sitter import Language, Parser

# 1. Create a parser for C
parser = Parser(Language(tsc.language()))

# 2. Feed it source code (as bytes)
source = open("wifi_driver.c", "rb").read()
tree = parser.parse(source)

# 3. Walk the AST tree
root = tree.root_node   # This is the "translation_unit" root

for node in walk(root):
    if node.type == "function_definition":
        # Found a function! Extract its name, signature, line numbers
        ...
    elif node.type == "call_expression":
        # Found a function call! Record the caller→callee edge
        ...
    elif node.type == "struct_specifier":
        # Found a struct! Extract its name and members
        ...
```

Tree-sitter Node Properties

Every node in the tree has:

Property	What it means	Example
`node.type`	The grammar rule name	`"function_definition"`, `"call_expression"`, `"identifier"`
`node.start_point`	`(row, column)` where the node starts	`(5, 0)` → line 6, column 0
`node.end_point`	`(row, column)` where the node ends	`(12, 1)` → line 13, column 1
`node.start_byte`	Byte offset in the source	`142`
`node.end_byte`	Byte offset end	`387`
`node.children`	Child nodes	The list of sub-nodes
`node.parent`	Parent node	The containing node

Property	What it means	Example
`node.child_by_field_name("name")`	Named child	Gets the "declarator" child of a function

To extract the actual text of a node, you slice the source bytes:

```
name = source_text[node.start_byte:node.end_byte]
# If node represents the identifier "wifi_connect", this returns "wifi_connect"
```

Why we also have libclang (and when Tree-sitter isn't enough)

Tree-sitter gives us **syntax** — the structure of the code. But it doesn't understand **semantics** — what the code means. For C specifically:

Capability	Tree-sitter	libclang
Find function definitions	▯	▯
Find function calls	▯	▯
Resolve `#ifdef` branches	×	▯ (needs compile_commands.json)
Resolve `typedef` chains	×	▯
Macro expansion	×	▯
Cross-file type resolution	×	▯
Works without build setup	▯	× (needs includes)
Fault-tolerant (broken code)	▯	× (fails on errors)

Our strategy: Use Tree-sitter as the primary parser (always works). When `compile_commands.json` is available (from the build system), use libclang for deeper semantic analysis like `#ifdef` resolution.

1.4 What Is a Database? (DuckDB Specifically)

Databases for C developers

You interact with "databases" every day — you just don't call them that:

Your world	Database world
A `.config` file with `CONFIG_WIFI=y` lines	A table with key-value rows
`grep -r "wifi_connect" .`	A SQL query: `SELECT * FROM Function WHERE name = 'wifi_connect'`
Device Tree: find all nodes with `compatible = "qcom,wifi"`	`SELECT * FROM DeviceTreeNode WHERE 'qcom,wifi' = ANY(compatible)`
`find . -name "*.c" -newer last_index`	`SELECT * FROM File WHERE last_modified > '2024-01-01'`

A database is just **structured storage with fast querying**. Instead of grep-ing through millions of lines of text every time, you store the structured data once and query it instantly.

Why DuckDB specifically?

There are hundreds of databases. We chose DuckDB because:

Feature	Why it matters for us
Single file	The entire index is one `.duckdb` file. No servers to install, no daemons. Like SQLite but much faster for analytics.
Embedded	Runs inside our Python process. No network calls to a separate database server.
SQL	Standard SQL queries — the most widely known query language.
DuckPGQ extension	Adds **property graph** queries (we'll explain this in §1.5).
VSS extension	Adds **vector similarity search** (we'll explain this in §1.6).
Fast analytics	Columnar storage — scanning millions of rows is extremely fast.
Zero admin	No configuration, no tuning, no maintenance. Just `import duckdb`.

DuckDB Crash Course — The Minimum You Need

```python
import duckdb

# 1. Connect (creates the file if it doesn't exist)
conn = duckdb.connect("my_index.duckdb")

# 2. Create a table
conn.execute("""
    CREATE TABLE Function (
        id BIGINT PRIMARY KEY,
        name TEXT,
        signature TEXT,
        start_line INT,
        end_line INT,
        complexity INT
    )
""")

# 3. Insert data
conn.execute("""
    INSERT INTO Function VALUES (1, 'wifi_connect', 'int wifi_connect(const
char *ssid, int timeout)', 6, 12, 2)
""")

# 4. Query data
result = conn.execute("SELECT * FROM Function WHERE name =
'wifi_connect'").fetchone()
# Returns: (1, 'wifi_connect', 'int wifi_connect(...)', 6, 12, 2)

# 5. Query with parameters (safe against injection)
```

```
result = conn.execute("SELECT * FROM Function WHERE name = ?",
['wifi_connect']).fetchone()

# 6. Join tables (combine data from multiple tables)
result = conn.execute("""
    SELECT f.name, f2.name AS calls_function
    FROM Function f
    JOIN calls c ON f.id = c.caller_id
    JOIN Function f2 ON c.callee_id = f2.id
    WHERE f.name = 'wifi_connect'
""").fetchall()
# Returns: [('wifi_connect', 'printf'), ('wifi_connect', 'hal_wifi_associate')]
```

Key SQL Concepts

Concept	What it means	C analogy
`TABLE`	A structured collection of rows	A `struct` array
`ROW`	One record in the table	One `struct` instance
`COLUMN`	A field in every row	A member of the `struct`
`PRIMARY KEY`	Unique identifier for each row	Like an array index, but named
`FOREIGN KEY` (`REFERENCES`)	A column that points to another table's primary key	Like a pointer to another struct
`JOIN`	Combine rows from multiple tables based on matching keys	Like following a pointer chain
`WHERE`	Filter rows	Like an `if` condition
`VIEW`	A saved query that acts like a virtual table	Like a `#define` for queries
`INDEX`	Speed up lookups on a column	Like building a hash table for fast lookup

1.5 What Is a Property Graph?

A property graph is a way to represent **relationships between things** as nodes and edges, where both nodes and edges can have properties (data attached to them).

You already think in graphs. Consider a driver binding:

```
DeviceTreeNode("wifi@1800000")  ──dt_binds_driver──▶
Function("qcom_wifi_probe")
         |                                                  |
         | compatible = "qcom,wcn6855"                      | calls
         |                                                  ▼
         |
Function("ieee80211_register_hw")
         |                                                  |
         |                                                  | calls
         |                                                  ▼
```

```
          |
Function("nl80211_init")
```

This IS a graph. Nodes are things (device tree entries, functions). Edges are relationships (binds-to, calls).

DuckPGQ — Graph Queries on DuckDB

DuckPGQ is an extension that lets us define a property graph on top of our regular tables and query it with graph-specific SQL. Here's how we define it:

```
CREATE PROPERTY GRAPH code_graph
  VERTEX TABLES (Function, File, Directory, Struct, Macro, Variable, ...)
  EDGE TABLES (
    calls          SOURCE KEY (caller_id) REFERENCES Function
                   DESTINATION KEY (callee_id) REFERENCES Function,
    contains_func  SOURCE KEY (file_id) REFERENCES File
                   DESTINATION KEY (func_id) REFERENCES Function,
    ...
  );
```

This tells DuckDB: "The `Function` table rows are nodes. The `calls` table rows are edges connecting two Function nodes." Now we can do graph queries like: "Find all functions reachable from `wifi_connect` within 3 hops."

Why graphs matter for Code Crawler

Without a graph, to answer "What happens if I change `wifi_connect`?", you'd need to:

1. `grep` for all callers of `wifi_connect`
2. For each caller, `grep` for THEIR callers
3. Repeat N times
4. Manually track which files, which processes, which hardware paths are affected

With a graph, it's one query: traverse all edges from `wifi_connect` and collect the blast radius.

1.6 What Is a Vector Embedding?

Imagine you could turn code into a "fingerprint" — a list of numbers that captures the **meaning** of what the code does, not just the text.

```
"int wifi_connect(const char *ssid)"  →  [0.23, -0.41, 0.87, ..., 0.12]   (384
numbers)
"int wlan_associate(char *network)"   →  [0.21, -0.39, 0.85, ..., 0.14]   (384
numbers, very similar!)
"void print_banner(void)"            →  [0.95, 0.33, -0.22, ..., -0.67]   (384
numbers, very different!)
```

The first two functions do similar things (connect to WiFi), so their number-lists are very close. The third function does something unrelated, so its numbers are far apart.

This "list of 384 numbers" is called a **vector embedding**. It's generated by a small AI model (`sentence-transformers/all-MiniLM-L6-v2` in our case).

Why this matters

It enables **semantic search** — finding code by meaning, not by exact text:

- Query: "connect to wireless network" → finds `wifi_connect`, `wlan_associate`, `ieee80211_connect`
- This works even though the query doesn't contain any of those exact function names

How cosine similarity works

Two vectors are "similar" if they point in the same direction. Cosine similarity measures this:

```
similarity = (A · B) / (|A| × |B|)

Where:
  A · B     = sum of (A[i] × B[i]) for all i    (dot product)
  |A|       = sqrt(sum of A[i]²)                  (magnitude)
```

- `similarity = 1.0` → identical meaning
- `similarity = 0.0` → completely unrelated
- `similarity = -1.0` → opposite meaning

DuckDB's VSS extension creates an **HNSW index** (a data structure optimized for fast nearest-neighbor search) so that finding the most similar vectors out of millions takes milliseconds, not hours.

1.7 What Is an Event Bus?

Analogy: Think of D-Bus or Netlink sockets. When the kernel wants to notify userspace that a WiFi interface came up, it doesn't call a specific function in your daemon. It broadcasts a Netlink event. Any daemon that subscribed to that event type receives it.

Our Event Bus works the same way, but inside our Python process:

```python
# PRODUCER: The crawler doesn't know or care who will consume this
event_bus.publish("file.parsed", parse_result)

# CONSUMER: The storage component subscribed earlier
event_bus.subscribe("file.parsed", storage.handle_parsed_file)

# CONSUMER: The tiering component also subscribed
event_bus.subscribe("file.parsed", tiering.update_tier_stats)
```

Why? Without the event bus, the crawler would need to `import storage` and `import tiering` and call their functions directly. This creates tight coupling — changing storage would break the crawler. With the event bus, components are completely independent.

Our Event Types

Event	Who Publishes	Who Listens	What Data
`file.discovered`	Pipeline	Tiering, Crawlers	`FileInfo`
`file.parsed`	Crawlers	Storage, Tiering	`ParseResult`
`build.detected`	Pipeline	Analyzers	`str` (build system type)
`tier.classified`	Tiering	Storage, Pipeline	`TierClassification`
`priority.scored`	Tiering	Storage, MCP	`PriorityScoreResult`
`manifest.built`	Tiering	Storage, MCP	`IndexManifestBundle`
`summary.generated`	Intelligence	Storage	`SummaryResult`
`patch.suggested`	Intelligence	Storage, UI	`PatchSuggestion`

1.8 What Is MCP (Model Context Protocol)?

MCP is a standard protocol for connecting AI models to tools and data sources. Think of it like a USB standard — any AI model that speaks MCP can plug into any MCP server.

Analogy: Like how nl80211 is a standard interface between userspace and the kernel WiFi stack — any WiFi driver that implements nl80211 works with any userspace tool (iw, wpa_supplicant, hostapd). MCP is the nl80211 of AI tools.

Code Crawler runs an MCP server. When an AI coding assistant (Cursor, Copilot, Claude) needs to understand your codebase, it calls our MCP tools:

```
AI Model  ──MCP call──▶  codecrawler MCP server  ──SQL──▶  DuckDB
                                                  ◀──data──
          ◀──response──
```

Our MCP Tools

Tool	What it does	Returns
`search_code(query)`	Semantic + keyword search	Ranked IndexManifest bundles
`get_call_hierarchy(func)`	Follow call edges in the graph	Call tree with summaries
`get_build_context(symbol)`	Look up `#ifdef CONFIG_*` status	Active/inactive paths
`trace_ipc_flow(func)`	Follow D-Bus/Ubus edges across processes	Cross-process call chain
`correlate_serial_log(lines)`	Hash log strings → find source	Functions that emit those logs
`analyze_impact(func)`	Blast radius analysis	All downstream dependencies

Tool	What it does	Returns
`sync_team()`	Pull team updates	Applied patches and summaries

2. The Core Engine — How It All Connects

2.1 The Complete Indexing Pipeline (Step-by-Step)

When you run `codecrawler index --project yocto --root /home/dev/rdk`, here is **exactly** what happens:

```
┌─────────────────────────────────────────────────────────────────────┐
│                    YOU RUN: codecrawler index                       │
└─────────────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌─────────────────────────────────────────────────────────────────────┐
│  STAGE 1: FILE DISCOVERY                                            │
│                                                                     │
│  os.walk(project_root)                                              │
│    → Skip hidden dirs (.git, .venv)                                 │
│    → For each file:                                                 │
│        1. Check extension → LANGUAGE_MAP[".c"] = "c"               │
│        2. Stat the file → get size_bytes                            │
│        3. SHA-256 hash the content → content_hash                   │
│        4. Create FileInfo(path, language, size, hash)               │
│        5. Publish "file.discovered" event                           │
│                                                                     │
│  Output: list[FileInfo] — e.g., 50,000 indexable files              │
└─────────────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌─────────────────────────────────────────────────────────────────────┐
│  STAGE 2: BUILD SYSTEM DETECTION                                    │
│                                                                     │
│  Scan project root for signature files:                             │
│    Yocto:     meta-*/conf/layer.conf, bblayers.conf                 │
│    Buildroot: Config.in, configs/*_defconfig                        │
│    Kernel:    Kconfig, arch/, drivers/                              │
│    Android:   Android.bp, build/envsetup.sh                         │
│    OpenWrt:   feeds.conf.default, target/linux/                     │
│                                                                     │
│  Score each system → highest score wins                             │
│  Publish "build.detected" event with "yocto"                        │
│                                                                     │
│  If Yocto detected:                                                 │
│    → Parse bblayers.conf → extract layer paths                      │
│    → Parse local.conf → extract MACHINE, DISTRO, features           │
│    → Parse .bb recipes → extract DEPENDS, SRC_URI                   │
└─────────────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
```

```
┌──────────────────────────────────────────────────────────────────┐
│  STAGE 3: TIER CLASSIFICATION (i0–i3)                            │
│                                                                  │
│  For each discovered directory:                                  │
│    Phase 1 — Heuristic/LLM classification:                       │
│      "gcc", "glibc", "busybox"       → i0 (ignore completely)  │
│      "systemd", "openssl", "hostapd" → i1 (stub: path+hash)    │
│      Unknown directories              → i2 (skeleton: sigs)     │
│      "vendor", "custom", "hal", "app" → i3 (full: everything)  │
│                                                                  │
│    Phase 2 — Git evidence:                                       │
│      git log --since="6 months" -- <dir>                         │
│      Has recent commits? → Bump to i2 minimum                    │
│                                                                  │
│    Phase 3 — Build config cross-reference:                       │
│      Directory in IMAGE_INSTALL? → Bump to i2+                   │
│      Directory excluded from build? → Keep i0/i1                 │
│                                                                  │
│  Publish "tier.classified" for each file                         │
│                                                                  │
│  Result: 95% of files classified as i0/i1 (skipped or minimal) │
│          5% of files classified as i2/i3 (actually parsed)       │
└──────────────────────────────────────────────────────────────────┘
                              │
                              │
                              ▼

┌──────────────────────────────────────────────────────────────────┐
│  STAGE 4: PARSING (The Core of Everything)                       │
│                                                                  │
│  For each file where tier >= 1:                                  │
│                                                                  │
│    1. Look up the correct crawler by language:                    |
│       registry.get_all(BaseCrawler) → [CCrawler, PyCrawler...] |
│       crawler_map["c"] = CCrawler                                |
│       crawler_map["python"] = PythonCrawler                      |
│                                                                  |
│    2. Call crawler.parse(file_info) → ParseResult                |
│                                                                  |
│    3. ParseResult contains:                                      |
│       - functions: [FunctionDef, FunctionDef, ...]               |
│       - structs:   [StructDef, StructDef, ...]                   |
│       - macros:    [MacroDef, MacroDef, ...]                     |
│       - variables: [VariableDef, VariableDef, ...]               |
│       - calls:     [CallEdge, CallEdge, ...]                     |
│       - includes:  [IncludeEdge, IncludeEdge, ...]               |
│                                                                  |
│    4. Publish "file.parsed" event with the ParseResult           |
└──────────────────────────────────────────────────────────────────┘
                              │
                              │
                              ▼

┌──────────────────────────────────────────────────────────────────┐
│  STAGE 5: DATABASE INGESTION                                     |
│                                                                  |
│  Storage component listens for "file.parsed" events:             |
│                                                                  |
│  For each ParseResult:                                           |
│    INSERT INTO File (path, hash, language, loc)                  |
```

```
│    For each function:                                          │
│      INSERT INTO Function (name, signature, start_line, ...)   │
│      INSERT INTO contains_func (file_id, func_id)              │
│    For each call:                                              │
│      INSERT INTO calls (caller_id, callee_id, call_site_line)  │
│    For each struct:                                            │
│      INSERT INTO Struct (name, members)                        │
│      INSERT INTO uses_struct (func_id, struct_id)              │
│    For each include:                                           │
│      INSERT INTO includes_file (source_id, target_id)          │
│    For each variable:                                          │
│      INSERT INTO Variable (name, var_type, is_global, ...)     │
│    For each macro:                                             │
│      INSERT INTO Macro (name, value, is_config_guard)          │
└────────────────────────────────────────────────────────────────┘
                              │
                              │
                              ▼

┌────────────────────────────────────────────────────────────────┐
│  STAGE 6: PRIORITY SCORING                                     │
│                                                                │
│  For each function in the database:                            │
│    Compute 6-dimension score (see section 7 for full math)     │
│    INSERT INTO PriorityScore (func_id, composite_score, ...)   │
│                                                                │
│  Publish "priority.scored" events                              │
└────────────────────────────────────────────────────────────────┘
                              │
                              │
                              ▼

┌────────────────────────────────────────────────────────────────┐
│  STAGE 7: MANIFEST BUILDING                                    │
│                                                                │
│  For each parsed file:                                         │
│    Compress from ~15,000 tokens to ~500 tokens                 │
│    Include: file metadata, function signatures, struct names,  │
│             call edges, include edges, global variables         │
│    Exclude: function bodies, local variables, comments          │
│    INSERT INTO IndexManifest (file_id, manifest_json)          │
│                                                                │
│  This is what MCP serves to AI agents — compact context        │
└────────────────────────────────────────────────────────────────┘
                              │
                              │
                              ▼

┌────────────────────────────────────────────────────────────────┐
│  STAGE 8: VECTOR EMBEDDING (Background)                        │
│                                                                │
│  For each function/file/variable:                              │
│    text = f"{name} {signature} {summary}"                      │
│    embedding = sentence_transformer.encode(text)               │
│    → Returns FLOAT[384] vector                                 │
│    UPDATE Function SET embedding = ? WHERE id = ?              │
│                                                                │
│  Build HNSW indexes for fast similarity search                 │
└────────────────────────────────────────────────────────────────┘
                              │
                              │
                              ▼

┌────────────────────────────────────────────────────────────────┐
```

```
│  DONE. The database now contains:                                  │
│    - Every function, struct, variable, macro                       │
│    - Every call relationship (who calls whom)                      │
│    - Every include relationship (who includes whom)                │
│    - Tier classifications and priority scores                      │
│    - Pre-built context manifests for AI agents                     │
│    - Vector embeddings for semantic search                         │
│                                                                    │
│  Total: a complete, queryable, searchable knowledge graph          │
│         of your entire codebase.                                   │
└────────────────────────────────────────────────────────────────────┘
```

2.2 How the Engine Is Language-Agnostic

The core engine **never touches language-specific parsing**. It only knows about the universal `ParseResult` DTO. This is the key architectural decision that makes the system extensible.

```
                    ┌──────────────────────┐
                    │    BaseCrawler (ABC)  │
                    │                      │
                    │  + name: str         │
                    │  + supported_langs   │
                    │  + parse(FileInfo)   │
                    │    → ParseResult     │
                    └──────────┬───────────┘
                               │
              ┌────────────────┼────────────────┐
              │                │                │
        ┌─────▼──────┐   ┌─────▼──────┐   ┌─────▼──────┐
        │ CCrawler   │   │ PyCrawler  │   │ ShCrawler  │
        │            │   │            │   │            │
        │ langs:     │   │ langs:     │   │ langs:     │
        │ ["c","cpp"]│   │ ["python"] │   │ ["shell"]  │
        │            │   │            │   │            │
        │ Uses:      │   │ Uses:      │   │ Uses:      │
        │ tree-sitter│   │ ast module │   │ regex      │
        └────────────┘   └────────────┘   └────────────┘
```

The contract: Every crawler, no matter the language, must return the same `ParseResult` structure. The core engine doesn't care HOW you parsed the file — it only cares that you gave it functions, structs, calls, and includes in the standard format.

How the engine selects the right crawler

```python
# In pipeline.py, Stage 4:
crawlers = registry.get_all(BaseCrawler)    # Get all registered crawlers
crawler_map = {}
for crawler in crawlers:
    for lang in crawler.supported_languages:
        crawler_map[lang] = crawler
```

```
# crawler_map = {"c": CCrawler, "cpp": CCrawler, "python": PyCrawler, "shell":
ShCrawler}

# For each file:
crawler = crawler_map.get(file_info.language)  # e.g., file is .c → gets
CCrawler
result = crawler.parse(file_info)              # Returns ParseResult regardless
of language
```

Adding a new language (e.g., Rust)

You would create `codecrawler/crawlers/rust_crawler.py`:

```
class RustCrawler(BaseCrawler):
    @property
    def supported_languages(self) -> list[str]:
        return ["rust"]

    def parse(self, file_info: FileInfo) -> ParseResult:
        # Use tree-sitter-rust to parse
        # Extract functions, structs, calls, etc.
        # Return a standard ParseResult
        ...
```

Register it as a plugin, and the engine automatically routes `.rs` files to it. **Zero changes to the core engine.**

2.3 The Universal Parse Contract (DTOs)

Every piece of data that crosses a component boundary uses a typed dataclass. Here's what each one means:

FileInfo — "Here's a file I found"

```
@dataclass(frozen=True)
class FileInfo:
    path: Path          # /home/dev/rdk/ccsp/wifi/wifi_hal.c
    language: str       # "c"
    size_bytes: int     # 45230
    content_hash: str   # "a3f8b2c1..." (SHA-256 of file content)
    tier: int           # 0–3 (assigned later by tiering)
```

FunctionDef — "Here's a function I extracted"

```
@dataclass(frozen=True)
class FunctionDef:
    name: str           # "wifi_connect"
    signature: str      # "int wifi_connect(const char *ssid, int timeout)"
    start_line: int     # 6
    end_line: int       # 12
```

```
    complexity: int     # 2 (cyclomatic complexity: 1 base + 1 for the if
statement)
    body_hash: str      # hash of the function body (for change detection)
```

CallEdge — "Function A calls function B"

```
@dataclass(frozen=True)
class CallEdge:
    caller: str              # "wifi_connect"
    callee: str              # "hal_wifi_associate"
    call_site_line: int      # 12 (the line where the call happens)
```

ParseResult — "Here's everything I found in this file"

```
@dataclass(frozen=True)
class ParseResult:
    file_info: FileInfo
    functions: list[FunctionDef]     # All function definitions
    structs: list[StructDef]         # All struct/class definitions
    macros: list[MacroDef]           # All #define macros
    variables: list[VariableDef]     # All variable declarations
    calls: list[CallEdge]           # All function calls (A→B edges)
    includes: list[IncludeEdge]     # All #include relationships
```

Think of ParseResult as a self-contained package: everything the engine needs to know about one file, in one object. The engine doesn't need to re-read the file or re-parse anything.

---

3. The C Parser — Deep Dive

Since you're a C developer, this section walks through exactly how our C parser (`codecrawler/crawlers/c_crawler.py`) processes your code.

3.1 How Tree-sitter Parses C Code

When `CCrawler.parse()` is called, here's the exact sequence:

```
# Step 1: Read the file
source = file_info.path.read_text(encoding="utf-8", errors="replace")

# Step 2: Create the Tree-sitter parser
import tree_sitter_c as tsc
from tree_sitter import Language, Parser

parser = Parser(Language(tsc.language()))

# Step 3: Parse source into AST
tree = parser.parse(source.encode("utf-8"))
```

```
root = tree.root_node
# root.type == "translation_unit" — this is the top of the tree
```

The tree-sitter C grammar knows every valid C construct: function definitions, struct declarations, `#define` macros, `if`/`while`/`for` statements, pointer declarations, typedefs, etc.

3.2 Walking the Parse Tree

We use a recursive generator to visit every node:

```
def _walk(self, node):
    """Visit this node, then recursively visit all children."""
    yield node                    # Process this node
    for child in node.children:   # Then go deeper
        yield from self._walk(child)
```

This is a **preorder depth-first traversal** — same as how you'd walk a device tree: visit the parent first, then each child recursively.

For each node, we check its `type` property and decide what to do:

```
for node in self._walk(root):
    if node.type == "function_definition":
        # Found: int wifi_connect(...) { ... }
        func = self._extract_function(node, source)

    elif node.type in ("struct_specifier", "class_specifier"):
        # Found: struct wifi_config { ... };
        struct = self._extract_struct(node, source)

    elif node.type == "declaration" and self._is_global_scope(node):
        # Found: static int retry_count = 0;  (at file scope)
        var = self._extract_variable(node, source)

    elif node.type == "call_expression":
        # Found: hal_wifi_associate(ssid, timeout)
        call = self._extract_call(node, source)
```

3.3 Extracting Functions, Structs, Variables, Calls

Function Extraction

Given this C code:

```
int wifi_connect(const char *ssid, int timeout) {
    return hal_wifi_associate(ssid, timeout);
}
```

The tree-sitter node for the function looks like:

```
function_definition
├── type: "int"
├── declarator (function_declarator)
│   ├── declarator (identifier): "wifi_connect"
│   └── parameters (parameter_list)
│       ├── parameter_declaration: "const char *ssid"
│       └── parameter_declaration: "int timeout"
└── body (compound_statement)
    └── return_statement
        └── call_expression: "hal_wifi_associate(ssid, timeout)"
```

Our extraction code:

```python
def _extract_function(self, node, source):
    # Get the declarator child (contains the name and parameters)
    declarator = node.child_by_field_name("declarator")

    # Drill down to find the actual identifier node
    name_node = declarator
    while name_node.type not in ("identifier", "field_identifier"):
        if name_node.type == "function_declarator":
            name_node = name_node.child_by_field_name("declarator")
        elif name_node.children:
            name_node = name_node.children[0]
        else:
            break

    # Extract the name by slicing the source text
    name = source[name_node.start_byte:name_node.end_byte]
    # name == "wifi_connect"

    # Get the signature (everything before the body)
    body = node.child_by_field_name("body")
    signature = source[node.start_byte:body.start_byte].strip()
    # signature == "int wifi_connect(const char *ssid, int timeout)"

    return FunctionDef(
        name="wifi_connect",
        signature="int wifi_connect(const char *ssid, int timeout)",
        start_line=1,
        end_line=3,
        complexity=1,  # No branches = complexity 1
    )
```

Struct Extraction

```python
struct wifi_config {
    char ssid[32];
    int channel;
```

```
    enum wifi_band band;
};
```

```
def _extract_struct(self, node, source):
    name_node = node.child_by_field_name("name")
    name = source[name_node.start_byte:name_node.end_byte]
    # name == "wifi_config"

    members = []
    body = node.child_by_field_name("body")
    for child in body.children:
        if child.type == "field_declaration":
            member_text =
source[child.start_byte:child.end_byte].strip().rstrip(";")
            members.append(member_text)
    # members == ["char ssid[32]", "int channel", "enum wifi_band band"]

    return StructDef(name="wifi_config", members=members)
```

Global Variable Detection

```
def _is_global_scope(self, node):
    """A declaration is global if its parent is the translation_unit (file
root)."""
    parent = node.parent
    return parent is not None and parent.type == "translation_unit"
```

This distinguishes `static int retry_count = 0;` (file scope → global) from `int i = 0;` inside a function (local → ignored).

Macro and Include Extraction

These use simple text scanning (macros and includes are preprocessor directives, not part of the C grammar tree):

```
def _extract_macros(self, source):
    for line in source.splitlines():
        if line.strip().startswith("#define"):
            parts = line.strip()[len("#define"):].strip().split(None, 1)
            name = parts[0].split("(")[0]  # Handle function-like macros
            value = parts[1] if len(parts) > 1 else ""
            # Is this a CONFIG_ guard?
            is_guard = name.startswith("CONFIG_") or name.endswith("_H")
            # is_guard matters because CONFIG_ macros control #ifdef branches

def _extract_includes(self, source, file_path):
    for line in source.splitlines():
        if line.strip().startswith("#include"):
            if '"' in line:
                target = line.split('"')[1]       # #include "wifi_hal.h"
```

```
            elif '<' in line:
                target = line.split('<')[1].split('>')[0]  # #include <stdio.h>
```

3.4 The Regex Fallback

If tree-sitter is not installed, we fall back to regex:

```
pattern = re.compile(r"^(\w[\w\s\*]+?)\s+(\w+)\s*\(([^)]*)\)\s*\{",
re.MULTILINE)
```

This catches simple patterns like `int wifi_connect(...) {` but misses complex signatures. It's a safety net, not a replacement.

---

4. The Database Layer — Everything About Storage

4.1 DuckDB — How We Actually Talk to It

Here is the complete sequence of database interactions in Code Crawler:

```
import duckdb

# ═══ CONNECTION ═══
# Creates the database file if it doesn't exist
conn = duckdb.connect(".codecrawler/index.duckdb")

# ═══ SCHEMA CREATION ═══
# Creates all tables (see §4.2)
conn.execute(SCHEMA_DDL)

# ═══ INSERTING DATA ═══
# After parsing a file:
conn.execute(
    "INSERT INTO File (id, path, hash, language, loc) VALUES (?, ?, ?, ?, ?)",
    [file_id, "/path/to/wifi_hal.c", "abc123...", "c", 450]
)

# After parsing functions:
conn.execute(
    "INSERT INTO Function (id, file_id, name, signature, start_line, end_line,
complexity) VALUES (?, ?, ?, ?, ?, ?, ?)",
    [func_id, file_id, "wifi_connect", "int wifi_connect(...)", 6, 12, 2]
)

# Record that this file contains this function:
conn.execute(
    "INSERT INTO contains_func (file_id, func_id) VALUES (?, ?)",
    [file_id, func_id]
)
```

```
# Record call edges:
conn.execute(
    "INSERT INTO calls (caller_id, callee_id, call_site_line) VALUES (?, ?,
?)",
    [wifi_connect_id, hal_wifi_associate_id, 12]
)

# ═══ QUERYING DATA ═══
# Find a function:
result = conn.execute(
    "SELECT * FROM Function WHERE name = ?", ["wifi_connect"]
).fetchone()

# Find what a function calls:
callees = conn.execute("""
    SELECT f2.name, c.call_site_line
    FROM calls c
    JOIN Function f2 ON c.callee_id = f2.id
    WHERE c.caller_id = ?
""", [wifi_connect_id]).fetchall()

# Find the full call chain (who calls whom calls whom):
chain = conn.execute("""
    SELECT f1.name AS caller, f2.name AS callee, fi.path AS file
    FROM calls c
    JOIN Function f1 ON c.caller_id = f1.id
    JOIN Function f2 ON c.callee_id = f2.id
    JOIN contains_func cf ON f2.id = cf.func_id
    JOIN File fi ON cf.file_id = fi.id
""").fetchall()
```

4.2 Our Complete Schema — Every Table Explained

Core Tables (the "things" in our codebase)

Table	What it stores	Key columns	C analogy
Directory	Every directory	`path`, `summary`, `depth`, `is_custom`	A folder in your project
File	Every source file	`path`, `hash`, `language`, `loc`, `embedding`	A `.c` or `.h` file
Function	Every function	`name`, `signature`, `start_line`, `end_line`, `complexity`, `embedding`	A C function
Struct	Every struct/class	`name`, `members[]`, `summary`	A `struct` definition
Macro	Every `#define`	`name`, `value`, `is_config_guard`	`#define CONFIG_WIFI 1`
Variable	Global/static vars	`name`, `var_type`, `is_global`, `write_count`, `embedding`	`static int g_wifi_state`

Table	What it stores	Key columns	C analogy
BuildConfig	Build config entries	`key`, `value`, `source_file`, `build_system`	`CONFIG_WIFI=y` from `.config`
DeviceTreeNode	DT entries	`path`, `compatible[]`, `properties`	A DT node like `wifi@1800000`
LogLiteral	Log string literals	`hash`, `literal_string`, `log_level`	`printk("wifi error: %d\n")`

Edge Tables (the "relationships" between things)

Edge tables store **connections**. They have exactly two foreign key columns pointing to two entities.

Edge Table	Connects	What it means	Example
contains_dir	Directory → Directory	Folder contains subfolder	`src/` contains `src/wifi/`
contains_file	Directory → File	Folder contains file	`src/wifi/` contains `wifi_hal.c`
contains_func	File → Function	File defines function	`wifi_hal.c` defines `wifi_connect()`
calls	Function → Function	Function calls function	`wifi_connect()` calls `hal_wifi_associate()`
uses_struct	Function → Struct	Function uses struct type	`wifi_connect()` uses `struct wifi_config`
includes_file	File → File	File includes another file	`wifi_hal.c` includes `wifi_hal.h`
guarded_by	Function → BuildConfig	Function is behind `#ifdef`	`wifi_6e_init()` guarded by `CONFIG_WIFI_6E`
dt_binds_driver	DeviceTreeNode → Function	DT node binds to probe	`wifi@1800000` binds `qcom_wifi_probe()`
calls_over_ipc	Function → Function	Cross-process call via D-Bus/Ubus	Python app → C daemon via D-Bus
emits_log	Function → LogLiteral	Function emits this log string	`wifi_connect()` emits `"WiFi: connected to %s"`

Intelligence Tables (computed metadata)

Table	What it stores	Purpose
Tier	`path`, `tier` (0–3), `confidence`	Which directories are important
PriorityScore	6 dimension scores + composite	Which functions matter most
IndexManifest	Compressed JSON per file	Pre-built context for AI agents
SummaryMeta	Which model summarized what, confidence	Track summary quality

Table	What it stores	Purpose
RuntimeTrace	GDB/Valgrind trace data per function	Runtime behavior data
SyncLog	Delta changes for team sync	Collaboration history
Annotation	AI-generated annotations + human approval	Enrichment layer

4.3 How Relationship Chains Work

Let's trace a real query: **"What files are affected if I change `wifi_connect()`?"**

```sql
-- Step 1: Find wifi_connect's ID
SELECT id FROM Function WHERE name = 'wifi_connect';
-- Returns: id = 42

-- Step 2: Find everything that CALLS wifi_connect (upstream callers)
SELECT f.name, fi.path
FROM calls c
JOIN Function f ON c.caller_id = f.id
JOIN contains_func cf ON f.id = cf.func_id
JOIN File fi ON cf.file_id = fi.id
WHERE c.callee_id = 42;
-- Returns: main_connect() in connection_manager.c
--          reconnect_handler() in wifi_monitor.c

-- Step 3: Follow further upstream (callers of callers)
-- This is recursive — in a graph database, it's a traversal
-- In SQL, you'd use a recursive CTE:
WITH RECURSIVE call_chain AS (
    -- Base case: direct callers of wifi_connect
    SELECT c.caller_id AS func_id, 1 AS depth
    FROM calls c WHERE c.callee_id = 42

    UNION ALL

    -- Recursive case: callers of callers
    SELECT c.caller_id, cc.depth + 1
    FROM calls c
    JOIN call_chain cc ON c.callee_id = cc.func_id
    WHERE cc.depth < 5  -- limit depth
)
SELECT DISTINCT f.name, fi.path, cc.depth
FROM call_chain cc
JOIN Function f ON cc.func_id = f.id
JOIN contains_func cf ON f.id = cf.func_id
JOIN File fi ON cf.file_id = fi.id
ORDER BY cc.depth;
```

This gives you the complete blast radius: every function and file that would be affected by a change to `wifi_connect()`.

4.4 The Property Graph (DuckPGQ)

The property graph is an alternative way to query the same relationships. Instead of writing complex JOINs in SQL, you can write graph queries:

```
-- CREATE the graph (done once at schema setup)
CREATE PROPERTY GRAPH code_graph
  VERTEX TABLES (Function, File, Directory, Struct, ...)
  EDGE TABLES (
    calls SOURCE KEY (caller_id) REFERENCES Function
          DESTINATION KEY (callee_id) REFERENCES Function,
    contains_func SOURCE KEY (file_id) REFERENCES File
                  DESTINATION KEY (func_id) REFERENCES Function,
    ...
  );
```

The vertex tables (Function, File, etc.) become **nodes**. The edge tables (calls, contains_func, etc.) become **edges** between nodes. Same data, different way to query it.

4.5 Vector Search (VSS)

After parsing, we compute embeddings for functions, files, and variables:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# Generate embedding for a function
text = "wifi_connect: int wifi_connect(const char *ssid, int timeout) —
Connects to WiFi"
embedding = model.encode(text)
# embedding = [0.23, -0.41, 0.87, ..., 0.12]  ← 384 floating point numbers

# Store in database
conn.execute(
    "UPDATE Function SET embedding = ? WHERE id = ?",
    [embedding.tolist(), func_id]
)
```

Create HNSW indexes for fast search:

```
INSTALL vss; LOAD vss;
CREATE INDEX func_embedding_idx ON Function USING HNSW (embedding) WITH (metric
= 'cosine');
```

Search by meaning:

```
query_embedding = model.encode("connect to wireless network")
results = conn.execute("""
    SELECT name, signature, array_cosine_distance(embedding, ?::FLOAT[384]) AS
```

```
distance
    FROM Function
    WHERE embedding IS NOT NULL
    ORDER BY distance ASC
    LIMIT 5
""", [query_embedding.tolist()]).fetchall()
# Returns: wifi_connect, wlan_associate, ieee80211_connect, ...
```

4.6 Pre-Materialized LLM Views

These are saved queries that combine data from multiple tables, ready for instant retrieval:

LLM_HighPriority — "Give me the most important functions"

```
CREATE VIEW LLM_HighPriority AS
    SELECT f.*, ps.composite_score, t.tier, sm.confidence
    FROM Function f
    JOIN PriorityScore ps ON f.id = ps.func_id     -- has a score
    JOIN contains_func cf ON f.id = cf.func_id
    JOIN contains_file cfl ON cf.file_id = cfl.file_id
    JOIN Tier t ON t.path = (SELECT path FROM File WHERE id = cf.file_id)
    LEFT JOIN SummaryMeta sm ON f.id = sm.entity_id
    WHERE t.tier >= 2                               -- only skeleton+ tiers
    ORDER BY ps.composite_score DESC;               -- highest priority first
```

An AI agent calls: `SELECT * FROM LLM_HighPriority LIMIT 20` and gets the 20 most important functions across the entire codebase in one query.

LLM_SharedState — "Show me dangerous global variables"

```
CREATE VIEW LLM_SharedState AS
    SELECT v.*, f.name AS func_name, fi.path AS file_path
    FROM Variable v
    JOIN Function f ON v.func_id = f.id
    JOIN File fi ON v.file_id = fi.id
    WHERE v.is_global = TRUE AND v.write_count > 1
    ORDER BY v.write_count DESC;
```

This finds global variables written by more than one function — potential thread-safety bugs in your drivers.

---

5. Data Flow — Source Code to Database

5.1 The Complete Journey of a C File

Let's trace `wifi_hal.c` from disk to database:

```c
// wifi_hal.c
#include "wifi_hal.h"
#include <stdio.h>

#define CONFIG_MAX_SSID_LEN 32

static int g_wifi_state = 0;

struct wifi_config {
    char ssid[CONFIG_MAX_SSID_LEN];
    int channel;
};

int wifi_connect(const char *ssid, int timeout) {
    struct wifi_config cfg;
    g_wifi_state = 1;
    printf("Connecting to %s\n", ssid);
    return hal_wifi_associate(ssid, timeout);
}

void wifi_disconnect(void) {
    g_wifi_state = 0;
    hal_wifi_disassociate();
}
```

Step 1: FileInfo created

```
FileInfo(
    path=Path("/rdk/ccsp/wifi/wifi_hal.c"),
    language="c",           # from LANGUAGE_MAP[".c"]
    size_bytes=512,
    content_hash="a3f8b2c1d4e5...",
    tier=3                  # classified as full indexing
)
```

Step 2: CCrawler.parse() produces ParseResult

```
ParseResult(
    file_info=<FileInfo above>,
    functions=[
        FunctionDef(name="wifi_connect", signature="int wifi_connect(const char
*ssid, int timeout)",
                    start_line=14, end_line=19, complexity=1),
        FunctionDef(name="wifi_disconnect", signature="void
wifi_disconnect(void)",
                    start_line=21, end_line=24, complexity=1),
    ],
    structs=[
        StructDef(name="wifi_config", members=["char
ssid[CONFIG_MAX_SSID_LEN]", "int channel"]),
    ],
```

```
    macros=[
        MacroDef(name="CONFIG_MAX_SSID_LEN", value="32", is_config_guard=True),
    ],
    variables=[
        VariableDef(name="g_wifi_state", var_type="int", is_global=True,
is_static=True, line=7),
    ],
    calls=[
        CallEdge(caller="", callee="printf", call_site_line=17),
        CallEdge(caller="", callee="hal_wifi_associate", call_site_line=18),
        CallEdge(caller="", callee="hal_wifi_disassociate", call_site_line=23),
    ],
    includes=[
        IncludeEdge(source_path="wifi_hal.c", target_path="wifi_hal.h"),
        IncludeEdge(source_path="wifi_hal.c", target_path="stdio.h"),
    ],
)
```

Step 3: Database ingestion

```
-- File
INSERT INTO File (id, path, hash, language, loc) VALUES (1,
'/rdk/ccsp/wifi/wifi_hal.c', 'a3f8b2...', 'c', 24);

-- Functions
INSERT INTO Function (id, file_id, name, signature, start_line, end_line,
complexity)
    VALUES (101, 1, 'wifi_connect', 'int wifi_connect(const char *ssid, int
timeout)', 14, 19, 1);
INSERT INTO Function (id, file_id, name, signature, start_line, end_line,
complexity)
    VALUES (102, 1, 'wifi_disconnect', 'void wifi_disconnect(void)', 21, 24,
1);

-- File→Function edges
INSERT INTO contains_func (file_id, func_id) VALUES (1, 101);
INSERT INTO contains_func (file_id, func_id) VALUES (1, 102);

-- Call edges
INSERT INTO calls (caller_id, callee_id, call_site_line) VALUES (101,
<printf_id>, 17);
INSERT INTO calls (caller_id, callee_id, call_site_line) VALUES (101,
<hal_wifi_associate_id>, 18);
INSERT INTO calls (caller_id, callee_id, call_site_line) VALUES (102,
<hal_wifi_disassociate_id>, 23);

-- Struct
INSERT INTO Struct (id, file_id, name, members) VALUES (201, 1, 'wifi_config',
['char ssid[32]', 'int channel']);

-- Variable (global)
INSERT INTO Variable (id, file_id, name, var_type, is_global, is_static)
    VALUES (301, 1, 'g_wifi_state', 'int', TRUE, TRUE);
```

```
-- Macro
INSERT INTO Macro (id, file_id, name, value, is_config_guard)
    VALUES (401, 1, 'CONFIG_MAX_SSID_LEN', '32', TRUE);

-- Include edges
INSERT INTO includes_file (source_id, target_id) VALUES (1, <wifi_hal_h_id>);
INSERT INTO includes_file (source_id, target_id) VALUES (1, <stdio_h_id>);
```

Step 4: What the database looks like after

```
Function table:
┌────┬────────┬──────────────────┬──────────────────────────────────────────────
────┬────┬────┬────┐
│ id  │ file_id │ name             │ signature
│ s  │ e  │ cx │
├────┼────────┼──────────────────┼──────────────────────────────────────────────
────┼────┼────┼────┤
│ 101 │ 1       │ wifi_connect     │ int wifi_connect(const char *ssid, int
timeout)│ 14 │ 19 │ 1  │
│ 102 │ 1       │ wifi_disconnect  │ void wifi_disconnect(void)
│ 21 │ 24 │ 1  │
└────┴────────┴──────────────────┴──────────────────────────────────────────────
────┴────┴────┴────┘

calls table:
┌──────────┬──────────┬──────┐
│ caller_id │ callee_id │ line │
├──────────┼──────────┼──────┤
│ 101       │ 501       │ 17   │   ← wifi_connect → printf
│ 101       │ 502       │ 18   │   ← wifi_connect → hal_wifi_associate
│ 102       │ 503       │ 23   │   ← wifi_disconnect → hal_wifi_disassociate
└──────────┴──────────┴──────┘

Variable table:
┌────┬────────┬────────────┬────────┬──────┬──────┐
│ id  │ file_id │ name       │ var_type │ global │ static │
├────┼────────┼────────────┼────────┼──────┼──────┤
│ 301 │ 1       │ g_wifi_state │ int    │ TRUE  │ TRUE  │
└────┴────────┴────────────┴────────┴──────┴──────┘
```

---

6. The Tiering System

6.1 Why Tiers Exist

A Yocto build tree can have **5+ million lines of code**. If we index every file at full detail:

- Parsing: days, not minutes
- Database: tens of GB
- AI context: impossible (100K-token context window can't hold 5M lines)
- Cost: sending all that to an LLM API costs thousands of dollars per query

The solution: **most of that code doesn't matter to you**. You're not modifying gcc. You're not debugging glibc. You're working on your vendor layer, your HAL, your CCSP components. The tiering system identifies what matters and skips the rest.

6.2 The Four Tiers

Tier	Label	What happens	Percentage of codebase	Examples
i0	Ignore	Completely skipped. Not even the filename is recorded.	~60%	gcc, glibc, busybox, toolchain, kernel core (if untouched)
i1	Stub	Only filename, path, and content hash stored. No parsing.	~25%	systemd, dbus, openssl, avahi, standard packages
i2	Skeleton	Function signatures, call edges, struct definitions extracted. No summaries, no body analysis.	~10%	Integration APIs, OpenWrt feeds, Android HAL stubs
i3	Full	Complete AST analysis: everything + LLM summaries + variable tracking + vector embeddings.	~5%	Your vendor code, custom daemons, CCSP components, HAL implementations

The math impact

```
Without tiering:  5,000,000 lines × 15 tokens/line = 75,000,000 tokens to
process
With tiering:     250,000 lines (i2+i3) × 15 tokens/line = 3,750,000 tokens
                  ──────────────────────────────────────────────────────────
                  95% REDUCTION in compute, storage, and AI context cost
```

6.3 Classification Pipeline

Phase 1: Heuristic Recognition

The system has lists of well-known directories:

```
KNOWN_I0_DIRS = {
    "binutils", "gcc", "glibc", "uclibc", "musl", "toolchain",
    "busybox", "coreutils", "util-linux", "ncurses", "zlib",
    ".git", "__pycache__", "node_modules",
}

KNOWN_I1_DIRS = {
    "systemd", "dbus", "avahi", "udev", "pam", "openssl",
    "libnl", "iptables", "iproute2", "hostapd", "wpa_supplicant",
}

KNOWN_I3_KEYWORDS = {
    "custom", "vendor", "proprietary", "app", "application",
```

```
    "service", "daemon", "agent", "manager", "hal",
}
```

In v5, this will be augmented with an actual LLM call: feed the directory tree to a 3B model and ask "which of these directories contain standard upstream code vs. custom vendor code?"

Phase 2: Git Evidence

```
git log --oneline --since="6 months ago" -- drivers/net/wireless/vendor/
# If there are recent commits → someone is actively working here → bump to i2
minimum
```

Phase 3: Build Config Cross-Reference

```
Is "wifi-hal" in IMAGE_INSTALL?  →  Yes  →  bump to i2+
Is "unused-package" built at all? →  No  →  keep at i0/i1
```

---

7. Priority Scoring — The Math

7.1 The 6-Dimension Formula

Every function gets scored on 6 independent dimensions. The scores are combined into a single number:

```
composite = (tier_weight × 0.25) + (usage_freq × 0.20) + (centrality × 0.15) +
            (build_active × 0.10) + (runtime_freq × 0.15) + (recency × 0.15)
```

The composite score ranges from 0.0 (irrelevant) to 1.0 (critical). The weights sum to 1.0.

7.2 Each Dimension — Detailed Math

Dimension 1: Tier Weight (25%)

```
tier_weight = tier_level / 3.0

tier_level = 0 (i0) → tier_weight = 0.000  (ignored code)
tier_level = 1 (i1) → tier_weight = 0.333  (stub)
tier_level = 2 (i2) → tier_weight = 0.667  (skeleton)
tier_level = 3 (i3) → tier_weight = 1.000  (actively developed)
```

Why 25%? The tier is the strongest signal. If code is in your vendor layer (i3), it's probably important. If it's in glibc (i0), it's definitely not.

Dimension 2: Usage Frequency (20%)

```
usage_frequency = call_count(function) / max_call_count_across_all_functions

Example:
  wifi_connect is called 50 times across the codebase
  The most-called function (main) is called 200 times
  usage_frequency = 50 / 200 = 0.25
```

Why? A function called 50 times has 50× the ripple effect of a function called once. Changing it affects more code paths.

Dimension 3: Graph Centrality (15%)

```
centrality = betweenness_centrality(function)   (capped at 1.0)
```

Betweenness centrality measures: **"How many shortest paths between other functions pass through this function?"**

```
A → B → C → D
        ↑
        |
    E → B → F

B has HIGH centrality because it's on the path between A→C, A→D, E→C, E→D, E→F
```

A function with high centrality is a **bottleneck** or **bridge** — changing it disconnects parts of the graph. Think of it like a HAL function that sits between 5 applications and 3 hardware drivers.

Why 15%? Centrality identifies critical infrastructure functions that don't get called often but are essential bridges.

Dimension 4: Build Guard Activation (10%)

```
build_guard_activation = 1.0 if function's #ifdef guard is ACTIVE in current
config
                       = 0.0 if function's #ifdef guard is INACTIVE (dead code)

Example:
  #ifdef CONFIG_WIFI_6E
  int wifi_6e_init(void) { ... }    ← only scores 1.0 if CONFIG_WIFI_6E=y
  #endif
```

Why 10%? Dead code behind inactive `#ifdef` guards is effectively not part of the binary. No point prioritizing it for analysis.

Dimension 5: Runtime Frequency (15%)

```
runtime_frequency = runtime_hits(function) / max_runtime_hits

Sources:
  - GDB breakpoint hit counts
  - Valgrind/ASan stack traces
  - perf profiler samples
```

Why? Static analysis tells you what code COULD do. Runtime tells you what it ACTUALLY does. A function that runs 100K times/sec on the target is more important than one that runs once at boot.

Dimension 6: Recency (15%)

```
recency = 1.0 / (1.0 + days_since_last_modification)

Modified today:      1 / (1 + 0) = 1.000
Modified yesterday:  1 / (1 + 1) = 0.500
Modified last week:  1 / (1 + 7) = 0.125
Modified last month: 1 / (1 + 30) = 0.032
Modified last year:  1 / (1 + 365) = 0.003
```

This is a **smooth decay function** — not a hard cutoff. Code modified today is ~300× more important than code modified a year ago.

Why this formula? It's a **hyperbolic decay** — the importance drops rapidly at first (don't care about last year's code) but has a long tail (code from 2 months ago still has some relevance). This is better than a hard "6 month cutoff" because it's continuous.

7.3 Worked Example

Let's score `wifi_connect()`:

```
Given:
  tier_level = 3 (vendor HAL code, i3)
  call_count = 50, max_call_count = 200
  betweenness = 0.45
  build_guard_active = True (CONFIG_WIFI=y)
  runtime_hits = 1000, max_runtime_hits = 5000
  last_modified = 2 days ago

Compute:
  tier_weight    = 3 / 3.0                = 1.000 × 0.25 = 0.250
  usage_freq     = 50 / 200               = 0.250 × 0.20 = 0.050
  centrality     = min(0.45, 1.0)         = 0.450 × 0.15 = 0.068
  build_active   = 1.0                    = 1.000 × 0.10 = 0.100
  runtime_freq   = 1000 / 5000           = 0.200 × 0.15 = 0.030
  recency        = 1/(1+2)               = 0.333 × 0.15 = 0.050
                                          ──────────────────────
  composite_score                                        = 0.548

This function scores 0.548 — it will rank in the top tier of importance.
```

Compare with `print_banner()`:

```
Given:
  tier_level = 1 (stub library)
  call_count = 1, max_call_count = 200
  betweenness = 0.0 (leaf function)
  build_guard_active = False
  runtime_hits = 1, max_runtime_hits = 5000
  last_modified = 400 days ago

Compute:
  tier_weight    = 1/3.0 = 0.333 × 0.25 = 0.083
  usage_freq     = 1/200 = 0.005 × 0.20 = 0.001
  centrality     = 0.0   × 0.15         = 0.000
  build_active   = 0.0   × 0.10         = 0.000
  runtime_freq   = 1/5000= 0.0002×0.15  = 0.000
  recency        = 1/401 = 0.002 × 0.15 = 0.000
                                          ________
  composite_score                         = 0.084

This function scores 0.084 — it will be in the bottom tier. AI agents will
never waste context on it.
```

7.4 Why These Specific Weights?

Weight	Value	Reasoning
Tier	0.25	Strongest single signal. Custom code vs. upstream determines importance more than anything else.
Usage	0.20	Call-count identifies hub functions. A function called 100 times has 100× the blast radius.
Centrality	0.15	Identifies bridge APIs (HAL layers, middleware) that connect subsystems.
Build	0.10	Binary check — either the code is compiled or it's dead. Lower weight because most active code passes this.
Runtime	0.15	Real execution data trumps static guesses. Hot-path functions in a 100K/sec loop matter more.
Recency	0.15	Active development focus. What were you working on this week? That's what you'll query about.

7.5 Self-Tuning Weights

In v5, the system will monitor which functions developers and AI agents actually query. If your team constantly queries recently-modified functions, the system increases `W_e` (recency weight). If your team queries by crash traces a lot, `W_r` (runtime weight) gets boosted.

This is **meta-learning**: the scoring system learns from your team's actual behavior, not just default assumptions.

---

8. The Event Bus Architecture

How It Works Internally

```
class EventBus:
    def __init__(self):
        # Dictionary mapping event names to lists of handler functions
        self._handlers = defaultdict(list)
        # {"file.parsed": [storage_handler, tiering_handler], ...}

    def subscribe(self, event_type, handler):
        self._handlers[event_type].append(handler)

    def publish(self, event_type, payload=None):
        for handler in self._handlers.get(event_type, []):
            try:
                handler(payload)    # Call each subscribed handler
            except Exception:
                log.exception(...)  # One handler failing doesn't crash others
```

Key design point: If one handler throws an exception, the others still run. This is fault isolation — a bug in the storage component doesn't crash the tiering component.

How components wire up at startup

```
# During application startup:
bus = EventBus()
registry = ServiceRegistry()

# Load and activate plugins
plugins = load_builtin_plugins()
plugin_registry = PluginRegistry(registry, bus)
plugin_registry.register_all(plugins)    # Each plugin registers its services
plugin_registry.activate_all()           # Each plugin subscribes to events

# Now when the pipeline runs:
pipeline = IndexingPipeline(config, registry, bus)
pipeline.run()
# Pipeline publishes events → plugins receive them via the bus
```

9. The Plugin System

Why Plugins?

Without plugins, adding a new crawler means editing core engine code. With plugins, you drop in a new file and it's automatically discovered.

Plugin Lifecycle

```
DISCOVER → REGISTER → ACTIVATE → (runtime) → DEACTIVATE

1. DISCOVER: Scan for plugins via entry points or filesystem
2. REGISTER: Call plugin.register(registry) — plugin adds its services
3. ACTIVATE: Call plugin.activate(event_bus) — plugin subscribes to events
4. RUNTIME:  Plugin handles events, produces data
5. DEACTIVATE: Call plugin.deactivate() — cleanup on shutdown
```

How the C Crawler Is a Plugin

```python
class CCrawlerPlugin(PluginBase):
    @property
    def manifest(self) -> PluginManifest:
        return PluginManifest(
            name="c_crawler",
            version="4.0.0",
            description="C/C++ parser using Tree-sitter + libclang",
            author="Code Crawler Team",
            plugin_type="crawler",
        )

    def register(self, registry):
        # Register a CCrawler instance as a BaseCrawler service
        registry.register(BaseCrawler, CCrawler())

    def activate(self, event_bus):
        # Could subscribe to events here if needed
        pass
```

When the pipeline needs crawlers, it asks: `registry.get_all(BaseCrawler)` — and gets the CCrawler that was registered by this plugin.

---

10. MCP Integration

How AI Agents Connect

```
┌──────────────┐      MCP Protocol      ┌──────────────────┐      SQL
┌──────────┐
│  AI Agent    │ ──────────────────────►│ MCP Server       │ ──────────────►│
DuckDB  │
│  (Cursor,    │     Tool call:         │ (codecrawler)    │  Query the     │
index   │
│   Claude)    │  search_code("wifi")   │                  │  database      │
.duckdb │
│              │ ◄──────────────────────│                  │ ◄──────────────│
│
```

```
│              │     Response:          │                  │  Return rows   │
│
│              │  [IndexManifest...]    │                  │                │
│
└──────────────┘                        └──────────────────┘
└───────────┘
```

The MCP server exposes 7 tools and 3 resources. When an AI agent needs context about your codebase, it calls these tools instead of reading raw files.

Resource URIs

AI agents can also request pre-built resources:

URI	What it returns
`codecrawler://manifest/path/to/file.c`	The IndexManifest for that file (~500 tokens)
`codecrawler://llm_view/high_priority`	Top N functions by composite score
`codecrawler://telemetry/wifi`	Recent crashes and warnings for the WiFi subsystem

11. Build System Detection

How Detection Works

The detector scores each build system by looking for signature files:

```
BUILD_SIGNATURES = {
    "yocto": ["meta-*/conf/layer.conf", "poky/", "build/conf/bblayers.conf",
...],
    "buildroot": ["Config.in", "configs/*_defconfig", "Makefile.legacy", ...],
    "kernel": ["Kconfig", "arch/", "drivers/", "include/linux/", ...],
    "android": ["Android.bp", "build/envsetup.sh", "device/", ...],
    "openwrt": ["feeds.conf.default", "target/linux/", ...],
}

# For each build system, count how many signatures exist
# Glob matches = 1 point, exact match = 2 points
# Highest score wins
```

What Happens After Detection

Build system	What gets parsed	Configuration extracted
Yocto	`bblayers.conf`, `local.conf`, `.bb` recipes	Layer paths, MACHINE, DISTRO, DISTRO_FEATURES, IMAGE_INSTALL

Build system	What gets parsed	Configuration extracted
Buildroot	`.config` file	Enabled/disabled packages (BR2_PACKAGE_*), target architecture
Kernel	`.config`, `Kconfig` files	Enabled/disabled CONFIG_* symbols (used for `#ifdef` resolution)

12. Telemetry Correlation

The Problem

You see this in your serial console:

```
[  12.345678] wifi: Failed to associate with AP (err=-110)
[  12.345700] wifi: Retry 3/3 exhausted for SSID "HomeNet"
```

Which function emitted these? In a 5-million line codebase, `grep` takes minutes and returns too many results.

Our Solution: Hash-Based Instant Lookup

At index time:

```
# For each source file, extract log string literals
for line in source.splitlines():
    for pattern in LOG_PATTERNS:  # printk, ALOGE, RDK_LOG, fprintf(stderr,...)
        match = pattern.search(line)
        if match:
            raw_string = match.group(1)
            # Strip format specifiers for stable hashing
            cleaned = re.sub(r'%[dufslxXp]', '%s', raw_string)
            # "Failed to associate with AP (err=%s)" → hash
            string_hash = hashlib.md5(cleaned.encode()).hexdigest()[:16]

            # Store in database
            INSERT INTO LogLiteral (hash, literal_string, log_level) VALUES (?,
?, ?)
            INSERT INTO emits_log (func_id, log_id) VALUES (?, ?)
```

At crash time:

```
# Take the crash log line
crash_line = "wifi: Failed to associate with AP (err=-110)"
# Clean it (strip timestamps, PIDs, actual values)
cleaned = "Failed to associate with AP (err=%s)"
# Hash it
crash_hash = hashlib.md5(cleaned.encode()).hexdigest()[:16]
```

```
# Instant lookup!
SELECT f.name, fi.path, f.start_line
FROM LogLiteral ll
JOIN emits_log el ON ll.id = el.log_id
JOIN Function f ON el.func_id = f.id
JOIN File fi ON f.file_id = fi.id
WHERE ll.hash = ?
-- Returns: wifi_associate() in drivers/net/wireless/vendor/wifi.c line 234
```

50 crash lines → 50 hash lookups → instant mapping to exact functions. No grep needed.

---

13. Complete API Reference

Core Engine Functions

Function	File	Purpose
`IndexingPipeline.run()`	`core/pipeline.py`	Execute the full 8-stage pipeline
`IndexingPipeline._discover_files()`	`core/pipeline.py`	Walk filesystem, emit `file.discovered` events
`IndexingPipeline._detect_build_system()`	`core/pipeline.py`	Auto-detect Yocto/Buildroot/Kernel
`IndexingPipeline._classify_tiers()`	`core/pipeline.py`	Run tier classification (i0–i3)
`IndexingPipeline._parse_files()`	`core/pipeline.py`	Route each file to correct crawler
`load_config(path)`	`core/config.py`	Load TOML config with defaults
`EventBus.subscribe(event, handler)`	`core/event_bus.py`	Register event handler
`EventBus.publish(event, data)`	`core/event_bus.py`	Fire event to all handlers
`ServiceRegistry.register(type, impl)`	`core/registry.py`	Register a service implementation
`ServiceRegistry.get(type)`	`core/registry.py`	Get a registered service

Crawler Functions

Function	File	Purpose
`BaseCrawler.parse(FileInfo)`	`crawlers/base.py`	Abstract: parse file → ParseResult
`CCrawler._parse_with_treesitter()`	`crawlers/c_crawler.py`	Tree-sitter AST parsing

Function	File	Purpose
`CCrawler._extract_function()`	`crawlers/c_crawler.py`	Extract FunctionDef from AST node
`CCrawler._extract_struct()`	`crawlers/c_crawler.py`	Extract StructDef from AST node
`CCrawler._extract_includes()`	`crawlers/c_crawler.py`	Extract #include edges
`CCrawler._extract_macros()`	`crawlers/c_crawler.py`	Extract #define macros
`CCrawler._extract_functions_fallback()`	`crawlers/c_crawler.py`	Regex fallback when tree-sitter unavailable

Storage Functions

Function	File	Purpose
`Database(path)`	`storage/database.py`	Create/open DuckDB database
`Database.initialize()`	`storage/database.py`	Create all tables and views
`Database.get_stats()`	`storage/database.py`	Get row counts for all tables
`create_schema(conn)`	`storage/schema.py`	Execute DDL for all 20+ tables
`create_property_graph(conn)`	`storage/graph.py`	Create DuckPGQ property graph
`get_call_hierarchy(conn, name)`	`storage/graph.py`	Traverse call graph from function
`get_ipc_flow(conn, name)`	`storage/graph.py`	Trace cross-process IPC flow
`install_vss(conn)`	`storage/vector.py`	Install VSS extension
`create_vector_indexes(conn)`	`storage/vector.py`	Create HNSW indexes
`semantic_search(conn, embedding)`	`storage/vector.py`	Cosine similarity search

Tiering Functions

Function	File	Purpose
`TierClassifier.classify(files)`	`tiering/classifier.py`	Classify files into tiers i0–i3
`PriorityScorer.score(func_id, ...)`	`tiering/priority_scorer.py`	Compute 6-dimension composite score
`ManifestBuilder.build(ParseResult)`	`tiering/manifest_builder.py`	Compress file to ~500 token manifest

Intelligence Functions

Function	File	Purpose

Function	File	Purpose
`ProactiveAgent.scan_shared_state()`	`intelligence/proactive_agent.py`	Find thread-unsafe globals
`Summarizer.summarize_function()`	`intelligence/summarizer.py`	Generate function summary
`TelemetryCorrelator.extract_log_literals()`	`intelligence/telemetry.py`	Extract and hash log strings
`TelemetryCorrelator.correlate_crash_log()`	`intelligence/telemetry.py`	Map crash lines to functions

Plugin Functions

Function	File	Purpose
`discover_plugins(paths)`	`plugins/loader.py`	Find plugins via entry points + filesystem
`load_builtin_plugins()`	`plugins/loader.py`	Load C, Python, Shell crawlers
`PluginRegistry.register_all(plugins)`	`plugins/registry.py`	Call register() on all plugins
`PluginRegistry.activate_all()`	`plugins/registry.py`	Call activate() on all plugins

Build Analyzer Functions

Function	File	Purpose
`detect_build_system(root)`	`analyzers/build_detector.py`	Auto-detect build system type
`analyze_yocto_project(root)`	`analyzers/yocto.py`	Parse Yocto layers, recipes, config
`parse_dotconfig(path)`	`analyzers/buildroot.py`	Parse Buildroot .config
`parse_kernel_dotconfig(path)`	`analyzers/kernel.py`	Parse kernel .config
`build_ifdef_symbol_table(config)`	`analyzers/kernel.py`	Build CONFIG_* → bool table for #ifdef resolution
`generate_compile_commands(root)`	`analyzers/kernel.py`	Generate compile_commands.json for libclang

Summary — The Big Picture

```
Your C code → Tree-sitter parser → AST → Walk tree → Extract elements →
    → DTOs (ParseResult) → Event Bus → Storage (DuckDB) →
        → Tables + Edges (relationship graph) →
            → Priority Scores (6-dim math) →
                → IndexManifests (compressed context) →
                    → MCP Server → AI Agents query your codebase
```

Every step has been designed so that:

1. **You only index what matters** (tiering eliminates 95%)
2. **AI agents get context in 1 call** (pre-built manifests)
3. **New languages plug in** without changing the engine
4. **Components don't break each other** (event bus decoupling)
5. **Math decides importance** (not gut feeling)

Welcome to Code Crawler. You now understand the foundations. Time to build v5. □

PART II — DEEP-DIVE FUNDAMENTALS

The chapters above gave you the Code Crawler overview. This part goes **much deeper** into every technology we use, written specifically for an embedded C developer who wants to truly understand the internals, not just the surface.

---

14. SQL Fundamentals — A Complete Course for C Developers

You've spent your career writing C. SQL is the language of databases. This chapter teaches you SQL from absolute zero, using analogies to C data structures you already know.

14.1 The Mental Model: SQL Tables Are Struct Arrays

In C, you'd store a list of functions like this:

```c
typedef struct {
    int id;
    int file_id;
    char name[256];
    char signature[512];
    int start_line;
    int end_line;
    int complexity;
} Function;
```

```
Function functions[10000];  // Array of Function structs
int function_count = 0;
```

In SQL, the equivalent is a **table**:

```
CREATE TABLE Function (
    id         BIGINT PRIMARY KEY,
    file_id    BIGINT,
    name       TEXT,
    signature  TEXT,
    start_line INT,
    end_line   INT,
    complexity INT
);
```

C concept	SQL concept
`typedef struct { ... } Function;`	`CREATE TABLE Function ( ... );`
`Function functions[10000];`	The table itself (it auto-grows)
`functions[i].name`	`SELECT name FROM Function WHERE id = i`
`function_count++; functions[count] = {...};`	`INSERT INTO Function VALUES (...)`
`functions[i].complexity = 5;`	`UPDATE Function SET complexity = 5 WHERE id = i`
`memmove(&functions[i], ...)` (delete)	`DELETE FROM Function WHERE id = i`
`for (int i=0; i<count; i++) if (functions[i].complexity > 5) ...`	`SELECT * FROM Function WHERE complexity > 5`

14.2 CREATE TABLE — Defining Your Schema

```
CREATE TABLE IF NOT EXISTS File (
    id            BIGINT PRIMARY KEY,     -- unique row identifier (like array
index)
    path          TEXT UNIQUE,            -- UNIQUE means no duplicates allowed
    hash          TEXT,                   -- no constraint, can be NULL
    last_modified TIMESTAMP,             -- date+time type
    is_custom     BOOL DEFAULT FALSE,    -- defaults to FALSE if not specified
    language      TEXT,
    loc           INT DEFAULT 0,
    embedding     FLOAT[384]             -- array of 384 floats (vector)
);
```

Column Types (with C equivalents)

SQL Type	C Equivalent	Example

SQL Type	C Equivalent	Example
`BIGINT`	`long long` or `int64_t`	`42`, `9999999999`
`INT` or `INTEGER`	`int` or `int32_t`	`42`
`FLOAT`	`float`	`3.14`
`DOUBLE`	`double`	`3.14159265358979`
`BOOL`	`bool` or `_Bool`	`TRUE`, `FALSE`
`TEXT`	`char*` (heap-allocated string)	`'wifi_connect'`
`TIMESTAMP`	`struct timespec`	`'2024-01-15 10:30:00'`
`JSON`	`char*` containing JSON	`'{"key": "value"}'`
`TEXT[]`	`char**` (array of strings)	`['member1', 'member2']`
`FLOAT[384]`	`float[384]` (fixed-size array)	`[0.1, 0.2, ..., 0.9]`

Constraints

```
CREATE TABLE Function (
    id         BIGINT PRIMARY KEY,              -- 1. PRIMARY KEY: unique, not
null, fast lookup
    file_id    BIGINT REFERENCES File(id),      -- 2. FOREIGN KEY: must match
an existing File.id
    name       TEXT,                             -- 3. No constraint: any
value, including NULL
    start_line INT,
    complexity INT DEFAULT 1,                   -- 4. DEFAULT: use 1 if not
specified
    tier       INT CHECK (tier BETWEEN 0 AND 3) -- 5. CHECK: must be 0, 1, 2,
or 3
);
```

C analogy for FOREIGN KEY: It's like a pointer. `file_id REFERENCES File(id)` means "this value MUST be the `id` of an existing row in the `File` table." If you try to insert a `Function` with `file_id = 999` but no `File` with `id = 999` exists, the database **rejects the insert**. This is like dereferencing a NULL pointer — except the database catches it for you instead of segfaulting.

14.3 INSERT — Adding Data

```
-- Basic insert
INSERT INTO Function (id, file_id, name, signature, start_line, end_line,
complexity)
VALUES (1, 100, 'wifi_connect', 'int wifi_connect(const char *ssid)', 14, 19,
2);

-- Insert with defaults (complexity will be 1)
INSERT INTO Function (id, file_id, name, signature, start_line, end_line)
VALUES (2, 100, 'wifi_disconnect', 'void wifi_disconnect(void)', 21, 24);
```

```
-- Insert multiple rows at once
INSERT INTO Function (id, file_id, name, signature, start_line, end_line,
complexity) VALUES
    (3, 101, 'hal_init', 'int hal_init(void)', 1, 15, 3),
    (4, 101, 'hal_cleanup', 'void hal_cleanup(void)', 17, 25, 1),
    (5, 102, 'main', 'int main(int argc, char *argv[])', 1, 50, 8);
```

Parameterized Queries (Preventing SQL Injection)

In C, you wouldn't do `sprintf(query, "... WHERE name = '%s'", user_input)` — that's like a buffer overflow. Same in SQL. Instead:

```
# WRONG (SQL injection vulnerability — like sprintf with user input)
conn.execute(f"SELECT * FROM Function WHERE name = '{user_input}'")

# RIGHT (parameterized — like using prepared statements)
conn.execute("SELECT * FROM Function WHERE name = ?", [user_input])
```

The `?` is a placeholder. DuckDB safely substitutes the value without interpreting it as SQL code.

14.4 SELECT — Querying Data

Basic Queries

```
-- Get everything from Function table (like iterating the entire array)
SELECT * FROM Function;

-- Get specific columns
SELECT name, signature, complexity FROM Function;

-- Filter rows (like an if condition in your loop)
SELECT * FROM Function WHERE complexity > 5;

-- Multiple conditions (AND, OR)
SELECT * FROM Function WHERE complexity > 3 AND file_id = 100;
SELECT * FROM Function WHERE name = 'wifi_connect' OR name = 'wifi_disconnect';

-- Pattern matching (like strstr or regex)
SELECT * FROM Function WHERE name LIKE 'wifi_%';    -- starts with 'wifi_'
SELECT * FROM Function WHERE name LIKE '%connect%';  -- contains 'connect'

-- Count rows
SELECT COUNT(*) FROM Function;
SELECT COUNT(*) FROM Function WHERE complexity > 5;

-- Get unique values
SELECT DISTINCT language FROM File;

-- Sort results
SELECT * FROM Function ORDER BY complexity DESC;     -- highest first
```

```
SELECT * FROM Function ORDER BY name ASC;            -- alphabetical

-- Limit results
SELECT * FROM Function ORDER BY complexity DESC LIMIT 10;  -- top 10 most
complex

-- Aggregate functions
SELECT MAX(complexity) FROM Function;                -- highest complexity
SELECT MIN(complexity) FROM Function;                -- lowest complexity
SELECT AVG(complexity) FROM Function;                -- average complexity
SELECT SUM(loc) FROM File;                           -- total lines of code
```

GROUP BY — Aggregate Per Category

```
-- How many functions per file?
SELECT file_id, COUNT(*) AS func_count
FROM Function
GROUP BY file_id
ORDER BY func_count DESC;
-- Returns:
-- file_id | func_count
-- 102     | 45
-- 100     | 12
-- 101     | 8

-- Average complexity per language
SELECT f.language, AVG(fn.complexity) AS avg_complexity
FROM Function fn
JOIN File f ON fn.file_id = f.id
GROUP BY f.language;

-- Filter groups (HAVING is WHERE for aggregated groups)
SELECT file_id, COUNT(*) AS func_count
FROM Function
GROUP BY file_id
HAVING COUNT(*) > 10;  -- only files with more than 10 functions
```

14.5 JOIN — Combining Tables

This is the most important SQL concept for understanding Code Crawler. JOINs follow pointer chains between tables.

The Problem JOINs Solve

You have two tables:

```
File table:                        Function table:
┌─────┬────────────────┬───────┐    ┌────┬─────────┬────────────────┐
│ id  │ path           │ lang  │    │ id │ file_id │ name           │
├─────┼────────────────┼───────┤    ├────┼─────────┼────────────────┤
│ 100 │ wifi_hal.c     │ c     │    │  1 │ 100     │ wifi_connect   │
```

```
  | 101 | hal_init.c    | c     |      |  2  | 100      | wifi_disc     |
  | 102 | main.c        | c     |      |  3  | 101      | hal_init      |
  └─────┴───────────────┴───────┘      |  4  | 102      | main          |
                                       └─────┴──────────┴───────────────┘
```

Question: "What file contains `wifi_connect`?"

In C, you'd follow the pointer:

```
Function *f = find_function("wifi_connect");   // f->file_id = 100
File *file = find_file(f->file_id);            // file->path = "wifi_hal.c"
```

In SQL, you use a JOIN:

```
SELECT fn.name, f.path
FROM Function fn
JOIN File f ON fn.file_id = f.id      -- "follow the pointer"
WHERE fn.name = 'wifi_connect';
-- Returns: wifi_connect | wifi_hal.c
```

How JOIN Works Internally

```
Step 1: Take every row in Function
Step 2: For each row, find the matching row(s) in File where Function.file_id =
File.id
Step 3: Combine the columns from both tables into a single result row
```

It's like a nested loop in C:

```
for (int i = 0; i < function_count; i++) {
    for (int j = 0; j < file_count; j++) {
        if (functions[i].file_id == files[j].id) {
            // This pair matches! Output both rows combined.
            printf("%s | %s\n", functions[i].name, files[j].path);
        }
    }
}
```

(In practice, databases use indexes to avoid the $O(n^2)$ nested loop — more like a hash table lookup.)

Multi-Table JOINs (Following Multiple Pointers)

Question: "Show me the calls from `wifi_connect`, including the file path of each callee."

This requires following THREE tables: `Function` → `calls` → `Function` → `File`

```
SELECT
    f1.name AS caller,
    f2.name AS callee,
    c.call_site_line,
    fi.path AS callee_file
FROM Function f1                                -- Start with the caller
function
JOIN calls c ON f1.id = c.caller_id             -- Follow to the calls edge
table
JOIN Function f2 ON c.callee_id = f2.id         -- Follow to the callee
function
JOIN contains_func cf ON f2.id = cf.func_id     -- Follow to the file-function
edge
JOIN File fi ON cf.file_id = fi.id              -- Follow to the file
WHERE f1.name = 'wifi_connect';
```

In C, this would be:

```
Function *caller = find_function("wifi_connect");
for (int i = 0; i < call_edge_count; i++) {
    if (call_edges[i].caller_id == caller->id) {
        Function *callee = find_function_by_id(call_edges[i].callee_id);
        File *file = find_file_for_function(callee->id);
        printf("%s calls %s (line %d) in %s\n",
               caller->name, callee->name,
               call_edges[i].call_site_line, file->path);
    }
}
```

JOIN Types

```
-- INNER JOIN (default): Only rows that match in BOTH tables
SELECT * FROM Function fn JOIN File f ON fn.file_id = f.id;
-- If a Function has no matching File, it's excluded

-- LEFT JOIN: All rows from the LEFT table, even if no match in RIGHT
SELECT * FROM Function fn LEFT JOIN PriorityScore ps ON fn.id = ps.func_id;
-- If a Function has no score yet, it's still included (score columns are NULL)

-- Think of it like:
-- JOIN     = inner_join (only matches)
-- LEFT JOIN = left_outer_join (all from left + matches from right)
```

14.6 Subqueries — Queries Inside Queries

```
-- Find functions in files that have more than 50 functions
SELECT fn.name, fn.signature
FROM Function fn
```

```
WHERE fn.file_id IN (
    SELECT f.id
    FROM File f
    JOIN Function fn2 ON f.id = fn2.file_id
    GROUP BY f.id
    HAVING COUNT(*) > 50
);
-- The inner query finds file IDs with >50 functions
-- The outer query finds all functions in those files
```

14.7 Common Table Expressions (CTEs) — Named Subqueries

CTEs make complex queries readable by breaking them into named steps:

```
-- Without CTE (hard to read):
SELECT * FROM Function WHERE file_id IN (SELECT id FROM File WHERE language =
'c')
  AND id IN (SELECT func_id FROM PriorityScore WHERE composite_score > 0.5);

-- With CTE (clear and readable):
WITH c_files AS (
    SELECT id FROM File WHERE language = 'c'
),
high_priority AS (
    SELECT func_id FROM PriorityScore WHERE composite_score > 0.5
)
SELECT fn.*
FROM Function fn
JOIN c_files cf ON fn.file_id = cf.id
JOIN high_priority hp ON fn.id = hp.func_id;
```

14.8 Recursive CTEs — Graph Traversal in SQL

This is how we walk the call graph:

```
-- Find ALL functions reachable from wifi_connect (transitive closure)
WITH RECURSIVE reachable AS (
    -- BASE CASE: Start with wifi_connect itself
    SELECT f.id, f.name, 0 AS depth
    FROM Function f
    WHERE f.name = 'wifi_connect'

    UNION ALL

    -- RECURSIVE CASE: Find everything the current set calls
    SELECT f2.id, f2.name, r.depth + 1
    FROM reachable r
    JOIN calls c ON r.id = c.caller_id       -- follow call edges outward
    JOIN Function f2 ON c.callee_id = f2.id  -- get the callee
    WHERE r.depth < 10                        -- limit depth to prevent
infinite loops
```

```
)
SELECT DISTINCT name, depth FROM reachable ORDER BY depth;
```

How this works step-by-step:

```
Iteration 0 (base case):
  reachable = {wifi_connect}

Iteration 1 (recursive — what does wifi_connect call?):
  reachable = {wifi_connect, printf, hal_wifi_associate}

Iteration 2 (recursive — what do printf and hal_wifi_associate call?):
  reachable = {wifi_connect, printf, hal_wifi_associate,
               __write, hal_platform_send, nl80211_send_cmd}

Iteration 3:
  reachable = {..., send, ioctl, nl_send_auto}

... and so on until depth 10 or no new functions are found
```

This is equivalent to BFS (Breadth-First Search) on the call graph.

14.9 Views — Saved Queries

```
-- A VIEW is a saved query that acts like a virtual table
CREATE VIEW ComplexFunctions AS
    SELECT fn.*, f.path, f.language
    FROM Function fn
    JOIN File f ON fn.file_id = f.id
    WHERE fn.complexity > 10;

-- Now you can query it like a regular table:
SELECT * FROM ComplexFunctions WHERE language = 'c';
-- This is equivalent to running the full query above with an extra WHERE
```

Views don't store data — they just save the query. Every time you `SELECT` from a view, DuckDB runs the underlying query.

14.10 Indexes — Making Queries Fast

Without an index, `SELECT * FROM Function WHERE name = 'wifi_connect'` scans **every row** in the table (like `for(i=0; i<count; i++) if(strcmp(...))`).

With an index, it does a hash/tree lookup (like `hashtable_get(functions_by_name, "wifi_connect")`).

```
-- Create an index on the name column
CREATE INDEX idx_function_name ON Function (name);

-- Now this is O(log n) instead of O(n):
```

```
SELECT * FROM Function WHERE name = 'wifi_connect';

-- Composite index (for queries that filter on multiple columns)
CREATE INDEX idx_func_file_line ON Function (file_id, start_line);
-- Makes this fast:
SELECT * FROM Function WHERE file_id = 100 AND start_line BETWEEN 10 AND 50;
```

C analogy: An index is like a hash table or a sorted array with binary search. Creating an index takes extra memory and slows down inserts, but makes lookups orders of magnitude faster.

14.11 Transactions — Atomic Operations

```
-- A transaction groups multiple operations into an atomic unit
-- Either ALL of them succeed, or NONE of them do (like a mutex protecting a
critical section)

BEGIN TRANSACTION;
    INSERT INTO File (id, path, hash, language, loc) VALUES (100, 'wifi.c',
'abc', 'c', 500);
    INSERT INTO Function (id, file_id, name, ...) VALUES (1, 100,
'wifi_connect', ...);
    INSERT INTO Function (id, file_id, name, ...) VALUES (2, 100,
'wifi_disconnect', ...);
    INSERT INTO contains_func (file_id, func_id) VALUES (100, 1);
    INSERT INTO contains_func (file_id, func_id) VALUES (100, 2);
COMMIT;
-- If any INSERT fails, NONE of them are applied (ROLLBACK)
```

Why this matters: If we crash halfway through inserting a file's functions, we don't want the database to have the file record but no function records. Transactions prevent this inconsistency.

14.12 NULL — The Absence of Data

```
-- NULL means "no value" — not 0, not empty string, just "unknown"
-- It's like a NULL pointer in C, but safe

INSERT INTO Function (id, file_id, name, signature, start_line, end_line,
complexity)
VALUES (10, 100, 'unknown_func', NULL, 0, 0, NULL);
-- signature is NULL, complexity is NULL

-- You can't use = to compare with NULL (it's not a value, it's "unknown")
SELECT * FROM Function WHERE signature = NULL;      -- WRONG (always returns
empty)
SELECT * FROM Function WHERE signature IS NULL;      -- CORRECT
SELECT * FROM Function WHERE signature IS NOT NULL;   -- CORRECT

-- COALESCE: provide a default when NULL
SELECT name, COALESCE(complexity, 1) AS complexity FROM Function;
-- If complexity is NULL, returns 1 instead
```

14.13 DuckDB-Specific Features

Array Columns

```sql
-- DuckDB supports array columns (like C arrays as struct members)
CREATE TABLE Struct (
    id      BIGINT PRIMARY KEY,
    name    TEXT,
    members TEXT[]     -- array of strings
);

-- Insert array data
INSERT INTO Struct VALUES (1, 'wifi_config', ['char ssid[32]', 'int channel',
'enum band band']);

-- Query array elements
SELECT name, members[1] AS first_member FROM Struct;  -- arrays are 1-indexed
in SQL!
-- Returns: wifi_config | char ssid[32]

-- Unnest: expand array into separate rows
SELECT name, UNNEST(members) AS member FROM Struct;
-- Returns:
-- wifi_config | char ssid[32]
-- wifi_config | int channel
-- wifi_config | enum band band

-- Array contains check
SELECT * FROM Struct WHERE ARRAY_CONTAINS(members, 'int channel');
```

JSON Columns

```sql
CREATE TABLE IndexManifest (
    file_id       BIGINT PRIMARY KEY,
    manifest_json JSON
);

INSERT INTO IndexManifest VALUES (100, '{
    "file": {"path": "wifi.c", "language": "c"},
    "functions": [{"name": "wifi_connect"}, {"name": "wifi_disconnect"}]
}');

-- Query JSON fields
SELECT manifest_json->>'$.file.path' AS path FROM IndexManifest;
-- Returns: wifi.c

-- Query JSON arrays
SELECT manifest_json->'$.functions[0].name' AS first_func FROM IndexManifest;
-- Returns: "wifi_connect"
```

Executing SQL from Python

```python
import duckdb

# Open/create database
conn = duckdb.connect("index.duckdb")

# Execute DDL (no return value)
conn.execute("CREATE TABLE test (id INT, name TEXT)")

# Execute with parameters
conn.execute("INSERT INTO test VALUES (?, ?)", [1, "hello"])

# Fetch one row
row = conn.execute("SELECT * FROM test WHERE id = ?", [1]).fetchone()
# row = (1, "hello")
# Access: row[0] = 1, row[1] = "hello"

# Fetch all rows
rows = conn.execute("SELECT * FROM test").fetchall()
# rows = [(1, "hello"), ...]

# Fetch as pandas DataFrame (optional)
df = conn.execute("SELECT * FROM test").fetchdf()

# Get column names
conn.execute("SELECT * FROM test")
columns = [desc[0] for desc in conn.description]
# columns = ["id", "name"]

# Use context manager (auto-closes)
with duckdb.connect("index.duckdb") as conn:
    conn.execute("SELECT * FROM test")

# Execute multiple statements
conn.execute("""
    INSERT INTO test VALUES (2, 'world');
    INSERT INTO test VALUES (3, 'foo');
""")
```

---

15. Python for C Developers — Reading Our Code

You know C inside out. Python is what Code Crawler is written in. This chapter gives you enough Python to read and understand every file in our codebase.

15.1 The Basics — C vs Python Side by Side

Variables and Types

```c
// C: You declare types explicitly
int count = 0;
char *name = "wifi_connect";
```

```
float score = 0.548;
bool active = true;
int arr[5] = {1, 2, 3, 4, 5};
```

```
# Python: Types are inferred (but we add type hints for clarity)
count: int = 0
name: str = "wifi_connect"
score: float = 0.548
active: bool = True
arr: list[int] = [1, 2, 3, 4, 5]
```

Control Flow

```
// C
if (count > 5) {
    printf("high\n");
} else if (count > 0) {
    printf("low\n");
} else {
    printf("zero\n");
}

for (int i = 0; i < 10; i++) {
    if (items[i] == NULL) continue;
    process(items[i]);
}

while (running) {
    poll_events();
}
```

```
# Python (indentation replaces braces)
if count > 5:
    print("high")
elif count > 0:
    print("low")
else:
    print("zero")

for item in items:         # Iterate directly over collection (no index needed)
    if item is None:
        continue
    process(item)

for i in range(10):        # range(10) = 0, 1, 2, ..., 9
    process(items[i])

while running:
    poll_events()
```

Functions

```c
// C
int wifi_connect(const char *ssid, int timeout) {
    if (timeout <= 0) return -1;
    return hal_connect(ssid, timeout);
}
```

```python
# Python
def wifi_connect(ssid: str, timeout: int) -> int:
    """Connect to a WiFi network."""      # Docstring (like a comment but
accessible at runtime)
    if timeout <= 0:
        return -1
    return hal_connect(ssid, timeout)
```

Strings

```c
// C: strings are painful
char buf[256];
snprintf(buf, sizeof(buf), "Connected to %s (timeout=%d)", ssid, timeout);
if (strncmp(name, "wifi_", 5) == 0) { ... }
char *ext = strrchr(filename, '.');
```

```python
# Python: strings are easy
buf = f"Connected to {ssid} (timeout={timeout})"    # f-string formatting
if name.startswith("wifi_"):
    ...
ext = filename.rsplit(".", 1)[-1]                    # split from right, take
last part

# String methods you'll see in our code:
"HELLO".lower()                 # "hello"
"hello".upper()                 # "HELLO"
"  hello  ".strip()             # "hello" (trim whitespace)
"a/b/c".split("/")             # ["a", "b", "c"]
"/".join(["a", "b", "c"])      # "a/b/c"
"CONFIG_WIFI".startswith("CONFIG_")  # True
"wifi_hal.c".endswith(".c")          # True
"wifi" in "wifi_connect"             # True (substring check)
```

Data Structures

```c
// C: arrays, linked lists, hash tables (you implement yourself)
int arr[100];
```

```
struct node *list;
// For hash tables: use GLib GHashTable or roll your own
```

```
# Python: built-in rich data structures

# List (dynamic array)
functions = ["wifi_connect", "wifi_disconnect", "hal_init"]
functions.append("new_func")       # Add to end
first = functions[0]               # Index access: "wifi_connect"
last = functions[-1]               # Negative indexing: "new_func"
count = len(functions)             # Length: 4
subset = functions[1:3]            # Slice: ["wifi_disconnect", "hal_init"]

# Dictionary (hash table)
crawler_map = {
    "c": CCrawler(),
    "python": PyCrawler(),
    "shell": ShellCrawler(),
}
crawler = crawler_map.get("c")     # O(1) lookup: returns CCrawler instance
crawler_map["rust"] = RustCrawler()  # Add new entry

# Set (hash set — unique values only)
known_dirs = {"gcc", "glibc", "busybox"}
if "gcc" in known_dirs:            # O(1) membership check
    print("found")

# Tuple (immutable, like a frozen list)
point = (10, 20)                   # Can't be modified after creation
x, y = point                      # Unpack: x=10, y=20

# List comprehension (compact loop that builds a list)
names = [f.name for f in functions]                          # Like map()
complex_funcs = [f for f in functions if f.complexity > 5]   # Like filter()
```

15.2 Classes — C Structs With Methods

```
// C: struct + function pointers
typedef struct {
    char name[256];
    int (*parse)(struct Crawler *self, const char *file);
    const char **supported_langs;
    int lang_count;
} Crawler;

int c_crawler_parse(Crawler *self, const char *file) {
    printf("Parsing C file: %s\n", file);
    return 0;
}

Crawler c_crawler = {
    .name = "C Crawler",
```

```
    .parse = c_crawler_parse,
    .supported_langs = (const char*[]){"c", "cpp"},
    .lang_count = 2,
};
```

```
# Python: class
class CCrawler:
    """C/C++ parser using Tree-sitter."""

    def __init__(self):                      # Constructor (__init__ is like
C's init function)
        self.name = "C Crawler"              # Instance variable (like struct
member)
        self._private_cache = {}             # Convention: _ prefix means
"internal"

    @property                                # Property: accessed like a
variable, runs a function
    def supported_languages(self) -> list[str]:
        return ["c", "cpp"]

    def parse(self, file_info: FileInfo) -> ParseResult:  # Method (self = like
'this' pointer)
        print(f"Parsing C file: {file_info.path}")
        return ParseResult(file_info=file_info)

# Usage:
crawler = CCrawler()                         # Create instance
langs = crawler.supported_languages          # Property access (no parentheses)
result = crawler.parse(my_file_info)         # Method call
```

15.3 Dataclasses — Lightweight Structs

Dataclasses are Python's answer to C structs. They auto-generate `__init__`, `__repr__`, and comparison methods:

```
from dataclasses import dataclass, field

# This:
@dataclass(frozen=True)         # frozen=True makes it immutable (like const
struct)
class FunctionDef:
    name: str                   # Required field
    signature: str              # Required field
    start_line: int             # Required field
    end_line: int               # Required field
    complexity: int = 1         # Optional field with default value
    body_hash: str = ""         # Optional field with default value

# Is equivalent to this C:
# typedef struct {
#     const char *name;
```

```
#     const char *signature;
#     int start_line;
#     int end_line;
#     int complexity;    // default: 1
#     const char *body_hash;  // default: ""
# } FunctionDef;

# Usage:
func = FunctionDef(
    name="wifi_connect",
    signature="int wifi_connect(const char *ssid)",
    start_line=14,
    end_line=19,
    complexity=2,
)
# Access fields just like C struct members:
print(func.name)        # "wifi_connect"
print(func.complexity)  # 2

# frozen=True means you can't modify it (compile-time const equivalent):
# func.name = "new_name"   ← This raises an error!
```

The `field()` Function

```
@dataclass(frozen=True)
class ParseResult:
    file_info: FileInfo
    functions: list[FunctionDef] = field(default_factory=list)   # Default:
empty list
    structs: list[StructDef] = field(default_factory=list)
    # ...
```

Why `field(default_factory=list)` instead of just `= []`? Because in Python, `= []` would share the SAME list across all instances (like a `static` variable in C). `field(default_factory=list)` creates a NEW empty list for each instance.

15.4 Abstract Base Classes — Interfaces

```
from abc import ABC, abstractmethod

class BaseCrawler(ABC):           # ABC = Abstract Base Class (like a pure
virtual class in C++)

    @property
    @abstractmethod               # MUST be implemented by subclasses
    def supported_languages(self) -> list[str]:
        """Return list of supported language identifiers."""

    @abstractmethod
    def parse(self, file_info: FileInfo) -> ParseResult:
        """Parse a file and return structured results."""
```

```python
# You CANNOT instantiate BaseCrawler directly:
# crawler = BaseCrawler()    ← TypeError: Can't instantiate abstract class

# You MUST subclass and implement all abstract methods:
class CCrawler(BaseCrawler):
    @property
    def supported_languages(self) -> list[str]:
        return ["c", "cpp"]

    def parse(self, file_info: FileInfo) -> ParseResult:
        # actual implementation here
        ...
```

C analogy: This is like a vtable / interface pattern:

```c
// C equivalent of abstract base class
typedef struct {
    const char **(*get_supported_languages)(void);
    ParseResult (*parse)(FileInfo *file_info);
} BaseCrawlerVTable;

// C equivalent of concrete implementation
const char *c_languages[] = {"c", "cpp", NULL};
const char **c_get_langs(void) { return c_languages; }
ParseResult c_parse(FileInfo *fi) { ... }

BaseCrawlerVTable c_crawler_vtable = {
    .get_supported_languages = c_get_langs,
    .parse = c_parse,
};
```

15.5 Context Managers — Automatic Cleanup

```python
# The `with` statement ensures cleanup happens even if an error occurs
# Like: int fd = open(...); ... close(fd); but guaranteed to close

with duckdb.connect("index.duckdb") as conn:
    conn.execute("SELECT * FROM Function")
    # When this block exits (even via exception), conn is automatically closed

# Without 'with' (you have to remember to close):
conn = duckdb.connect("index.duckdb")
try:
    conn.execute("SELECT * FROM Function")
finally:
    conn.close()
```

C analogy: It's like RAII in C++ or `__attribute__((cleanup(func)))` in GCC:

```
// C equivalent
void __attribute__((cleanup(db_close))) *conn = db_open("index.duckdb");
// conn is automatically closed when it goes out of scope
```

15.6 Generators — Lazy Iterators

Our tree walker uses a generator:

```
def _walk(self, node):
    """Visit every node in the tree, lazily."""
    yield node                      # Return this node, then pause
    for child in node.children:     # When asked for the next item, continue
here
        yield from self._walk(child)  # Recursively yield all descendant nodes

# Usage:
for node in self._walk(root):
    if node.type == "function_definition":
        # process function
```

C analogy: A generator is like an iterator pattern. Instead of building a complete array of all nodes and returning it, it returns one node at a time, pausing between each one. This saves memory — we don't need to store the entire flattened tree.

```
// C equivalent would be a state machine:
typedef struct {
    TreeNode *stack[MAX_DEPTH];
    int stack_ptr;
} TreeWalker;

TreeNode *walk_next(TreeWalker *w) {
    if (w->stack_ptr == 0) return NULL;
    TreeNode *current = w->stack[--w->stack_ptr];
    for (int i = current->child_count - 1; i >= 0; i--) {
        w->stack[w->stack_ptr++] = current->children[i];
    }
    return current;
}
```

15.7 Exception Handling — Error Recovery

```
try:
    result = crawler.parse(file_info)
    event_bus.publish("file.parsed", result)
except FileNotFoundError:
    logger.warning("File not found: %s", file_info.path)
except UnicodeDecodeError:
    logger.warning("Binary file, skipping: %s", file_info.path)
```

```
except Exception:
    logger.exception("Unexpected error parsing %s", file_info.path)
    # logger.exception() includes the full stack trace (backtrace)
```

C analogy: This is like `errno` checking but structured:

```
// C equivalent (much more verbose)
result = parse_file(file_info);
if (result == NULL) {
    if (errno == ENOENT) {
        log_warning("File not found: %s", file_info->path);
    } else if (errno == EILSEQ) {
        log_warning("Binary file: %s", file_info->path);
    } else {
        log_error("Unexpected error: %s (errno=%d)", file_info->path, errno);
    }
}
```

15.8 Imports and Packages

```
# Our project structure:
# codecrawler/
# ├── __init__.py          ← Makes codecrawler a package
# ├── core/
# │   ├── __init__.py      ← Makes codecrawler.core a package
# │   ├── types.py
# │   └── event_bus.py
# └── crawlers/
#     ├── __init__.py
#     └── c_crawler.py

# In c_crawler.py, to use types from core:
from codecrawler.core.types import FileInfo, ParseResult, FunctionDef
from codecrawler.crawlers.base import BaseCrawler

# __init__.py controls what's "public" from a package:
# In codecrawler/core/__init__.py:
from codecrawler.core.types import FileInfo, ParseResult  # These are the
public API
```

15.9 The `pathlib.Path` Library — File Path Operations

You'll see `Path` everywhere in our code. It's Python's filesystem path library:

```
from pathlib import Path

# Create a path
p = Path("/home/dev/rdk/ccsp/wifi/wifi_hal.c")
```

```
# Properties
p.name        # "wifi_hal.c"
p.stem        # "wifi_hal" (name without extension)
p.suffix      # ".c"
p.parent      # Path("/home/dev/rdk/ccsp/wifi")
p.parts       # ("home", "dev", "rdk", "ccsp", "wifi", "wifi_hal.c")

# Operations
p.exists()                    # True if file exists
p.is_file()                   # True if it's a file
p.is_dir()                    # True if it's a directory
p.stat().st_size              # File size in bytes
p.read_text(encoding="utf-8") # Read entire file as string
p.read_bytes()                # Read entire file as bytes

# Joining paths (like snprintf with paths)
root = Path("/home/dev/rdk")
config = root / "build" / "conf" / "local.conf"   # Operator / joins paths
# config == Path("/home/dev/rdk/build/conf/local.conf")

# Globbing (like find + fnmatch)
for c_file in root.rglob("*.c"):     # Recursive glob: find all .c files
    print(c_file)

# Relative path
p.relative_to(root)    # Path("ccsp/wifi/wifi_hal.c")
```

15.10 Logging — Our Debug Print System

```
import logging

logger = logging.getLogger(__name__)    # Create a logger named after the
module

# Log levels (like syslog levels):
logger.debug("Parsing %s", file_path)     # LOG_DEBUG — verbose, for
development
logger.info("Found %d functions", count)  # LOG_INFO — normal operation
logger.warning("File not found: %s", path) # LOG_WARNING — recoverable issue
logger.error("Database query failed: %s", e) # LOG_ERR — operation failed
logger.exception("Crash: %s", e)          # LOG_ERR + stack trace

# In our code, you'll see:
logger.info("Discovered %d indexable files", len(self._discovered_files))
```

---

16. Tree-sitter Grammar Deep Dive

16.1 How Tree-sitter Grammar Rules Work

Tree-sitter uses a **formal grammar** to define how C source code maps to tree nodes. The grammar is written in JavaScript and compiled to a C parser.

Here's a simplified version of the C grammar rules:

```
// Simplified tree-sitter-c grammar.js
module.exports = grammar({
    name: 'c',

    rules: {
        // The entire file is a "translation_unit" containing declarations
        translation_unit: $ => repeat($._top_level_item),

        // A top-level item can be:
        _top_level_item: $ => choice(
            $.function_definition,     // int main() { ... }
            $.declaration,             // int g_count = 0;
            $.preproc_include,         // #include <stdio.h>
            $.preproc_def,             // #define MAX 100
            $.preproc_ifdef,           // #ifdef CONFIG_WIFI
            $.struct_specifier,        // struct foo { ... };
            $.typedef_declaration,     // typedef ... ;
        ),

        // A function definition has: type + declarator + body
        function_definition: $ => seq(
            field('type', $._type_specifier),         // "int"
            field('declarator', $._declarator),        // "main(int argc, char
*argv[])"
            field('body', $.compound_statement),        // "{ ... }"
        ),

        // A compound statement (function body) is: { statements... }
        compound_statement: $ => seq(
            '{',
            repeat($._statement),
            '}',
        ),

        // Statements include:
        _statement: $ => choice(
            $.expression_statement,    // foo();
            $.if_statement,           // if (...) { ... }
            $.while_statement,        // while (...) { ... }
            $.for_statement,          // for (...;...;...) { ... }
            $.return_statement,       // return x;
            $.compound_statement,     // { nested block }
            $.declaration,            // int x = 5; (local variable)
        ),

        // A call expression: function_name(arguments)
        call_expression: $ => seq(
            field('function', $._expression),
            field('arguments', $.argument_list),
        ),
```

```
    }
});
```

What this means for parsing

When tree-sitter sees:

```
int wifi_connect(const char *ssid) {
    printf("hello\n");
    return 0;
}
```

It matches the rules:

1. `translation_unit` → contains a `function_definition`
2. `function_definition` → `type:int` + `declarator:wifi_connect(const char *ssid)` + `body:{...}`
3. Inside `body` (compound_statement): two statements
4. First statement: `expression_statement` → `call_expression` → `function:printf` + `arguments:("hello\n")`
5. Second statement: `return_statement` → value `0`

16.2 The S-Expression Representation

Tree-sitter can output the tree as an S-expression (a Lisp-like text format). This is extremely useful for understanding what the parser sees:

```
tree = parser.parse(source.encode())
print(tree.root_node.sexp())
```

For our `wifi_connect` example:

```
(translation_unit
  (function_definition
    type: (primitive_type)                  ; "int"
    declarator: (function_declarator
      declarator: (identifier)              ; "wifi_connect"
      parameters: (parameter_list
        (parameter_declaration
          type: (type_qualifier)            ; "const"
          type: (primitive_type)            ; "char"
          declarator: (pointer_declarator
            declarator: (identifier)))))    ; "ssid"
    body: (compound_statement
      (expression_statement
        (call_expression
          function: (identifier)            ; "printf"
          arguments: (argument_list
            (string_literal
```

```
                (string_content)))))            ; "hello\n"
        (return_statement
          (number_literal)))))                  ; 0
```

How to read S-expressions

- `(node_type ...)` = a tree node with children inside
- `field_name: (node_type ...)` = a named child
- Just `(identifier)` at a leaf = a leaf node you can extract text from

16.3 All C Node Types We Care About

Node Type	What it represents	How we use it
`translation_unit`	The entire file	Root node — start of our tree walk
`function_definition`	`int foo(...) { ... }`	Extract FunctionDef
`function_declarator`	`foo(int x, int y)`	Get function name and params
`identifier`	Any name: `foo`, `count`, `MAX`	Extract actual text
`call_expression`	`foo(a, b)`	Extract CallEdge
`struct_specifier`	`struct foo { ... }`	Extract StructDef
`field_declaration`	`int x;` inside struct	Extract struct members
`declaration`	`int count = 0;`	Extract VariableDef (if global scope)
`parameter_declaration`	`const char *ssid`	Part of function signature
`preproc_include`	`#include <stdio.h>`	Extract IncludeEdge
`preproc_def`	`#define MAX 100`	Extract MacroDef
`preproc_ifdef`	`#ifdef CONFIG_X`	Detect build guards
`if_statement`	`if (...) { ... }`	Increment complexity
`while_statement`	`while (...) { ... }`	Increment complexity
`for_statement`	`for (...) { ... }`	Increment complexity
`switch_statement`	`switch (...) { ... }`	Increment complexity
`case_statement`	`case X:`	Increment complexity
`conditional_expression`	`x ? a : b`	Increment complexity
`binary_expression`	`a && b`, `a || b`	Increment complexity (for && and
`compound_statement`	`{ ... }`	Function body
`return_statement`	`return x;`	Track return paths
`pointer_declarator`	`*ptr`	Detect pointer types
`type_qualifier`	`const`, `volatile`, `static`	Detect storage class

16.4 Walking Complex C Code — Step by Step

Let's parse a realistic embedded C function:

```c
#ifdef CONFIG_WIFI
static int wifi_scan_handler(struct nl_msg *msg, void *arg)
{
    struct wifi_scan_results *results = arg;
    struct nlattr *tb[NL80211_ATTR_MAX + 1];
    struct genlmsghdr *gnlh = nlmsg_data(nlmsg_hdr(msg));

    nla_parse(tb, NL80211_ATTR_MAX, genlmsg_attrdata(gnlh, 0),
              genlmsg_attrlen(gnlh, 0), NULL);

    if (!tb[NL80211_ATTR_BSS]) {
        fprintf(stderr, "BSS info missing\n");
        return NL_SKIP;
    }

    results->count++;
    return NL_OK;
}
#endif
```

Tree-sitter produces (simplified):

```
translation_unit
└── preproc_ifdef
    ├── name: "CONFIG_WIFI"
    └── function_definition
        ├── storage_class: "static"
        ├── type: "int"
        ├── declarator: function_declarator
        │   ├── declarator: "wifi_scan_handler"
        │   └── parameters: parameter_list
        │       ├── parameter: "struct nl_msg *msg"
        │       └── parameter: "void *arg"
        └── body: compound_statement
            ├── declaration: "struct wifi_scan_results *results = arg;"
            ├── declaration: "struct nlattr *tb[...];"
            ├── declaration: "struct genlmsghdr *gnlh = ...;"
            ├── expression_statement: call_expression("nla_parse", ...)
            ├── if_statement
            │   ├── condition: unary_expression("!tb[NL80211_ATTR_BSS]")
            │   └── consequence: compound_statement
            │       ├── expression_statement: call_expression("fprintf",
stderr, ...)
            │       └── return_statement: "NL_SKIP"
            ├── expression_statement: update_expression("results->count++")
            └── return_statement: "NL_OK"
```

Our tree walker would extract:

```
# From the function_definition node:
FunctionDef(
    name="wifi_scan_handler",
    signature="static int wifi_scan_handler(struct nl_msg *msg, void *arg)",
    start_line=2,
    end_line=16,
    complexity=2,  # Base 1 + 1 for the if statement
)

# From the call_expression nodes:
CallEdge(caller="", callee="nla_parse", call_site_line=8)
CallEdge(caller="", callee="fprintf", call_site_line=12)
# Note: nlmsg_data, nlmsg_hdr, etc. are also calls but nested in the argument

# From the preproc_ifdef:
MacroDef(name="CONFIG_WIFI", is_config_guard=True)

# The function is also tagged as guarded_by CONFIG_WIFI
```

16.5 Cyclomatic Complexity — The Counting Rules

Cyclomatic complexity measures "how many independent paths through a function." It correlates with how hard the function is to test and understand.

Formula: `complexity = 1 + (number of decision points)`

C Construct	Adds to complexity	Why
Function entry	+1 (base)	There's always at least one path
`if (...)`	+1	Two paths: true or false
`else if (...)`	+1	Another branch
`while (...)`	+1	Loop or skip: two paths
`for (...)`	+1	Loop or skip: two paths
`case X:` (in switch)	+1	Each case is a branch
`? :` (ternary)	+1	Two paths in an expression
`&&`	+1	Short-circuit creates a hidden branch
`||`	+1	Short-circuit creates a hidden branch
`catch (...)`	+1	Exception path

Examples:

```c
// Complexity = 1 (no decisions)
void simple(void) {
    printf("hello\n");
}
```

```c
// Complexity = 2 (1 base + 1 if)
int check(int x) {
    if (x > 0)
        return 1;
    return 0;
}

// Complexity = 4 (1 base + 1 if + 1 else-if + 1 &&)
int complex(int x, int y) {
    if (x > 0 && y > 0)           // +1 if, +1 &&
        return 1;
    else if (x < 0)               // +1 else-if
        return -1;
    return 0;
}

// Complexity = 8 (our wifi_scan_handler example might be around this)
// Rule of thumb: complexity > 10 = function needs refactoring
```

16.6 Handling Tree-sitter Errors and Edge Cases

Tree-sitter is **fault-tolerant** — even if the code has syntax errors, it still produces a tree. Error nodes appear as `(ERROR ...)` in the tree:

```c
// Broken C code:
int wifi_connect(const char *ssid {  // Missing closing parenthesis
    return 0;
}
```

Tree-sitter output:

```
(translation_unit
  (function_definition
    type: (primitive_type)
    declarator: (function_declarator
      declarator: (identifier)
      parameters: (parameter_list
        (parameter_declaration
          type: (type_qualifier)
          type: (primitive_type)
          declarator: (pointer_declarator
            declarator: (identifier)))
        (ERROR "{")                    ; <-- Error node for the syntax error
        ))
    body: (compound_statement
      (return_statement
        (number_literal)))))
```

The parser still found the function name, its parameter, and the return statement. It just flagged the `{` as an error because it appeared where a `)` was expected.

Our code handles this gracefully — we simply skip error nodes and extract whatever valid information we can:

```
def _extract_function(self, node, source):
    declarator = node.child_by_field_name("declarator")
    if not declarator:          # If the tree is too broken to find a
declarator
        return None             # Skip this function, don't crash
    # ... rest of extraction
```

---

17. Graph Theory & Algorithms for Code Analysis

17.1 What Is a Graph? (The Math Definition)

A graph is a set of **nodes** (vertices) and **edges** (connections between nodes).

```
G = (V, E)
where:
  V = set of nodes   = {A, B, C, D, E}
  E = set of edges   = {(A,B), (B,C), (B,D), (C,E), (D,E)}
```

Visual:

```
A ──→ B ──→ C
      │      │
      ▼      ▼
      D ──→ E
```

In Code Crawler, nodes are code entities (functions, files, structs) and edges are relationships (calls, includes, uses).

17.2 Directed vs Undirected Graphs

Directed graph (what we use): edges have direction. `A → B` means "A calls B" — not the same as "B calls A."

```
wifi_connect → printf         (wifi_connect calls printf)
wifi_connect → hal_associate  (wifi_connect calls hal_associate)
main → wifi_connect           (main calls wifi_connect)
```

Undirected graph: edges go both ways. "A is in the same file as B" (symmetric relationship).

17.3 Graph Terminology

Term	Definition	Code Crawler Example
Node/Vertex	An entity in the graph	A function, file, struct

Term	Definition	Code Crawler Example
Edge	A connection between two nodes	A function call, an include
In-degree	Number of edges pointing INTO a node	How many functions call THIS function
Out-degree	Number of edges pointing OUT of a node	How many functions THIS function calls
Path	A sequence of edges from node A to node B	The chain: main → wifi_connect → hal_associate
Cycle	A path that leads back to the starting node	A → B → C → A (recursive calls)
DAG	Directed Acyclic Graph (no cycles)	Include graph (if no circular includes)
Connected component	A set of nodes where every node can reach every other	A module/subsystem
Adjacency	Two nodes directly connected by an edge	Direct caller/callee
Transitive closure	All nodes reachable via any path	All functions in the call chain
Strongly connected component	A set of nodes where every node can reach every other (directed)	Mutual recursion groups

17.4 Depth-First Search (DFS)

What: Start at a node, go as deep as possible along each branch before backtracking.

C analogy: Like walking a directory tree depth-first:

```
void walk_dir(const char *path) {
    DIR *d = opendir(path);
    struct dirent *entry;
    while ((entry = readdir(d)) != NULL) {
        if (entry->d_type == DT_DIR) {
            walk_dir(full_path);  // Go deeper FIRST
        } else {
            process_file(entry);
        }
    }
    closedir(d);
}
```

In Code Crawler, DFS is used for:

- Tree-sitter AST walking (visit node, then visit all children depth-first)
- Finding all functions reachable from a given function
- Detecting cycles in the call graph

Implementation:

```python
def dfs(graph, start_node, visited=None):
    """Visit all nodes reachable from start_node, depth-first."""
    if visited is None:
        visited = set()

    visited.add(start_node)
    print(f"Visiting: {start_node}")

    for neighbor in graph[start_node]:    # For each function this one calls
        if neighbor not in visited:        # If we haven't visited it yet
            dfs(graph, neighbor, visited)  # Go deeper

    return visited

# Example call graph:
call_graph = {
    "main": ["wifi_connect", "init_logging"],
    "wifi_connect": ["printf", "hal_associate"],
    "init_logging": ["syslog_init"],
    "printf": [],
    "hal_associate": ["nl80211_send"],
    "syslog_init": [],
    "nl80211_send": ["ioctl"],
    "ioctl": [],
}

# DFS from main:
dfs(call_graph, "main")
# Output: main → wifi_connect → printf → hal_associate → nl80211_send → ioctl →
init_logging → syslog_init
```

17.5 Breadth-First Search (BFS)

What: Start at a node, visit all neighbors first, then their neighbors, etc. (level by level).

Why we use it: BFS gives us the **shortest path** — the minimum number of hops between two functions. This is what our recursive CTE does in SQL.

```python
from collections import deque

def bfs(graph, start_node, max_depth=10):
    """Visit all nodes reachable from start_node, level by level."""
    visited = {start_node: 0}  # node → depth
    queue = deque([start_node])

    while queue:
        current = queue.popleft()
        depth = visited[current]

        if depth >= max_depth:
            continue

        for neighbor in graph.get(current, []):
```

```
            if neighbor not in visited:
                visited[neighbor] = depth + 1
                queue.append(neighbor)

    return visited

# BFS from main:
result = bfs(call_graph, "main")
# {main: 0, wifi_connect: 1, init_logging: 1, printf: 2, hal_associate: 2,
#  syslog_init: 2, nl80211_send: 3, ioctl: 4}

# This tells us:
# - wifi_connect is 1 hop from main
# - ioctl is 4 hops from main
```

17.6 Betweenness Centrality — The Key Metric

What it measures: How often a node appears on the shortest paths between all pairs of other nodes.

Intuition: A function with high betweenness centrality is a **BRIDGE** — many communication paths go through it. If you remove it, parts of the codebase become disconnected.

Formula:

```
                    σ(s, t | v)
C_B(v) = Σ    ──────────────────
         s≠v≠t    σ(s, t)

Where:
  σ(s, t)     = total number of shortest paths from s to t
  σ(s, t | v) = number of those paths that pass through v
```

Example:

```
    A ──→ B ──→ C
    │           ↑
    │           │
    └──→ D ──→ E ──→ F
```

Paths and which nodes they pass through:

From	To	Shortest Path	Passes through
A	C	A→B→C	B
A	E	A→D→E	D
A	F	A→D→E→F	D, E
B	F	B→C (no path to F)	—
D	C	D→E→C	E

Node B: only on the A→C path. Centrality = low. Node D: on A→E, A→F paths. Centrality = medium. Node E: on A→F, D→C paths. Centrality = medium-high.

Why this matters for Code Crawler:

- A HAL function that bridges 5 apps and 3 drivers has very high betweenness
- Changing it affects all those paths
- Our priority scorer uses this to identify critical infrastructure functions

Simplified algorithm (Brandes' algorithm):

```
def betweenness_centrality(graph, nodes):
    """Compute betweenness centrality for all nodes."""
    centrality = {v: 0.0 for v in nodes}

    for source in nodes:
        # BFS from source
        stack = []
        predecessors = {v: [] for v in nodes}
        num_shortest_paths = {v: 0 for v in nodes}
        num_shortest_paths[source] = 1
        distance = {v: -1 for v in nodes}
        distance[source] = 0
        queue = deque([source])

        while queue:
            v = queue.popleft()
            stack.append(v)
            for w in graph.get(v, []):
                if distance[w] < 0:  # First visit
                    queue.append(w)
                    distance[w] = distance[v] + 1
                if distance[w] == distance[v] + 1:  # Shortest path
                    num_shortest_paths[w] += num_shortest_paths[v]
                    predecessors[w].append(v)

        # Back-propagation of centrality
        dependency = {v: 0.0 for v in nodes}
        while stack:
            w = stack.pop()
            for v in predecessors[w]:
                ratio = num_shortest_paths[v] / num_shortest_paths[w]
                dependency[v] += ratio * (1 + dependency[w])
            if w != source:
                centrality[w] += dependency[w]

    # Normalize
    n = len(nodes)
    if n > 2:
        for v in centrality:
            centrality[v] /= ((n-1) * (n-2))

    return centrality
```

17.7 Connected Components — Finding Subsystems

A connected component is a group of functions that can all reach each other through call chains. This naturally identifies **subsystems** in your codebase.

```
Component 1 (WiFi subsystem):
  wifi_connect → hal_associate → nl80211_send → ioctl
  wifi_disconnect → hal_disassociate → nl80211_send → ioctl

Component 2 (Logging subsystem):
  init_logging → syslog_init → openlog
  log_message → syslog → write

Component 3 (Isolated utility):
  crc32_compute  (no outgoing or incoming calls from other components)
```

This is useful for:

- Understanding module boundaries
- Detecting unexpected coupling between subsystems
- Impact analysis ("if I change the WiFi subsystem, does the logging subsystem care?")

17.8 Topological Sort — Build Order

A topological sort orders nodes so that every edge points "forward" — no node appears before its dependencies.

C analogy: This is exactly what `make` does. If `main.o` depends on `wifi.o` which depends on `hal.o`, the build order is: `hal.o → wifi.o → main.o`.

```
Include graph:
  main.c → wifi.h → hal.h → types.h
  main.c → util.h → types.h

Topological sort:
  types.h → hal.h → wifi.h → util.h → main.c
  (types.h comes first because everything depends on it)
```

In Code Crawler, topological sort helps determine:

- The processing order for files (parse dependencies first)
- The include chain for header resolution
- Initialization order for services

---

18. Vector Mathematics & Embeddings Deep Dive

18.1 What Is a Vector? (From First Principles)

A vector is just a list of numbers. In C, it's literally an array:

```
float embedding[384] = {0.23, -0.41, 0.87, ..., 0.12};
```

In our context, each number represents a "dimension" of meaning. Imagine a simplified 3D example:

```
Dimension 0: "networking-ness" (how much is this about networking)
Dimension 1: "hardware-ness"  (how much is this about hardware)
Dimension 2: "string-ness"    (how much is this about string manipulation)

wifi_connect  = [0.95, 0.60, 0.10]   (very networking, somewhat hardware, not
strings)
hal_send_cmd  = [0.70, 0.90, 0.05]   (networking + hardware, not strings)
printf        = [0.05, 0.05, 0.95]   (not networking, not hardware, very
strings)
```

Real embeddings have 384 dimensions instead of 3 — enough to capture the full nuance of meaning.

18.2 How Embedding Models Work

The embedding model (`all-MiniLM-L6-v2`) is a neural network trained on millions of text pairs:

```
Training data:
  ("connect to wireless network", "join wifi access point") → SIMILAR
  ("connect to wireless network", "parse HTML document")    → DIFFERENT
```

The model learned to map semantically similar text to nearby vectors:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# Encode text into a 384-dimensional vector
v1 = model.encode("connect to wireless network")
v2 = model.encode("join wifi access point")
v3 = model.encode("parse HTML document")

# v1 and v2 are close together, v3 is far away
```

What we feed to the model

For each function, we create a descriptive text:

```
text = f"{func.name}: {func.signature}"
# "wifi_connect: int wifi_connect(const char *ssid, int timeout)"

# If we have a summary (from the LLM summarizer):
text += f" — {func.summary}"
# "wifi_connect: int wifi_connect(...) — Establishes WiFi connection to the
```

```
specified SSID"

embedding = model.encode(text)
# Returns: np.array of shape (384,)
```

18.3 Cosine Similarity — The Distance Metric

The Math

Given two vectors A and B of dimension n:

```
                    A · B                    Σ(A[i] × B[i])
cos(θ) = ──────────────────────── = ──────────────────────────────────
              ||A|| × ||B||            √(Σ A[i]²) × √(Σ B[i]²)
```

Step by step with real numbers:

```
A = [0.5, 0.3, 0.8]   (wifi_connect embedding, simplified to 3D)
B = [0.4, 0.2, 0.7]   (wlan_associate embedding)
C = [0.9, 0.1, 0.0]   (print_banner embedding)

Cosine similarity A·B:
  Dot product:  (0.5×0.4) + (0.3×0.2) + (0.8×0.7) = 0.20 + 0.06 + 0.56 = 0.82
  Magnitude A:  √(0.25 + 0.09 + 0.64) = √0.98 = 0.990
  Magnitude B:  √(0.16 + 0.04 + 0.49) = √0.69 = 0.831

  cos(θ) = 0.82 / (0.990 × 0.831) = 0.82 / 0.822 = 0.998
  → Very similar! (close to 1.0)

Cosine similarity A·C:
  Dot product:  (0.5×0.9) + (0.3×0.1) + (0.8×0.0) = 0.45 + 0.03 + 0.00 = 0.48
  Magnitude A:  0.990 (same as before)
  Magnitude C:  √(0.81 + 0.01 + 0.00) = √0.82 = 0.906

  cos(θ) = 0.48 / (0.990 × 0.906) = 0.48 / 0.897 = 0.535
  → Somewhat different (farther from 1.0)
```

C implementation (for reference):

```c
float cosine_similarity(const float *a, const float *b, int n) {
    float dot = 0.0f, mag_a = 0.0f, mag_b = 0.0f;
    for (int i = 0; i < n; i++) {
        dot   += a[i] * b[i];
        mag_a += a[i] * a[i];
        mag_b += b[i] * b[i];
    }
    return dot / (sqrtf(mag_a) * sqrtf(mag_b));
}

// Usage:
```

```
float sim = cosine_similarity(wifi_connect_embedding, wlan_associate_embedding,
384);
// sim ≈ 0.95 (very similar)
```

Why Cosine, Not Euclidean Distance?

Euclidean distance measures **absolute position**: `d = √(Σ(A[i] - B[i])²)`

Cosine similarity measures **direction**: do the vectors point the same way?

Two functions can have different "magnitudes" (e.g., different code lengths) but still be semantically similar. Cosine ignores magnitude and focuses on direction.

18.4 HNSW — The Data Structure Behind Fast Vector Search

Problem: Given 100,000 function embeddings, finding the 10 most similar to a query takes:

- **Brute force**: Compare query with all 100,000 vectors → 100,000 × 384 multiplications = slow
- **HNSW index**: Compare query with ~log(100,000) ≈ 50 vectors → fast!

HNSW stands for **Hierarchical Navigable Small World** graph. It's like a multi-level skip list for vectors:

```
Level 3 (sparse):    A ──────────────────── E
                     |                      |
Level 2 (medium):    A ─── C ─── E ─── G
                     |     |     |     |
Level 1 (dense):     A — B — C — D — E — F — G — H
                     |   |   |   |   |   |   |   |
Level 0 (all nodes): A B C D E F G H I J K L M N O P
```

How search works:

1. Start at the top level with few, far-apart nodes
2. Greedily move to the nearest neighbor at each level
3. Drop down a level and repeat
4. At the bottom level, you're in the neighborhood of the answer

This is why DuckDB's VSS extension creates an HNSW index — it makes vector search logarithmic instead of linear.

```
-- Without HNSW: scans all 100,000 vectors (O(n))
SELECT * FROM Function
WHERE embedding IS NOT NULL
ORDER BY array_cosine_distance(embedding, ?::FLOAT[384]) ASC
LIMIT 10;
-- Time: ~500ms

-- With HNSW index: navigates the graph (O(log n))
CREATE INDEX func_emb ON Function USING HNSW (embedding) WITH (metric =
'cosine');
-- Same query, but now: ~5ms
```

18.5 Dimensionality — Why 384?

The model `all-MiniLM-L6-v2` produces 384-dimensional vectors because:

```
Input text → Tokenizer → 256 tokens max → Transformer (6 layers) → 384-dim
output
```

- **Too few dimensions** (e.g., 32): Can't capture enough meaning. "wifi_connect" and "bluetooth_connect" would be too similar.
- **Too many dimensions** (e.g., 4096): Wastes space, slower search, diminishing returns.
- **384**: Sweet spot for code — enough resolution to distinguish "network connect" from "filesystem open" from "memory allocate".

Storage cost per entity:

```
384 dimensions × 4 bytes per float = 1,536 bytes per embedding
× 100,000 functions = 146 MB total for vector storage
```

That's small enough for embedded systems and fast enough for real-time search.

18.6 Practical Example: Semantic Search End-to-End

```python
# ═══ AT INDEX TIME ═══

# 1. Parse wifi_hal.c → extract functions
# 2. For each function, create descriptive text:
texts = [
    "wifi_connect: int wifi_connect(const char *ssid, int timeout) — Connects
to WiFi network",
    "wifi_disconnect: void wifi_disconnect(void) — Disconnects from current
WiFi network",
    "wifi_scan: int wifi_scan(struct scan_results *results) — Scans for
available networks",
]

# 3. Encode all at once (batched for efficiency)
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
embeddings = model.encode(texts)   # shape: (3, 384)

# 4. Store in database
for i, emb in enumerate(embeddings):
    conn.execute("UPDATE Function SET embedding = ? WHERE id = ?",
[emb.tolist(), func_ids[i]])

# 5. Create HNSW index
conn.execute("CREATE INDEX func_emb_idx ON Function USING HNSW (embedding) WITH
(metric = 'cosine')")


# ═══ AT QUERY TIME (MCP tool call) ═══
```

```python
# 1. AI agent sends: search_code("how to connect to wireless network")
query_text = "how to connect to wireless network"

# 2. Encode the query
query_embedding = model.encode(query_text)  # shape: (384,)

# 3. DuckDB vector search
results = conn.execute("""
    SELECT name, signature, summary,
           array_cosine_distance(embedding, ?::FLOAT[384]) AS distance
    FROM Function
    WHERE embedding IS NOT NULL
    ORDER BY distance ASC
    LIMIT 5
""", [query_embedding.tolist()]).fetchall()

# 4. Results (distance = 0 means identical, higher = more different):
# name           | distance | Why matched
# wifi_connect   | 0.12     | "connect" + "wireless" are very close to the
query
# wlan_associate | 0.18     | "associate" is semantically similar to "connect"
# wifi_scan      | 0.35     | "wireless" matches but "scan" ≠ "connect"
# bt_connect     | 0.42     | "connect" matches but "bluetooth" ≠ "wireless"
# printf         | 0.89     | Completely unrelated
```

---

19. Design Patterns in Code Crawler

Every software pattern we use has an analogy in embedded C. This chapter maps our patterns to concepts you already know.

19.1 Observer Pattern (Event Bus)

What: Objects subscribe to events and get notified when they occur.

C analogy: You already use this! **Netlink sockets** and **D-Bus signals** are observer patterns:

```c
// C: Subscribing to Netlink events
struct nl_cb *cb = nl_cb_alloc(NL_CB_DEFAULT);
nl_cb_set(cb, NL_CB_VALID, NL_CB_CUSTOM, wifi_scan_handler, NULL);
// When a valid Netlink message arrives, wifi_scan_handler is called

// Same thing, but in Python with our EventBus:
event_bus.subscribe("file.parsed", storage.handle_parsed_file)
# When a "file.parsed" event is published, handle_parsed_file is called
```

Advantages:

- **Loose coupling**: The publisher doesn't know who's listening
- **Extensibility**: Add new listeners without modifying the publisher

- **Testability**: Mock the event bus in tests

Disadvantages:

- **Debugging**: Events can be hard to trace (no direct function call to follow)
- **Ordering**: Handler execution order isn't guaranteed

19.2 Strategy Pattern (Pluggable Parsers)

What: Define a family of algorithms (parsers), make them interchangeable.

C analogy: Function pointers in a struct — like how Linux drivers work:

```
// C: Linux driver model (strategy pattern)
struct file_operations wifi_fops = {
    .open    = wifi_open,
    .read    = wifi_read,
    .write   = wifi_write,
    .ioctl   = wifi_ioctl,
    .release = wifi_close,
};
// The kernel calls fops->open() without knowing which driver implements it

// Python: Our crawler pattern (same concept)
class BaseCrawler(ABC):
    def parse(self, file_info): ...    # Abstract "strategy"

class CCrawler(BaseCrawler):
    def parse(self, file_info): ...    # C-specific strategy

class PyCrawler(BaseCrawler):
    def parse(self, file_info): ...    # Python-specific strategy

# The pipeline calls crawler.parse() without knowing which language
```

19.3 Registry Pattern (Service Locator)

What: A central registry where components register their services and other components look them up.

C analogy: Like the Linux kernel's platform device/driver matching:

```
// C: Kernel driver registration
static struct platform_driver wifi_driver = {
    .probe  = wifi_probe,
    .remove = wifi_remove,
    .driver = { .name = "qcom-wifi" },
};
platform_driver_register(&wifi_driver);
// The kernel platform bus now knows about this driver
// When a matching device appears, it calls wifi_probe()

// Python: Our ServiceRegistry
registry = ServiceRegistry()
```

```
registry.register(BaseCrawler, CCrawler())
# The pipeline now knows about this crawler
# When a .c file needs parsing, it retrieves and calls CCrawler.parse()
```

19.4 Pipeline Pattern (Staged Processing)

What: Data flows through a sequence of processing stages, each transforming it.

C analogy: A signal processing chain:

```
// C: Audio processing pipeline
raw_samples = read_adc();
filtered    = apply_lowpass(raw_samples);
amplified   = apply_gain(filtered, 2.0);
output      = apply_compressor(amplified);
write_dac(output);

// Python: Our indexing pipeline
files       = discover_files(project_root)           # Stage 1
build_type  = detect_build_system(project_root)      # Stage 2
tiered      = classify_tiers(files)                  # Stage 3
parsed      = [crawler.parse(f) for f in tiered]     # Stage 4
scored      = [scorer.score(f) for f in parsed]      # Stage 5
manifests   = [builder.build(p) for p in parsed]     # Stage 6
```

19.5 Data Transfer Object (DTO) Pattern

What: Immutable data containers that carry data between components without behavior.

C analogy: `const struct` used as a message:

```
// C: Netlink message (a DTO)
struct nl_msg {
    struct nlmsghdr hdr;
    unsigned char data[];
};
// Created by producer, consumed by consumer, never modified in transit

// Python: Our DTOs
@dataclass(frozen=True)  # frozen = immutable
class ParseResult:
    file_info: FileInfo
    functions: list[FunctionDef]
    # ...
# Created by crawler, consumed by storage and tiering, never modified
```

Why immutable?

- Thread safety: multiple components can read the same DTO simultaneously
- Correctness: prevents accidental mutation by a consumer

- Debugging: if you see a DTO, you know it hasn't been changed since creation

19.6 Plugin/Extension Pattern

What: Allow new functionality to be added without modifying existing code.

C analogy: Linux kernel modules:

```
// C: Kernel module (a plugin)
static int __init wifi_module_init(void) {
    platform_driver_register(&wifi_driver);
    return 0;
}
static void __exit wifi_module_exit(void) {
    platform_driver_unregister(&wifi_driver);
}
module_init(wifi_module_init);    // Register lifecycle hooks
module_exit(wifi_module_exit);    // Cleanup lifecycle hooks

// Python: Our plugin system
class CCrawlerPlugin(PluginBase):
    def register(self, registry):          # Like module_init
        registry.register(BaseCrawler, CCrawler())

    def activate(self, event_bus):         # Subscribe to events
        pass

    def deactivate(self):                  # Like module_exit
        pass
```

19.7 Facade Pattern (MCP Server)

What: Provide a simple, unified interface to a complex subsystem.

C analogy: HAL (Hardware Abstraction Layer):

```
// C: WiFi HAL hides complexity
int wifi_connect(const char *ssid, int timeout);
// Behind this simple API:
//   - nl80211 socket creation
//   - NL_CMD_CONNECT message construction
//   - Netlink send/receive
//   - Error handling
//   - Timeout management
//   - State machine updates

// Python: MCP server hides complexity
@mcp_tool("search_code")
def search_code(query: str):
    # Behind this simple API:
    #   - Text embedding generation
    #   - HNSW vector search
    #   - Priority score filtering
```

```
    #   - Manifest retrieval
    #   - Result ranking and formatting
```

19.8 Dependency Injection

What: Instead of creating dependencies internally, receive them from the outside.

```
# BAD: Hard-coded dependency (tight coupling)
class Pipeline:
    def __init__(self):
        self.database = DuckDBDatabase("index.duckdb")  # Hardcoded!
        self.crawler = CCrawler()                        # Hardcoded!

# GOOD: Injected dependencies (loose coupling)
class Pipeline:
    def __init__(self, registry: ServiceRegistry, bus: EventBus):
        self.registry = registry  # Get crawlers from registry
        self.bus = bus            # Communicate via event bus
```

C analogy: Passing function pointers instead of calling functions directly:

```
// BAD: Direct call (tight coupling)
void process(int data) {
    wifi_send(data);  // Hardcoded to wifi
}

// GOOD: Function pointer (dependency injection)
typedef int (*send_fn)(int data);
void process(int data, send_fn send) {
    send(data);  // Could be wifi_send, bt_send, or uart_send
}
```

19.9 Builder Pattern (Manifest Builder)

What: Construct a complex object step by step.

```
# Our ManifestBuilder constructs an IndexManifest step by step:
class ManifestBuilder:
    def build(self, parse_result: ParseResult) -> IndexManifestBundle:
        manifest = {}

        # Step 1: Add file metadata
        manifest["file"] = {
            "path": str(parse_result.file_info.path),
            "language": parse_result.file_info.language,
            "loc": count_lines(parse_result),
        }

        # Step 2: Add function signatures (not bodies — too large)
```

```
        manifest["functions"] = [
            {"name": f.name, "sig": f.signature, "complexity": f.complexity}
            for f in parse_result.functions
        ]

        # Step 3: Add struct names
        manifest["structs"] = [s.name for s in parse_result.structs]

        # Step 4: Add call edges (the graph)
        manifest["calls"] = [
            {"from": c.caller, "to": c.callee}
            for c in parse_result.calls
        ]

        # Step 5: Add includes
        manifest["includes"] = [i.target_path for i in parse_result.includes]

        # Result: ~500 tokens instead of ~15,000
        return IndexManifestBundle(
            file_path=str(parse_result.file_info.path),
            manifest_json=json.dumps(manifest),
            token_estimate=estimate_tokens(manifest),
        )
```

---

20. Complete End-to-End Walkthrough: Indexing a Real Project

Let's walk through indexing a small but realistic project — your custom WiFi manager daemon.

20.1 Project Structure

```
wifi-manager/
├── Makefile
├── include/
│   ├── wifi_hal.h
│   ├── wifi_manager.h
│   └── config.h
├── src/
│   ├── main.c
│   ├── wifi_hal.c
│   ├── wifi_manager.c
│   ├── config_parser.c
│   └── utils.c
├── scripts/
│   └── monitor.sh
└── .config
    └── CONFIG_WIFI=y
         CONFIG_WIFI_6E=n
         CONFIG_BT=y
```

20.2 Step 1: You Run the Command

```
codecrawler index --root ./wifi-manager
```

This calls `IndexingPipeline.run()` which triggers:

20.3 Step 2: File Discovery

```
# os.walk traverses the directory tree:
# wifi-manager/ → include/ → src/ → scripts/

# For each file, check if it's indexable:
LANGUAGE_MAP = {
    ".c": "c", ".h": "c", ".cpp": "cpp", ".cc": "cpp",
    ".py": "python", ".sh": "shell", ".bash": "shell",
}

# Results:
discovered_files = [
    FileInfo(path="include/wifi_hal.h",     language="c",     size=2048,
hash="aa11..."),
    FileInfo(path="include/wifi_manager.h", language="c",     size=1024,
hash="bb22..."),
    FileInfo(path="include/config.h",       language="c",     size=512,
hash="cc33..."),
    FileInfo(path="src/main.c",             language="c",     size=4096,
hash="dd44..."),
    FileInfo(path="src/wifi_hal.c",         language="c",     size=8192,
hash="ee55..."),
    FileInfo(path="src/wifi_manager.c",     language="c",     size=6144,
hash="ff66..."),
    FileInfo(path="src/config_parser.c",    language="c",     size=3072,
hash="1177..."),
    FileInfo(path="src/utils.c",            language="c",     size=1536,
hash="2288..."),
    FileInfo(path="scripts/monitor.sh",     language="shell", size=768,
hash="3399..."),
]
# 9 files discovered
# Makefile and .config are not in LANGUAGE_MAP → skipped

# For each file, event_bus.publish("file.discovered", file_info) is called
```

20.4 Step 3: Build System Detection

```
# Scan for signatures:
#   Yocto:     No meta-* dirs found         → score = 0
#   Buildroot: No Config.in found           → score = 0
#   Kernel:    No Kconfig, no arch/ dir     → score = 0
#   Generic:   Found Makefile               → score = 1
```

```
# Result: "generic" build system (simple Makefile project)
# No special build analysis is triggered

# However, we DO find .config file
# Parse it:
config = {
    "CONFIG_WIFI": True,
    "CONFIG_WIFI_6E": False,
    "CONFIG_BT": True,
}
# This is stored in the BuildConfig table for #ifdef resolution
```

20.5 Step 4: Tier Classification

```
# For a small custom project, everything is classified as i3 (Full):
# No "gcc", "glibc", "busybox" directories → nothing to exclude

# Results:
tiers = {
    "include/": TierClassification(path="include/", tier=3, confidence=0.95),
    "src/":     TierClassification(path="src/",     tier=3, confidence=0.95),
    "scripts/": TierClassification(path="scripts/", tier=3, confidence=0.90),
}
# All files get tier=3 → full analysis on everything
```

20.6 Step 5: Parsing

```
# Route each file to the correct crawler:
# .c and .h files → CCrawler
# .sh files → ShellCrawler

# ═══ Parsing wifi_hal.c with CCrawler ═══

# wifi_hal.c contents:
"""
#include "wifi_hal.h"
#include <stdio.h>
#include <string.h>

#ifdef CONFIG_WIFI
static int g_wifi_state = WIFI_STATE_DISCONNECTED;
static struct wifi_stats g_stats = {0};

int wifi_hal_connect(const char *ssid, int channel, int timeout) {
    if (!ssid || channel < 1 || channel > 165) {
        fprintf(stderr, "Invalid parameters\n");
        return -EINVAL;
    }

    g_wifi_state = WIFI_STATE_CONNECTING;
```

```
    int ret = nl80211_connect(ssid, channel);

    if (ret == 0) {
        g_wifi_state = WIFI_STATE_CONNECTED;
        g_stats.connect_count++;
        printf("Connected to %s on channel %d\n", ssid, channel);
    } else {
        g_wifi_state = WIFI_STATE_DISCONNECTED;
        fprintf(stderr, "Connection failed: %d\n", ret);
    }

    return ret;
}

void wifi_hal_disconnect(void) {
    nl80211_disconnect();
    g_wifi_state = WIFI_STATE_DISCONNECTED;
    g_stats.disconnect_count++;
}

int wifi_hal_scan(struct scan_results *results, int max_results) {
    if (!results || max_results <= 0) return -EINVAL;
    memset(results, 0, sizeof(*results) * max_results);
    return nl80211_scan(results, max_results);
}

const struct wifi_stats *wifi_hal_get_stats(void) {
    return &g_stats;
}
#endif
"""

# Tree-sitter parses this and our tree walker extracts:
parse_result = ParseResult(
    file_info=FileInfo(path="src/wifi_hal.c", language="c", ...),
    functions=[
        FunctionDef(name="wifi_hal_connect",
                    signature="int wifi_hal_connect(const char *ssid, int
channel, int timeout)",
                    start_line=10, end_line=28, complexity=4),
                    # complexity=4: base(1) + if(!ssid)(1) + if(ret==0)(1) +
else(1)
        FunctionDef(name="wifi_hal_disconnect",
                    signature="void wifi_hal_disconnect(void)",
                    start_line=30, end_line=34, complexity=1),
        FunctionDef(name="wifi_hal_scan",
                    signature="int wifi_hal_scan(struct scan_results *results,
int max_results)",
                    start_line=36, end_line=40, complexity=2),
                    # complexity=2: base(1) + if(!results)(1)
        FunctionDef(name="wifi_hal_get_stats",
                    signature="const struct wifi_stats
*wifi_hal_get_stats(void)",
                    start_line=42, end_line=44, complexity=1),
    ],
    structs=[],  # wifi_stats is defined in wifi_hal.h, not here
```

```
    macros=[],   # CONFIG_WIFI is an #ifdef, not a #define
    variables=[
        VariableDef(name="g_wifi_state", var_type="int", is_global=True,
is_static=True),
        VariableDef(name="g_stats", var_type="struct wifi_stats",
is_global=True, is_static=True),
    ],
    calls=[
        CallEdge(caller="wifi_hal_connect", callee="fprintf",
call_site_line=12),
        CallEdge(caller="wifi_hal_connect", callee="nl80211_connect",
call_site_line=16),
        CallEdge(caller="wifi_hal_connect", callee="printf",
call_site_line=21),
        CallEdge(caller="wifi_hal_connect", callee="fprintf",
call_site_line=24),
        CallEdge(caller="wifi_hal_disconnect", callee="nl80211_disconnect",
call_site_line=31),
        CallEdge(caller="wifi_hal_scan", callee="memset", call_site_line=38),
        CallEdge(caller="wifi_hal_scan", callee="nl80211_scan",
call_site_line=39),
    ],
    includes=[
        IncludeEdge(source="wifi_hal.c", target="wifi_hal.h"),
        IncludeEdge(source="wifi_hal.c", target="stdio.h"),
        IncludeEdge(source="wifi_hal.c", target="string.h"),
    ],
)

# event_bus.publish("file.parsed", parse_result) is called
# This triggers:
#   1. Storage component → inserts rows into database
#   2. Tiering component → updates tier statistics

# ═══ This process repeats for all 9 files ═══
```

20.7 Step 6: Database Ingestion

After ALL files are parsed, the database contains:

```
╔═══════════════════════════════════════════════════════════════════╗
║  DATABASE STATE AFTER INDEXING                                    ╒
╚═══════════════════════════════════════════════════════════════════╝
╔                                                                   ╒
║  Directory table: 4 rows                                          ╒
║    wifi-manager/, include/, src/, scripts/                        ╒
║                                                                   ╒
║  File table: 9 rows                                               ╒
║    wifi_hal.h, wifi_manager.h, config.h,                          ╒
║    main.c, wifi_hal.c, wifi_manager.c, config_parser.c,           ╒
║    utils.c, monitor.sh                                            ╒
║                                                                   ╒
║  Function table: ~25 rows (across all files)                      ╒
║    main, wifi_hal_connect, wifi_hal_disconnect, wifi_hal_scan,    ╒
```

```
│    wifi_hal_get_stats, wifi_mgr_init, wifi_mgr_start,            │
│    wifi_mgr_stop, parse_config, load_config, save_config,        │
│    str_trim, str_split, log_init, ...                            │
│                                                                   │
│  calls table: ~50 rows                                           │
│    main → wifi_mgr_init, main → wifi_mgr_start,                  │
│    wifi_mgr_start → wifi_hal_connect,                            │
│    wifi_hal_connect → nl80211_connect, ...                       │
│                                                                   │
│  Variable table: ~8 rows (global/static variables)               │
│    g_wifi_state, g_stats, g_config, g_running, ...               │
│                                                                   │
│  includes_file table: ~20 rows                                   │
│    main.c → wifi_manager.h, wifi_hal.c → wifi_hal.h, ...         │
│                                                                   │
│  BuildConfig table: 3 rows                                       │
│    CONFIG_WIFI=y, CONFIG_WIFI_6E=n, CONFIG_BT=y                  │
│                                                                   │
│  guarded_by table: ~6 rows                                       │
│    wifi_hal_connect → CONFIG_WIFI,                               │
│    wifi_hal_disconnect → CONFIG_WIFI, ...                        │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

20.8 Step 7: Priority Scoring

Now we score every function:

```
Function              | Tier | Usage | Centrality | Build | Runtime | Recency |
SCORE
──────────────────────┼──────┼───────┼────────────┼───────┼─────────┼─────────┼
───────
wifi_hal_connect      | 1.00 | 0.40  | 0.60       | 1.00  | 0.00    | 0.50    |
0.570
wifi_mgr_start        | 1.00 | 0.20  | 0.50       | 1.00  | 0.00    | 0.50    |
0.510
main                  | 1.00 | 0.10  | 0.30       | 1.00  | 0.00    | 0.50    |
0.460
wifi_hal_scan         | 1.00 | 0.15  | 0.20       | 1.00  | 0.00    | 0.33    |
0.400
parse_config          | 1.00 | 0.10  | 0.10       | 1.00  | 0.00    | 0.25    |
0.360
wifi_hal_disconnect   | 1.00 | 0.10  | 0.10       | 1.00  | 0.00    | 0.50    |
0.398
wifi_hal_get_stats    | 1.00 | 0.05  | 0.05       | 1.00  | 0.00    | 0.50    |
0.368
str_trim              | 1.00 | 0.25  | 0.00       | 1.00  | 0.00    | 0.10    |
0.315
log_init              | 1.00 | 0.05  | 0.00       | 1.00  | 0.00    | 0.10    |
0.275
```

`wifi_hal_connect` scores highest because:

- It's in the vendor layer (tier=3 → weight=1.0)
- It's called by multiple functions (usage=0.40)
- It bridges the manager and nl80211 layers (centrality=0.60)
- Its `#ifdef CONFIG_WIFI` guard is active (build=1.0)
- It was recently modified (recency=0.50)

20.9 Step 8: What MCP Returns to an AI Agent

When an AI agent calls `search_code("wifi connection logic")`:

```
{
  "results": [
    {
      "file": "src/wifi_hal.c",
      "score": 0.570,
      "manifest": {
        "functions": [
          {"name": "wifi_hal_connect", "sig": "int wifi_hal_connect(const char
*ssid, int channel, int timeout)", "complexity": 4},
          {"name": "wifi_hal_disconnect", "sig": "void
wifi_hal_disconnect(void)", "complexity": 1},
          {"name": "wifi_hal_scan", "sig": "int wifi_hal_scan(struct
scan_results *, int)", "complexity": 2},
          {"name": "wifi_hal_get_stats", "sig": "const struct wifi_stats
*wifi_hal_get_stats(void)", "complexity": 1}
        ],
        "calls": [
          {"from": "wifi_hal_connect", "to": "nl80211_connect"},
          {"from": "wifi_hal_connect", "to": "fprintf"},
          {"from": "wifi_hal_disconnect", "to": "nl80211_disconnect"},
          {"from": "wifi_hal_scan", "to": "nl80211_scan"}
        ],
        "globals": ["g_wifi_state", "g_stats"],
        "includes": ["wifi_hal.h", "stdio.h", "string.h"],
        "build_guards": {"CONFIG_WIFI": true}
      }
    },
    {
      "file": "src/wifi_manager.c",
      "score": 0.510,
      "manifest": { ... }
    }
  ],
  "token_count": 487
}
```

The AI agent receives **487 tokens** of structured context — enough to understand the WiFi subsystem without reading the full source code. Compare this to reading the raw files, which would cost **~5,000 tokens**. That's a 10× compression.

---

21. The Configuration System

21.1 TOML Format — What It Is

TOML (Tom's Obvious Minimal Language) is a config file format. It's like `.ini` files but with types:

```toml
# .codecrawler.toml — Configuration for Code Crawler

[project]
name = "wifi-daemon"
root = "/home/dev/rdk"
build_system = "yocto"  # "yocto", "buildroot", "kernel", "generic", "auto"

[index]
# Which file extensions to index
include_extensions = [".c", ".h", ".cpp", ".py", ".sh"]
# Directories to always skip (glob patterns)
exclude_patterns = [".git", "*.o", "*.d", "__pycache__"]
# Maximum file size to index (bytes)
max_file_size = 1048576  # 1MB
# Re-index only changed files
incremental = true

[tiers]
# Override automatic tier classification
# Format: "directory_glob" = tier_level
overrides = { "vendor/*" = 3, "toolchain/*" = 0 }

[scoring]
# Priority score weights (must sum to 1.0)
[scoring.weights]
tier = 0.25
usage = 0.20
centrality = 0.15
build = 0.10
runtime = 0.15
recency = 0.15

# Enable self-tuning of weights based on query patterns
self_tuning = false

[llm]
provider = "ollama"        # "ollama", "openai", "anthropic"
model = "llama3.2:3b"     # Model name
base_url = "http://localhost:11434"
# Maximum tokens per LLM call
max_tokens = 4096
# Temperature for summarization (lower = more deterministic)
temperature = 0.3

[embeddings]
model = "sentence-transformers/all-MiniLM-L6-v2"
dimension = 384
# Batch size for embedding generation
batch_size = 32

[mcp]
```

```
host = "127.0.0.1"
port = 8080
# Whether to start the MCP server automatically on `codecrawler index`
auto_start = false

[database]
path = ".codecrawler/index.duckdb"
# Enable WAL (Write-Ahead Logging) for crash safety
wal_mode = true

[watch]
# File system event debounce (milliseconds)
debounce_ms = 500
# Re-index on file save
auto_reindex = true

[ui]
# Start the web UI
enable = false
port = 3000
```

21.2 How Config Flows Through the System

```
# 1. Load config (core/config.py)
config = load_config(Path(".codecrawler.toml"))

# 2. Config is a typed dataclass:
@dataclass
class Config:
    project: ProjectConfig
    index: IndexConfig
    tiers: TierConfig
    scoring: PriorityScoringConfig
    llm: LLMConfig
    embeddings: EmbeddingConfig
    mcp: MCPConfig
    database: DatabaseConfig
    watch: WatchConfig
    ui: UIConfig

# 3. Each component receives only its relevant section:
database = Database(config.database.path)
scorer = PriorityScorer(config.scoring)
pipeline = IndexingPipeline(config, registry, bus)

# 4. Components use their config:
class PriorityScorer:
    def __init__(self, config: PriorityScoringConfig):
        self.weights = config.weights  # {"tier": 0.25, "usage": 0.20, ...}

# 5. CLI overrides take precedence:
# codecrawler index --root /other/path
# This overrides config.project.root even if .codecrawler.toml says differently
```

21.3 Config Precedence

```
Priority (highest first):
1. CLI arguments          codecrawler index --root /foo
2. Environment variables  CODECRAWLER_ROOT=/foo
3. Project .toml file     .codecrawler.toml in project root
4. Global config          ~/.config/codecrawler/config.toml
5. Built-in defaults      Hardcoded in defaults.py
```

21.4 The Default Config Template

When you run `codecrawler init`, it generates a `.codecrawler.toml` with all settings commented out, showing defaults:

```
# From codecrawler/config/defaults.py
DEFAULT_CONFIG_TEMPLATE = """
# Code Crawler Configuration
# Uncomment and modify settings as needed.
# All values shown are defaults.

# [project]
# name = ""                    # Auto-detected from directory name
# root = "."                   # Project root directory
# build_system = "auto"        # Auto-detect build system

# [index]
# include_extensions = [".c", ".h", ".cpp", ".cc", ".py", ".sh", ".bash"]
# exclude_patterns = [".git", "*.o", "*.d", "__pycache__", "node_modules"]
# max_file_size = 1048576
# incremental = true

# ... (all other sections with defaults)
"""
```

---

22. MCP Protocol — Deep Dive Into the Wire Format

22.1 What Happens on the Wire

MCP uses JSON-RPC 2.0 over stdio (standard input/output). Your AI coding assistant spawns our MCP server as a subprocess and communicates via pipes.

```
┌──────────────────┐                          ┌──────────────────────┐
│  AI Assistant    │                          │  MCP Server          │
│  (Cursor, Claude) │                          │  (codecrawler mcp)   │
│                  │                          │                      │
│                  │  spawn as subprocess     │                      │
│                  │ ────────────────────────→│                      │
│                  │  stdin/stdout pipes       │                      │
```

```
│                    │                              │                    │
│  1. List tools     │  {"jsonrpc":"2.0",           │                    │
│                    │   "method":"tools/list",     │                    │
│                    │   "id":1}                    │                    │
│                    │ ─────────stdin──────────→    │                    │
│                    │                              │  Parse request     │
│                    │  {"jsonrpc":"2.0","id":1,    │  Return tool list  │
│                    │   "result":{"tools":[...]}}│                    │
│                    │ ←─────────stdout────────     │                    │
│                    │                              │                    │
│  2. Call tool      │  {"jsonrpc":"2.0",           │                    │
│                    │   "method":"tools/call",     │                    │
│                    │   "params":{"name":          │                    │
│                    │     "search_code",           │                    │
│                    │     "arguments":{            │                    │
│                    │       "query":"wifi conn"    │                    │
│                    │     }},                      │                    │
│                    │   "id":2}                    │                    │
│                    │ ─────────stdin──────────→    │                    │
│                    │                              │  1. Encode query   │
│                    │                              │  2. Vector search  │
│                    │                              │  3. Score+filter   │
│                    │                              │  4. Build response │
│                    │  {"jsonrpc":"2.0","id":2,    │                    │
│                    │   "result":{"content":[      │                    │
│                    │     {"type":"text",          │                    │
│                    │      "text":"...manifest     │                    │
│                    │        JSON here..."}        │                    │
│                    │   ]}}                        │                    │
│                    │ ←─────────stdout────────     │                    │
└────────────────────┘                              └────────────────────┘
```

22.2 Tool Registration

When the AI assistant first connects, it asks "What tools do you have?" Our server responds:

```
{
  "tools": [
    {
      "name": "search_code",
      "description": "Search the indexed codebase by semantic meaning or
keyword. Returns ranked IndexManifest bundles.",
      "inputSchema": {
        "type": "object",
        "properties": {
          "query": {
            "type": "string",
            "description": "Natural language or keyword query"
          },
          "language": {
            "type": "string",
            "description": "Filter by language (optional)",
            "enum": ["c", "cpp", "python", "shell"]
          },
```

```
            "limit": {
              "type": "integer",
              "description": "Maximum results to return",
              "default": 10
            }
          },
          "required": ["query"]
        }
      },
      {
        "name": "get_call_hierarchy",
        "description": "Get the complete call hierarchy for a function — who
calls it and what it calls.",
        "inputSchema": {
          "type": "object",
          "properties": {
            "function_name": { "type": "string" },
            "depth": { "type": "integer", "default": 5 },
            "direction": {
              "type": "string",
              "enum": ["callers", "callees", "both"],
              "default": "both"
            }
          },
          "required": ["function_name"]
        }
      },
      {
        "name": "get_build_context",
        "description": "Look up the build configuration status of a CONFIG_*
symbol.",
        "inputSchema": {
          "type": "object",
          "properties": {
            "symbol": { "type": "string", "description": "e.g., CONFIG_WIFI" }
          },
          "required": ["symbol"]
        }
      },
      {
        "name": "trace_ipc_flow",
        "description": "Trace inter-process communication edges from a
function.",
        "inputSchema": {
          "type": "object",
          "properties": {
            "function_name": { "type": "string" }
          },
          "required": ["function_name"]
        }
      },
      {
        "name": "correlate_serial_log",
        "description": "Map serial/crash log lines to the source functions that
emit them.",
        "inputSchema": {
```

```
          "type": "object",
          "properties": {
            "log_lines": {
              "type": "array",
              "items": { "type": "string" },
              "description": "Lines from serial console or crash log"
            }
          },
          "required": ["log_lines"]
        }
      },
      {
        "name": "analyze_impact",
        "description": "Analyze the blast radius of changing a function.",
        "inputSchema": {
          "type": "object",
          "properties": {
            "function_name": { "type": "string" },
            "max_depth": { "type": "integer", "default": 5 }
          },
          "required": ["function_name"]
        }
      },
      {
        "name": "sync_team",
        "description": "Pull latest team annotations and summaries from the sync
server.",
        "inputSchema": {
          "type": "object",
          "properties": {}
        }
      }
    ]
  }
```

22.3 Resource Registration

Resources are pre-built data the AI can request by URI:

```
  {
    "resources": [
      {
        "uri": "codecrawler://manifest/{path}",
        "name": "File Manifest",
        "description": "Pre-built IndexManifest for a specific file",
        "mimeType": "application/json"
      },
      {
        "uri": "codecrawler://llm_view/high_priority",
        "name": "High Priority Functions",
        "description": "Top functions by composite priority score",
        "mimeType": "application/json"
      },
      {
```

```
      "uri": "codecrawler://telemetry/{subsystem}",
      "name": "Telemetry View",
      "description": "Recent crash/warning data for a subsystem",
      "mimeType": "application/json"
    }
  ]
}
```

22.4 How `search_code` Works Internally

```
@mcp_tool("search_code")
async def search_code(query: str, language: str = None, limit: int = 10):
    """Search the codebase by semantic meaning or keyword."""

    # Step 1: Generate embedding for the query text
    query_embedding = embedding_model.encode(query).tolist()

    # Step 2: Vector similarity search in DuckDB
    sql = """
        SELECT f.id, f.name, f.signature, fi.path, f.summary,
               ps.composite_score,
               array_cosine_distance(f.embedding, ?::FLOAT[384]) AS
semantic_distance
        FROM Function f
        JOIN contains_func cf ON f.id = cf.func_id
        JOIN File fi ON cf.file_id = fi.id
        LEFT JOIN PriorityScore ps ON f.id = ps.func_id
        WHERE f.embedding IS NOT NULL
    """
    params = [query_embedding]

    if language:
        sql += " AND fi.language = ?"
        params.append(language)

    sql += " ORDER BY semantic_distance ASC LIMIT ?"
    params.append(limit)

    results = db.execute(sql, params).fetchall()

    # Step 3: For each matching function, retrieve its file's manifest
    manifests = []
    seen_files = set()
    for row in results:
        file_path = row[3]
        if file_path not in seen_files:
            seen_files.add(file_path)
            manifest = db.execute(
                "SELECT manifest_json FROM IndexManifest WHERE file_path = ?",
                [file_path]
            ).fetchone()
            if manifest:
                manifests.append({
                    "file": file_path,
```

```
                    "score": row[5] or 0.0,
                    "manifest": json.loads(manifest[0]),
                })

    # Step 4: Return to AI agent
    return manifests
```

22.5 The MCP Server Startup Sequence

```
$ codecrawler mcp --database .codecrawler/index.duckdb

1. Load configuration from .codecrawler.toml
2. Open DuckDB connection (read-only for safety)
3. Load embedding model (if embeddings exist in DB)
4. Register all 7 tools + 3 resources
5. Start listening on stdin for JSON-RPC messages
6. For each message:
   a. Parse JSON-RPC envelope
   b. Route to correct handler based on "method"
   c. Execute handler (database queries, etc.)
   d. Write JSON-RPC response to stdout
7. On EOF (AI assistant disconnects), cleanup and exit
```

---

23. Testing Strategy

23.1 What to Test and Where

Component	What to test	Test type	Key assertions
DTOs	Frozen dataclass creation	Unit	Cannot modify after creation
EventBus	Subscribe, publish, unsubscribe	Unit	All handlers called, exceptions isolated
ServiceRegistry	Register, get, get_all	Unit	Correct instances returned
CCrawler	Parse C files → ParseResult	Unit	Functions, structs, calls extracted
PriorityScorer	Score computation	Unit	Math is correct (use worked examples)
TierClassifier	Directory classification	Unit	Known dirs mapped to correct tiers
ManifestBuilder	Compression ratio	Unit	Output < 600 tokens
Database	Schema creation, insert, query	Integration	Tables exist, data roundtrips
Pipeline	Full indexing run	Integration	Database populated correctly
MCP Server	Tool calls → responses	End-to-end	Correct JSON-RPC responses

23.2 Unit Test Examples

Testing the Priority Scorer

```python
import pytest
from datetime import datetime, timezone, timedelta
from codecrawler.tiering.priority_scorer import PriorityScorer

def test_priority_score_perfect_function():
    """A function with all dimensions at maximum scores 1.0."""
    scorer = PriorityScorer()
    result = scorer.score(
        func_id=1,
        tier_level=3,           # Max tier
        call_count=100,         # Most called function
        max_call_count=100,
        betweenness=1.0,        # Maximum centrality
        build_guard_active=True,
        runtime_hits=1000,
        max_runtime_hits=1000,
        last_modified=datetime.now(timezone.utc),  # Modified right now
    )
    assert result.composite_score == pytest.approx(1.0, abs=0.01)

def test_priority_score_worst_function():
    """A function with all dimensions at minimum scores ~0."""
    scorer = PriorityScorer()
    result = scorer.score(
        func_id=2,
        tier_level=0,
        call_count=0,
        max_call_count=100,
        betweenness=0.0,
        build_guard_active=False,
        runtime_hits=0,
        max_runtime_hits=100,
        last_modified=None,
    )
    assert result.composite_score < 0.01

def test_recency_decay():
    """Recency should decay smoothly over time."""
    scorer = PriorityScorer()
    now = datetime.now(timezone.utc)

    score_today = scorer.score(1, last_modified=now)
    score_week = scorer.score(1, last_modified=now - timedelta(days=7))
    score_month = scorer.score(1, last_modified=now - timedelta(days=30))

    # More recent = higher score
    assert score_today.recency_score > score_week.recency_score
    assert score_week.recency_score > score_month.recency_score

    # Specific values
    assert score_today.recency_score == pytest.approx(1.0, abs=0.01)
    assert score_week.recency_score == pytest.approx(0.125, abs=0.01)
```

Testing the C Crawler

```python
def test_c_crawler_extract_function():
    """CCrawler should extract function definitions from C code."""
    crawler = CCrawler()
    source = '''
    int wifi_connect(const char *ssid, int timeout) {
        return hal_connect(ssid, timeout);
    }
    '''
    # Create a temporary file with this content
    result = crawler.parse(FileInfo(path=tmp_file, language="c", ...))

    assert len(result.functions) == 1
    assert result.functions[0].name == "wifi_connect"
    assert "const char *ssid" in result.functions[0].signature
    assert result.functions[0].complexity >= 1

def test_c_crawler_extract_calls():
    """CCrawler should extract function call edges."""
    result = crawler.parse(...)
    call_names = [c.callee for c in result.calls]
    assert "hal_connect" in call_names

def test_c_crawler_extract_globals():
    """CCrawler should extract global variables."""
    source = 'static int g_count = 0;\nint main() { return g_count; }'
    result = crawler.parse(...)
    globals = [v for v in result.variables if v.is_global]
    assert len(globals) == 1
    assert globals[0].name == "g_count"
```

Testing the Event Bus

```python
def test_event_bus_publishes_to_subscribers():
    bus = EventBus()
    received = []

    bus.subscribe("test.event", lambda data: received.append(data))
    bus.publish("test.event", {"key": "value"})

    assert len(received) == 1
    assert received[0] == {"key": "value"}

def test_event_bus_handler_exception_doesnt_crash():
    """One handler crashing shouldn't prevent other handlers from running."""
    bus = EventBus()
    received = []

    def bad_handler(data): raise ValueError("crash!")
    def good_handler(data): received.append(data)

    bus.subscribe("test.event", bad_handler)
    bus.subscribe("test.event", good_handler)
    bus.publish("test.event", "hello")
```

```
    # good_handler should still have been called
    assert received == ["hello"]
```

23.3 Integration Test — Full Pipeline

```
def test_full_indexing_pipeline(tmp_path):
    """Index a small C project and verify database contents."""
    # Create test files
    (tmp_path / "main.c").write_text('''
        #include "wifi.h"
        int main() { return wifi_connect("test", 30); }
    ''')
    (tmp_path / "wifi.c").write_text('''
        int wifi_connect(const char *ssid, int timeout) { return 0; }
    ''')
    (tmp_path / "wifi.h").write_text('''
        int wifi_connect(const char *ssid, int timeout);
    ''')

    # Run the pipeline
    config = Config(project=ProjectConfig(root=tmp_path))
    bus = EventBus()
    registry = ServiceRegistry()

    # Register crawlers
    registry.register(BaseCrawler, CCrawler())

    pipeline = IndexingPipeline(config, registry, bus)
    pipeline.run()

    # Verify database
    db = Database(tmp_path / ".codecrawler" / "index.duckdb")

    # Should have 3 files
    files = db.execute("SELECT COUNT(*) FROM File").fetchone()[0]
    assert files == 3

    # Should have at least 2 functions (main + wifi_connect)
    funcs = db.execute("SELECT COUNT(*) FROM Function").fetchone()[0]
    assert funcs >= 2

    # Should have a call edge from main to wifi_connect
    calls = db.execute("""
        SELECT f1.name, f2.name
        FROM calls c
        JOIN Function f1 ON c.caller_id = f1.id
        JOIN Function f2 ON c.callee_id = f2.id
    """).fetchall()
    assert ("main", "wifi_connect") in [(c[0], c[1]) for c in calls]
```

23.4 Running Tests

```
# Run all tests
python -m pytest tests/ -v

# Run specific test file
python -m pytest tests/test_priority_scorer.py -v

# Run with coverage report
python -m pytest tests/ --cov=codecrawler --cov-report=html

# Run only tests matching a pattern
python -m pytest tests/ -k "test_c_crawler" -v
```

---

24. Comprehensive Glossary

Every term used in this guide and in the Code Crawler codebase, defined in one place.

Term	Definition
ABC	Abstract Base Class. A Python class that cannot be instantiated — it defines an interface that subclasses must implement. Our `BaseCrawler` is an ABC.
AST	Abstract Syntax Tree. A tree representation of source code structure, with syntax details (whitespace, semicolons) removed. Tree-sitter produces a concrete syntax tree (CST) that is similar.
Betweenness Centrality	A graph metric measuring how many shortest paths between other nodes pass through a given node. High betweenness = bridge/bottleneck function.
BFS	Breadth-First Search. A graph traversal that visits all neighbors before going deeper. Used in our recursive CTEs.
Call Edge	A directed relationship in our graph: "Function A calls Function B." Stored in the `calls` table.
Call Graph	The complete graph of all function calls in a codebase. Nodes = functions, edges = calls.
CLI	Command Line Interface. Our `codecrawler` command-line tool built with Click.
Composite Score	The final priority score for a function (0.0–1.0), computed from 6 weighted dimensions.
Cosine Similarity	A measure of how similar two vectors are, based on the angle between them. Range: -1 to 1 (1 = identical direction).
CTE	Common Table Expression. A named subquery in SQL (`WITH name AS (...)`). Used for readable complex queries.
Cyclomatic Complexity	A metric counting independent code paths through a function. Formula: 1 + number of decision points (if, while, for, case, &&, ||).
DAG	Directed Acyclic Graph. A graph with no cycles. Include graphs should be DAGs (no circular includes).

Term	Definition
Dataclass	A Python decorator (`@dataclass`) that auto-generates `__init__`, `__repr__`, and comparison methods from type annotations. Our DTOs are frozen dataclasses.
DDL	Data Definition Language. SQL statements that define schema: CREATE TABLE, ALTER TABLE, DROP TABLE.
DFS	Depth-First Search. A graph traversal that goes as deep as possible before backtracking. Used in our tree-sitter AST walker.
DTO	Data Transfer Object. An immutable container that carries data between components. Our `ParseResult`, `FunctionDef`, `FileInfo` are DTOs.
DuckDB	An embedded analytical database. Single-file, serverless, fast for read-heavy workloads. Our storage backend.
DuckPGQ	DuckDB's Property Graph extension. Defines graphs on top of tables and enables graph queries.
Edge Table	A database table that stores relationships between entities (e.g., `calls`, `includes_file`). Has two foreign key columns pointing to the connected entities.
Embedding	A fixed-length vector of floating-point numbers that represents the semantic meaning of text. Generated by a neural network model.
Event Bus	A pub/sub messaging system within our process. Components publish events and subscribe to events without knowing about each other.
Foreign Key	A column that references the primary key of another table. Like a pointer between structs.
Frozen	A dataclass with `frozen=True` is immutable — its fields cannot be changed after creation. Like a `const struct`.
Generator	A Python function that uses `yield` to produce values one at a time (lazy evaluation). Saves memory compared to building a complete list.
Graph	A data structure consisting of nodes (vertices) connected by edges. Used to model function calls, includes, and other relationships.
HNSW	Hierarchical Navigable Small World. A data structure for fast approximate nearest-neighbor search in high-dimensional vector spaces.
i0/i1/i2/i3	The four indexing tiers: i0 (Ignore), i1 (Stub), i2 (Skeleton), i3 (Full).
Incremental Indexing	Only re-indexing files that have changed (detected by content hash comparison).
Include Edge	A directed relationship: "File A includes File B" via `#include`. Stored in the `includes_file` table.
IndexManifest	A compressed representation of a file (~500 tokens instead of ~15,000). Contains function signatures, call edges, globals, includes. Served to AI agents via MCP.
JSON-RPC	A remote procedure call protocol using JSON. MCP uses JSON-RPC 2.0 over stdio.
JOIN	SQL operation that combines rows from multiple tables based on matching key columns. Like following pointer chains in C.

Term	Definition
LEFT JOIN	A JOIN that includes all rows from the left table, even if no match exists in the right table (unmatched columns are NULL).
LLM	Large Language Model. An AI model trained on text (GPT-4, Claude, Llama, etc.). We use LLMs for code summarization and classification (v5).
Manifest	See IndexManifest.
MCP	Model Context Protocol. A standard protocol connecting AI models to external tools and data sources. Our MCP server exposes the indexed codebase.
Normalization	Scaling a value to a 0–1 range. Used in priority scoring: `normalized = value / max_value`.
NULL	The absence of a value in SQL. Not zero, not empty string — undefined. Requires `IS NULL` comparison.
Parse Tree	The tree structure produced by a parser from source code. Each node represents a syntactic construct (function, statement, expression).
ParseResult	Our main DTO containing everything extracted from one file: functions, structs, variables, calls, includes, macros.
Pipeline	A sequence of processing stages where each stage transforms data and passes it to the next. Our indexing pipeline has 8 stages.
Plugin	A modular component that can be added/removed without changing core code. Crawlers are plugins. Similar to Linux kernel modules.
Primary Key	A column (or set of columns) that uniquely identifies each row in a table. Like an array index, but named and guaranteed unique.
Priority Score	A 6-dimension composite metric (0–1) measuring how important a function is. Higher = more likely to be relevant to queries.
Property Graph	A graph where both nodes and edges can have properties (key-value data). DuckPGQ implements this on top of DuckDB tables.
Pub/Sub	Publish/Subscribe pattern. Publishers emit events; subscribers receive them. Our EventBus implements pub/sub.
Recursive CTE	A CTE that references itself, enabling iterative processing. Used for graph traversal (following call chains to arbitrary depth).
Registry	A central lookup service where components register themselves. Other components get services from the registry.
Schema	The structure of a database: tables, columns, types, constraints, indexes. Defined by DDL statements.
Semantic Search	Finding results by meaning rather than exact text match. Enabled by vector embeddings and cosine similarity.
Sentence Transformer	A neural network model that converts text into vector embeddings. We use `all-MiniLM-L6-v2` (384 dimensions).

Term	Definition
S-Expression	A text representation of tree structure using nested parentheses: `(parent (child1) (child2))`. Used by Tree-sitter for debugging.
SQL	Structured Query Language. The standard language for database operations. DuckDB speaks SQL.
Tier	A classification level (0–3) assigned to directories/files, determining how deeply they are analyzed.
TOML	Tom's Obvious Minimal Language. Our configuration file format. Supports typed values, sections, arrays, and inline tables.
Topological Sort	Ordering graph nodes so that all edges point forward (no node before its dependencies). Like `make` dependency resolution.
Transitive Closure	All nodes reachable from a starting node via any path (not just direct neighbors).
Translation Unit	In C, a single `.c` file after preprocessing (with all `#include`d headers inlined). In Tree-sitter, the root node of the AST.
Tree-sitter	A parser generator producing fast, fault-tolerant parsers for languages. Used for C/C++ parsing. Outputs a concrete syntax tree.
Vector	An ordered list of numbers. In our context, a 384-dimensional embedding representing semantic meaning.
VIEW	A saved SQL query that acts like a virtual table. Our LLM views pre-join tables for fast AI access.
VSS	Vector Similarity Search. DuckDB extension for vector indexing and nearest-neighbor queries.
WAL	Write-Ahead Logging. A database technique for crash recovery — changes are written to a log before being applied.

Summary of All Resources

Where Everything Lives

```
codecrawler/
├── __init__.py              ← Package version (4.0.0)
├── __main__.py              ← python -m codecrawler entry point
├── cli.py                   ← Click CLI with 7 commands
├── core/
│   ├── types.py             ← All DTOs (FileInfo, ParseResult, FunctionDef,
...)
│   ├── event_bus.py         ← Pub/sub communication
│   ├── registry.py          ← Service registry for DI
│   ├── config.py            ← TOML loader + typed config classes
│   └── pipeline.py          ← 8-stage indexing orchestrator
├── storage/
```

```
│   ├── schema.py            ← 20+ table DDL
│   ├── database.py          ← DuckDB connection manager
│   ├── graph.py             ← DuckPGQ graph queries
│   └── vector.py            ← VSS embedding indexes
├── crawlers/
│   ├── base.py              ← BaseCrawler ABC
│   ├── c_crawler.py         ← C/C++ parser (Tree-sitter + fallback)
│   ├── python_crawler.py    ← Python parser (ast module)
│   └── shell_crawler.py     ← Shell parser (regex)
├── analyzers/
│   ├── build_detector.py    ← Build system auto-detection
│   ├── yocto.py             ← Yocto project analyzer
│   ├── buildroot.py         ← Buildroot .config parser
│   └── kernel.py            ← Kernel config + compile_commands
├── tiering/
│   ├── classifier.py        ← i0–i3 tier classification
│   ├── priority_scorer.py   ← 6-dimension scoring engine
│   └── manifest_builder.py  ← File-to-manifest compressor
├── intelligence/
│   ├── proactive_agent.py   ← Thread-safety vulnerability scanner
│   ├── summarizer.py        ← Code summarization engine
│   └── telemetry.py         ← Log correlation engine
├── plugins/
│   ├── base.py              ← PluginBase ABC + PluginManifest
│   ├── loader.py            ← Plugin discovery
│   └── registry.py          ← Plugin lifecycle manager
├── mcp/
│   └── server.py            ← MCP tool + resource definitions
└── config/
    └── defaults.py          ← Default TOML template
```

What to Study First

If you're planning v5 implementation, study in this order:

1. `core/types.py` — Understand all DTOs first. Every component talks through these.
2. `core/event_bus.py` — How components communicate.
3. `core/pipeline.py` — How the 8 stages are orchestrated.
4. `crawlers/c_crawler.py` — The most complex parser. If you understand this, you understand all crawlers.
5. `storage/schema.py` — The database schema. This is the ground truth of what data we store.
6. `tiering/priority_scorer.py` — The math. Only 115 lines, very readable.
7. `mcp/server.py` — How AI agents consume our data.

Further Reading

- **DuckDB docs**: https://duckdb.org/docs/
- **Tree-sitter docs**: https://tree-sitter.github.io/tree-sitter/
- **MCP specification**: https://modelcontextprotocol.io/
- **Sentence Transformers**: https://www.sbert.net/

This study guide was auto-generated alongside the v4 implementation. Total: 24 chapters covering the complete Code Crawler system from zero to architect-level understanding.

 Code Crawler v4 — *Index Smart, Query Fast, Ship Confident*

PART III — SOURCE CODE WALKTHROUGHS & ADVANCED TOPICS

This part provides **line-by-line walkthroughs** of every critical source file and covers advanced topics. After reading this, you'll understand not just *what* the code does but *why* every line exists.

---

25. Line-by-Line Walkthrough: `core/types.py`

This file defines all DTOs. It's the "lingua franca" that every component speaks.

```
"""
Shared data-transfer objects (DTOs) for cross-component data exchange.

Every dataclass here is frozen (immutable). This guarantees:
1. Thread safety — multiple components can read simultaneously
2. Cache safety — DTOs can be used as dictionary keys
3. No surprise mutations — what you create is what you get

Design rule: NO behavior (methods) in DTOs. They are pure data containers.
If you need to compute something from a DTO, put the logic in the component
that consumes it, not in the DTO itself.
"""
from dataclasses import dataclass, field
from pathlib import Path
from datetime import datetime
from typing import Optional

# ── FILE LEVEL ──────────────────────────────────────────────────────────────


@dataclass(frozen=True)
class FileInfo:
    """Represents one indexable file on disk.

    Created by: Pipeline Stage 1 (file discovery)
    Consumed by: Crawlers (parse this file), Storage (insert into File table)

    Fields explained:
      path:          Absolute or root-relative path. TEXT in database.
      language:      From LANGUAGE_MAP lookup. Determines which crawler handles
it.
      size:          os.stat(path).st_size. Used for i/o budget estimation.
      hash:          SHA-256 of file contents. For incremental indexing — if
hash
```

```
                        matches the stored hash, we skip re-parsing.
      last_modified: From os.stat(path).st_mtime. For recency scoring.
    """
    path: Path                                  #
/home/dev/rdk/ccsp/wifi/wifi_hal.c
    language: str                               # "c", "python", "shell"
    size: int = 0                               # bytes
    hash: str = ""                              # SHA-256, hex-encoded
    last_modified: Optional[datetime] = None  # from filesystem mtime

# ─── FUNCTION LEVEL
──────────────────────────────────────────────────────────────────

@dataclass(frozen=True)
class FunctionDef:
    """One function extracted from source code.

    Created by: Crawlers (CCrawler, PyCrawler, ShellCrawler)
    Consumed by: Storage (→ Function table), Scorer, ManifestBuilder

    The 'complexity' field is cyclomatic complexity:
      1 = simplest possible (no branches)
      2-5 = normal
      6-10 = complex
      11+ = too complex, candidate for refactoring

    body_hash is SHA-256 of the function body text. Used to detect when a
    function's implementation changed even if its signature didn't.
    """
    name: str                     # "wifi_connect"
    signature: str                # "int wifi_connect(const char *ssid, int
timeout)"
    start_line: int               # 14    (1-indexed, inclusive)
    end_line: int                 # 35    (1-indexed, inclusive)
    complexity: int = 1           # cyclomatic complexity (1 = base)
    body_hash: str = ""           # SHA-256 of function body
    doc_comment: str = ""         # JSDoc, Doxygen, or docstring if present
    is_static: bool = False       # C: static keyword present
    return_type: str = ""         # "int", "void", "struct foo *"

# ─── STRUCT LEVEL
────────────────────────────────────────────────────────────────────

@dataclass(frozen=True)
class StructDef:
    """One struct/class/union definition.

    For C:     struct foo { int x; char *name; };
    For Python: class Foo: ...

    'members' contains the raw text of each member declaration:
      ["int x", "char *name"]
    We store raw text rather than parsed types because type parsing is
    language-specific and not needed for the index — AI agents can read
    the raw text just fine.
    """
```

```
    name: str                                       # "wifi_config"
    kind: str = "struct"                            # "struct", "union", "enum",
"class"
    members: list[str] = field(default_factory=list)  # ["char ssid[32]", "int
channel"]
    start_line: int = 0
    end_line: int = 0

# ─── EDGES
────────────────────────────────────────────────────────────────────────────

@dataclass(frozen=True)
class CallEdge:
    """Function A calls Function B at a specific line.

    This is a DIRECTED edge in the call graph.
    'caller' is empty string initially — it gets filled in during extraction
    (the crawler walks functions and records calls found within each body).

    'is_indirect' marks function pointer calls:
      callback(data);    ← indirect (we don't know what 'callback' points to)
      wifi_connect(ssid); ← direct
    """
    caller: str = ""         # "wifi_hal_connect" — filled during extraction
    callee: str = ""         # "nl80211_connect"
    call_site_line: int = 0  # line number of the call
    is_indirect: bool = False  # via function pointer?

@dataclass(frozen=True)
class IncludeEdge:
    """File A includes File B (#include directive).

    For C: #include "wifi_hal.h"     → source=current_file, target="wifi_hal.h"
    For C: #include <stdio.h>        → source=current_file, target="stdio.h"

    'is_system' distinguishes <> vs "" includes:
      <stdio.h>    → system include (likely i0/i1 tier, don't index deeply)
      "wifi_hal.h" → project include (likely i2/i3 tier, index fully)
    """
    source_path: str = ""   # file doing the including
    target_path: str = ""   # file being included
    is_system: bool = False # <stdio.h> vs "wifi_hal.h"

# ─── VARIABLES AND MACROS
──────────────────────────────────────────────────────────────────────────

@dataclass(frozen=True)
class VariableDef:
    """A global or static variable definition.

    Only tracks global/static scope variables, not local variables.
    In embedded C, globals are the main source of shared state issues
    (thread safety, re-entrancy), so they're important to track.
    """
    name: str               # "g_wifi_state"
    var_type: str = ""      # "int", "struct wifi_stats"
```

```
    is_global: bool = False # true if file scope
    is_static: bool = False # true if 'static' keyword
    is_const: bool = False  # true if 'const' keyword
    line: int = 0

@dataclass(frozen=True)
class MacroDef:
    """A preprocessor #define macro.

    name: The macro identifier
    value: The macro value (may be empty for guard macros)
    is_config_guard: True if this is a CONFIG_* #ifdef guard
    """
    name: str               # "MAX_RETRIES" or "CONFIG_WIFI"
    value: str = ""         # "10" or "" for guards
    is_config_guard: bool = False
    line: int = 0

# ─── COMPOSITE RESULTS ──────────────────────────────────────────────────────

@dataclass(frozen=True)
class ParseResult:
    """Everything extracted from one file. This is the main output of a
crawler.

    Created by: Crawlers
    Published to EventBus: topic "file.parsed"
    Consumed by: Storage, Tiering, ManifestBuilder

    WHY frozen with field(default_factory=list)?
    Because Python default mutable arguments are shared across instances:
      def bad(items=[]):  ← all calls share the SAME list!
    field(default_factory=list) creates a NEW list for each instance.
    """
    file_info: FileInfo
    functions: list[FunctionDef] = field(default_factory=list)
    structs: list[StructDef] = field(default_factory=list)
    variables: list[VariableDef] = field(default_factory=list)
    calls: list[CallEdge] = field(default_factory=list)
    includes: list[IncludeEdge] = field(default_factory=list)
    macros: list[MacroDef] = field(default_factory=list)

# ─── TIERING RESULTS ────────────────────────────────────────────────────────

@dataclass(frozen=True)
class TierClassification:
    """Tier assignment for a directory or file.

    tier values:
      0 (i0): IGNORE. Don't index at all. (e.g., gcc source, glibc)
      1 (i1): STUB. Only record file existence + language.
      2 (i2): SKELETON. Extract function signatures, no call graph.
      3 (i3): FULL. Extract everything — signatures, calls, globals, etc.
```

```
    confidence: How sure the classifier is (0.0–1.0).
    reason: Human-readable explanation.
    """
    path: str               # "vendor/gcc"
    tier: int               # 0, 1, 2, or 3
    confidence: float = 0.5 # 0.0 = pure guess, 1.0 = certain
    reason: str = ""        # "known upstream package: gcc"

@dataclass(frozen=True)
class PriorityScoreResult:
    """6-dimension priority score for a function.

    All individual scores are normalized to [0, 1].
    composite_score = weighted sum of all 6 dimensions.
    """
    func_id: int
    tier_score: float = 0.0        # Tier weight: i3→1.0, i2→0.66, i1→0.33,
i0→0.0
    usage_score: float = 0.0       # call_count / max_call_count
    centrality_score: float = 0.0  # betweenness centrality (normalized)
    build_score: float = 0.0       # 1.0 if build guard active, 0.0 if guarded
out
    runtime_score: float = 0.0     # runtime_hits / max_runtime_hits (from
telemetry)
    recency_score: float = 0.0     # 1/(1 + days_since_modified/7) (hyperbolic
decay)
    composite_score: float = 0.0   # Weighted sum of above 6 dimensions

# ─── MANIFEST
──────────────────────────────────────────────────────────────────────────────

@dataclass(frozen=True)
class IndexManifestBundle:
    """Compressed file representation for AI consumption.

    The manifest_json contains:
      {
        "file": {"path": ..., "language": ..., "loc": ...},
        "functions": [{"name": ..., "sig": ..., "complexity": ...}, ...],
        "structs": [...],
        "calls": [{"from": ..., "to": ...}, ...],
        "includes": [...],
        "globals": [...]
      }

    Token estimate tells the pipeline how much context budget this will use.
    Target: ~500 tokens per file manifest (vs ~15,000 for raw source).
    """
    file_path: str
    manifest_json: str       # JSON string
    token_estimate: int = 0  # Approximate GPT tokens

@dataclass(frozen=True)
class SummaryResult:
    """LLM-generated summary for a function or file.
```

```
    quality_tier:
      "heuristic" — generated by keyword extraction (fast, free)
      "llm_basic"  — generated by local LLM (medium quality)
      "llm_advanced" — generated by cloud LLM (highest quality)
    """
    entity_id: int
    entity_type: str = "function"  # "function" or "file"
    summary: str = ""
    quality_tier: str = "heuristic"

@dataclass(frozen=True)
class PatchSuggestion:
    """A proactive suggestion from the intelligence module.

    severity:
      "info"     — informational note
      "warning"  — potential issue (e.g., shared global without mutex)
      "critical" — likely bug (e.g., multi-writer race condition)
    """
    file_path: str
    function_name: str = ""
    start_line: int = 0
    end_line: int = 0
    severity: str = "info"
    message: str = ""
    suggested_fix: str = ""
```

---

26. Line-by-Line Walkthrough: `core/pipeline.py`

The pipeline is the heart of Code Crawler. It orchestrates the entire indexing process.

```
"""
The 8-stage indexing pipeline.

Why stages? Each stage transforms data and publishes events:
  Stage 1: Discover files    → FileInfo objects
  Stage 2: Detect build      → BuildConfig
  Stage 3: Classify tiers    → TierClassification
  Stage 4: Parse files       → ParseResult (via crawlers)
  Stage 5: Score priorities   → PriorityScoreResult
  Stage 6: Build manifests   → IndexManifestBundle
  Stage 7: Generate embeddings → vectors in Function table
  Stage 8: Run intelligence   → PatchSuggestion, SummaryResult

Stages are SEQUENTIAL (each feeds the next).
Within each stage, files are processed in PARALLEL (where possible).
"""
import hashlib
import logging
import os
from pathlib import Path
from typing import Optional
```

```
from codecrawler.core.types import FileInfo, ParseResult
from codecrawler.core.event_bus import EventBus
from codecrawler.core.registry import ServiceRegistry
from codecrawler.core.config import Config
from codecrawler.crawlers.base import BaseCrawler

logger = logging.getLogger(__name__)

# Extension → language mapping
# This determines which files we index and which crawler handles them
LANGUAGE_MAP = {
    ".c": "c", ".h": "c",
    ".cpp": "cpp", ".cxx": "cpp", ".cc": "cpp", ".hpp": "cpp", ".hxx": "cpp",
    ".py": "python",
    ".sh": "shell", ".bash": "shell",
    ".json": "json",
    ".toml": "toml",
    ".yaml": "yaml", ".yml": "yaml",
    ".mk": "makefile",
    ".bb": "bitbake", ".bbappend": "bitbake", ".bbclass": "bitbake",
    ".conf": "conf",
}


class IndexingPipeline:
    """Orchestrates the complete indexing workflow.

    Lifecycle:
      pipeline = IndexingPipeline(config, registry, bus)
      pipeline.run()   ← runs all 8 stages

    The pipeline uses the ServiceRegistry to find crawlers:
      registry.get_all(BaseCrawler) → [CCrawler, PyCrawler, ShellCrawler]

    It uses the EventBus to publish progress:
      bus.publish("file.discovered", file_info)
      bus.publish("file.parsed", parse_result)
      bus.publish("stage.complete", stage_name)
    """

    def __init__(self, config: Config, registry: ServiceRegistry, bus:
EventBus):
        self._config = config
        self._registry = registry
        self._bus = bus

        # Build a language→crawler lookup table
        # So we can quickly find the right crawler for any file
        self._crawler_map: dict[str, BaseCrawler] = {}
        for crawler in registry.get_all(BaseCrawler):
            for lang in crawler.supported_languages:
                self._crawler_map[lang] = crawler

        # Accumulate results across stages
        self._discovered_files: list[FileInfo] = []
```

```
        self._parse_results: list[ParseResult] = []

    def run(self) -> None:
        """Execute all pipeline stages in order."""
        logger.info("Starting indexing pipeline for %s",
self._config.project.root)

        # Stage 1: Find all indexable files
        self._stage_discover()
        self._bus.publish("stage.complete", "discover")

        # Stage 2: Detect the build system
        self._stage_detect_build()
        self._bus.publish("stage.complete", "build_detect")

        # Stage 3: Classify files into tiers
        self._stage_classify_tiers()
        self._bus.publish("stage.complete", "tier_classify")

        # Stage 4: Parse each file with the appropriate crawler
        self._stage_parse()
        self._bus.publish("stage.complete", "parse")

        # Stage 5: Compute priority scores
        self._stage_score()
        self._bus.publish("stage.complete", "score")

        # Stage 6: Build index manifests
        self._stage_build_manifests()
        self._bus.publish("stage.complete", "manifest")

        logger.info("Pipeline complete. Indexed %d files, %d functions",
                    len(self._discovered_files),
                    sum(len(r.functions) for r in self._parse_results))

    # ── STAGE 1: File Discovery ──────────────────────────────────────

    def _stage_discover(self) -> None:
        """Walk the filesystem and identify indexable files.

        This is equivalent to: find /project -type f | while read f; do ...
done

        For each file:
        1. Check extension against LANGUAGE_MAP
        2. Check against exclude patterns (e.g., .git, build/)
        3. Check file size (skip files larger than max_file_size)
        4. Compute SHA-256 hash (for incremental indexing)
        5. Create FileInfo DTO and publish event
        """
        root = Path(self._config.project.root)
        exclude = set(self._config.index.exclude_patterns)
        max_size = self._config.index.max_file_size

        for dirpath, dirnames, filenames in os.walk(root):
            # Skip excluded directories (modifying dirnames in-place tells
```

```
os.walk to skip them)
            dirnames[:] = [d for d in dirnames if d not in exclude]

            for filename in filenames:
                filepath = Path(dirpath) / filename
                ext = filepath.suffix.lower()

                # Check if we support this file type
                if ext not in LANGUAGE_MAP:
                    continue

                # Check against exclude patterns
                if any(filepath.match(pat) for pat in exclude):
                    continue

                # Check file size
                try:
                    stat = filepath.stat()
                    if stat.st_size > max_size:
                        logger.debug("Skipping large file: %s (%d bytes)",
filepath, stat.st_size)
                        continue
                except OSError:
                    continue

                # Compute content hash for incremental detection
                content_hash = self._hash_file(filepath)

                # Create FileInfo DTO
                file_info = FileInfo(
                    path=filepath.relative_to(root),
                    language=LANGUAGE_MAP[ext],
                    size=stat.st_size,
                    hash=content_hash,
                    last_modified=None,  # Will be set from git or mtime
                )

                self._discovered_files.append(file_info)
                self._bus.publish("file.discovered", file_info)

        logger.info("Discovered %d indexable files",
len(self._discovered_files))

    # ── STAGE 4: Parsing ──────────────────────────────────────────────

    def _stage_parse(self) -> None:
        """Parse each discovered file using the appropriate language crawler.

        For each file:
        1. Look up its language in _crawler_map
        2. Call crawler.parse(file_info)
        3. Handle errors gracefully (skip file, don't crash)
        4. Publish the ParseResult
        """
        for file_info in self._discovered_files:
            crawler = self._crawler_map.get(file_info.language)
```

```
            if not crawler:
                logger.debug("No crawler for language '%s', skipping %s",
                             file_info.language, file_info.path)
                continue

            if not crawler.can_parse(file_info):
                continue

            try:
                result = crawler.parse(file_info)
                self._parse_results.append(result)
                self._bus.publish("file.parsed", result)

                logger.debug("Parsed %s: %d functions, %d calls",
                             file_info.path,
                             len(result.functions),
                             len(result.calls))

            except Exception:
                # IMPORTANT: Never crash the pipeline because of one bad file
                # Log the error and continue with the next file
                logger.exception("Failed to parse %s", file_info.path)

    # ── HELPER ──────────────────────────────────────────────────────────

    @staticmethod
    def _hash_file(filepath: Path, chunk_size: int = 65536) -> str:
        """Compute SHA-256 hash of a file in chunks (memory-efficient).

        We read in 64KB chunks instead of reading the entire file into memory.
        For a 10MB file, this uses 64KB of memory instead of 10MB.

        C analogy:
        while ((n = read(fd, buf, sizeof(buf))) > 0)
            SHA256_Update(&ctx, buf, n);
        """
        sha256 = hashlib.sha256()
        with open(filepath, "rb") as f:
            while True:
                chunk = f.read(chunk_size)
                if not chunk:
                    break
                sha256.update(chunk)
        return sha256.hexdigest()
```

27. Line-by-Line Walkthrough: `crawlers/c_crawler.py`

The C crawler is our most important parser. It handles C and C++ files, which make up the vast majority of embedded codebases.

```python
"""C/C++ parser using Tree-sitter with regex fallback.

Architecture:
  PRIMARY:  Tree-sitter (fast, accurate, handles errors)
  FALLBACK: Regex (when Tree-sitter can't load the C grammar)

Why two methods?
  Tree-sitter requires the tree-sitter-c compiled grammar to be installed.
  On some systems (e.g., minimal Docker containers), this might not be
  available. The regex fallback ensures we always produce SOME output,
  even if it's less accurate than Tree-sitter.

  Tree-sitter advantage: Understands scope, nesting, preprocessor directives.
  Regex disadvantage: Can be fooled by comments, strings, nested braces.
"""

import logging
import re
from pathlib import Path
from typing import Optional

from codecrawler.core.types import (
    FileInfo, ParseResult, FunctionDef, StructDef,
    VariableDef, CallEdge, IncludeEdge, MacroDef,
)
from codecrawler.crawlers.base import BaseCrawler

logger = logging.getLogger(__name__)

# ─── REGEX PATTERNS (for fallback mode) ──────────────────────────────────

# Matches C function definitions like: int wifi_connect(const char *ssid) {
# Breakdown of the regex:
#   ^                         Start of line
#   ([\w\s\*]+?)              Return type (int, void, static int, struct foo *)
#   \s+                       Whitespace
#   (\w+)                     Function name (wifi_connect)
#   \s*                       Optional whitespace
#   \(([^)]*)\)               Parameters in parentheses
#   \s*                       Optional whitespace
#   \{                        Opening brace
FUNC_RE = re.compile(
    r"^([\w\s\*]+?)\s+(\w+)\s*\(([^)]*)\)\s*\{",
    re.MULTILINE,
)

# Matches #include statements:  #include "wifi.h"  OR  #include <stdio.h>
INCLUDE_RE = re.compile(
    r'^\s*#\s*include\s+([<"])([^>"]+)[>"]',
    re.MULTILINE,
)

# Matches function calls:  function_name(anything)
# But NOT control flow:    if(x), while(x), for(x), switch(x), return(x)
CALL_RE = re.compile(
    r'\b(?!if|while|for|switch|return|sizeof|typeof)\b(\w+)\s*\(',
```

```
    re.MULTILINE,
)

# Matches #define:  #define NAME VALUE
DEFINE_RE = re.compile(
    r'^\s*#\s*define\s+(\w+)(?:\s+(.+))?$',
    re.MULTILINE,
)

# Matches struct definitions:  struct name { ... };
STRUCT_RE = re.compile(
    r'\bstruct\s+(\w+)\s*\{([^}]*)\};',
    re.DOTALL,
)


class CCrawler(BaseCrawler):
    """Parse C/C++ source files into structured data.

    Usage:
        crawler = CCrawler()
        result = crawler.parse(file_info)
        # result.functions = list of FunctionDef
        # result.calls = list of CallEdge
        # ...
    """

    def __init__(self):
        self._ts_parser = None
        self._ts_language = None
        self._try_load_tree_sitter()

    def _try_load_tree_sitter(self):
        """Attempt to load the Tree-sitter C grammar.

        This might fail if tree-sitter-c is not installed.
        Not a fatal error — we fall back to regex.
        """
        try:
            import tree_sitter_c as tsc
            from tree_sitter import Language, Parser

            c_language = Language(tsc.language())
            parser = Parser(c_language)

            self._ts_parser = parser
            self._ts_language = c_language
            logger.info("Tree-sitter C grammar loaded successfully")
        except (ImportError, Exception) as e:
            logger.warning("Tree-sitter unavailable, using regex fallback: %s",
e)

    @property
    def supported_languages(self) -> list[str]:
        return ["c", "cpp"]
```

```
    def can_parse(self, file_info: FileInfo) -> bool:
        return file_info.language in self.supported_languages

    def parse(self, file_info: FileInfo) -> ParseResult:
        """Parse a C/C++ file and extract all structural information.

        Strategy:
        1. Read the file as bytes (Tree-sitter needs bytes, not str)
        2. If Tree-sitter is available, use it (more accurate)
        3. Otherwise, fall back to regex (less accurate but always works)
        """
        source = Path(file_info.path).read_text(encoding="utf-8",
errors="replace")

        if self._ts_parser:
            return self._parse_tree_sitter(file_info, source)
        else:
            return self._parse_regex(file_info, source)

    # ── TREE-SITTER PARSING ──────────────────────────────────────────

    def _parse_tree_sitter(self, file_info: FileInfo, source: str) ->
ParseResult:
        """Parse using Tree-sitter for accurate AST-based extraction."""
        tree = self._ts_parser.parse(source.encode("utf-8"))
        root = tree.root_node

        functions = []
        structs = []
        variables = []
        calls = []
        includes = []
        macros = []

        # Walk every node in the AST
        for node in self._walk(root):
            if node.type == "function_definition":
                func = self._extract_function(node, source)
                if func:
                    functions.append(func)
                    # Extract calls within this function's body
                    body = node.child_by_field_name("body")
                    if body:
                        func_calls = self._extract_calls(body, source,
func.name)
                        calls.extend(func_calls)

            elif node.type == "struct_specifier" and node.parent.type !=
"parameter_declaration":
                struct = self._extract_struct(node, source)
                if struct:
                    structs.append(struct)

            elif node.type == "declaration" and node.parent.type ==
"translation_unit":
                # Top-level declaration = global variable
```

```
                var = self._extract_variable(node, source)
                if var:
                    variables.append(var)

            elif node.type == "preproc_include":
                inc = self._extract_include(node, source)
                if inc:
                    includes.append(inc)

            elif node.type == "preproc_def":
                macro = self._extract_macro(node, source)
                if macro:
                    macros.append(macro)

        return ParseResult(
            file_info=file_info,
            functions=functions,
            structs=structs,
            variables=variables,
            calls=calls,
            includes=includes,
            macros=macros,
        )

    def _walk(self, node):
        """Depth-first walk of the AST tree.

        This is a GENERATOR (uses yield). It visits every node exactly once.
        Memory-efficient: doesn't build a list of all nodes.

        For a file with 1000 AST nodes, this creates 0 extra data structures —
        it just maintains the call stack during recursion.
        """
        yield node
        for child in node.children:
            yield from self._walk(child)

    def _extract_function(self, node, source: str) -> Optional[FunctionDef]:
        """Extract a FunctionDef from a function_definition AST node.

        node structure:
          function_definition
            ├── type: (primitive_type)           → return type ("int")
            ├── declarator: (function_declarator)
            │   ├── declarator: (identifier)     → name ("wifi_connect")
            │   └── parameters: (parameter_list) → params
            └── body: (compound_statement)       → function body

        We navigate this tree to extract each piece.
        """
        declarator = node.child_by_field_name("declarator")
        if not declarator:
            return None

        # Find the function name (dig through pointer_declarator if present)
        name_node = self._find_name_node(declarator)
```

```
        if not name_node:
            return None

        name = source[name_node.start_byte:name_node.end_byte]

        # Full signature is the source text from function start to body start
        body = node.child_by_field_name("body")
        if body:
            sig = source[node.start_byte:body.start_byte].strip()
        else:
            sig = source[node.start_byte:node.end_byte].strip()

        # Compute cyclomatic complexity
        complexity = self._compute_complexity(body) if body else 1

        # Check for 'static' keyword
        is_static = any(
            child.type == "storage_class_specifier" and
            source[child.start_byte:child.end_byte] == "static"
            for child in node.children
        )

        return FunctionDef(
            name=name,
            signature=sig.replace("\n", " "),  # Single line
            start_line=node.start_point[0] + 1,  # Tree-sitter is 0-indexed,
we're 1-indexed
            end_line=node.end_point[0] + 1,
            complexity=complexity,
            is_static=is_static,
        )

    def _compute_complexity(self, body_node) -> int:
        """Count cyclomatic complexity by counting decision points.

        Walk all nodes in the function body.
        Each branching construct adds 1 to complexity.
        Base complexity = 1 (one path through the function).
        """
        complexity = 1  # Base
        decision_types = {
            "if_statement", "while_statement", "for_statement",
            "switch_statement", "case_statement",
            "conditional_expression",  # ternary: x ? a : b
        }
        logical_ops = {"&&", "||"}

        for node in self._walk(body_node):
            if node.type in decision_types:
                complexity += 1
            elif node.type == "binary_expression":
                # Check for && and ||
                # operator is the second child: (left operator right)
                if len(node.children) >= 2:
                    op = node.children[1]
                    op_text = op.type  # "&&" or "||"
```

```
                        if op_text in logical_ops:
                            complexity += 1

        return complexity
```

28. DuckDB Operations Cookbook

This chapter provides copy-paste SQL recipes for every operation Code Crawler performs.

28.1 Schema Creation Recipe

```
-- Run this once when creating a new database

-- Core entity tables
CREATE TABLE IF NOT EXISTS Directory (
    id          BIGINT PRIMARY KEY DEFAULT nextval('dir_seq'),
    path        TEXT UNIQUE NOT NULL,
    parent_id   BIGINT REFERENCES Directory(id),
    tier        INT DEFAULT 3 CHECK (tier BETWEEN 0 AND 3),
    is_custom   BOOL DEFAULT FALSE
);

CREATE TABLE IF NOT EXISTS File (
    id              BIGINT PRIMARY KEY DEFAULT nextval('file_seq'),
    directory_id    BIGINT REFERENCES Directory(id),
    path            TEXT UNIQUE NOT NULL,
    filename        TEXT,
    extension       TEXT,
    language        TEXT,
    hash            TEXT,
    last_modified   TIMESTAMP,
    last_indexed    TIMESTAMP DEFAULT current_timestamp,
    loc             INT DEFAULT 0,
    is_custom       BOOL DEFAULT FALSE,
    embedding       FLOAT[384]
);

CREATE TABLE IF NOT EXISTS Function (
    id              BIGINT PRIMARY KEY DEFAULT nextval('func_seq'),
    name            TEXT NOT NULL,
    qualified_name  TEXT,
    signature       TEXT,
    start_line      INT,
    end_line        INT,
    complexity      INT DEFAULT 1,
    body_hash       TEXT,
    doc_comment     TEXT,
    summary         TEXT,
    is_static       BOOL DEFAULT FALSE,
    return_type     TEXT,
    embedding       FLOAT[384]
);
```

```
-- Edge tables (relationships between entities)
CREATE TABLE IF NOT EXISTS contains_func (
    file_id     BIGINT REFERENCES File(id),
    func_id     BIGINT REFERENCES Function(id),
    PRIMARY KEY (file_id, func_id)
);

CREATE TABLE IF NOT EXISTS calls (
    caller_id       BIGINT REFERENCES Function(id),
    callee_id       BIGINT REFERENCES Function(id),
    call_site_line  INT,
    is_indirect     BOOL DEFAULT FALSE,
    PRIMARY KEY (caller_id, callee_id, call_site_line)
);

CREATE TABLE IF NOT EXISTS includes_file (
    source_id   BIGINT REFERENCES File(id),
    target_id   BIGINT REFERENCES File(id),
    is_system   BOOL DEFAULT FALSE,
    PRIMARY KEY (source_id, target_id)
);
```

28.2 Insert Recipes

```
-- Insert a new file
INSERT INTO File (path, filename, extension, language, hash, loc, is_custom)
VALUES ('src/wifi_hal.c', 'wifi_hal.c', '.c', 'c', 'abc123...', 500, true)
RETURNING id;
-- RETURNING id gives us the auto-generated ID

-- Insert a function
INSERT INTO Function (name, signature, start_line, end_line, complexity)
VALUES ('wifi_connect', 'int wifi_connect(const char *ssid)', 14, 35, 4)
RETURNING id;

-- Link function to file
INSERT INTO contains_func (file_id, func_id) VALUES (100, 1);

-- Insert a call edge
INSERT INTO calls (caller_id, callee_id, call_site_line)
VALUES (1, 2, 21)
ON CONFLICT DO NOTHING;  -- Skip if this exact edge already exists

-- Batch insert (much faster than individual inserts)
INSERT INTO Function (name, signature, start_line, end_line, complexity) VALUES
    ('wifi_connect', 'int wifi_connect(...)', 14, 35, 4),
    ('wifi_disconnect', 'void wifi_disconnect(void)', 37, 42, 1),
    ('wifi_scan', 'int wifi_scan(...)', 44, 60, 2);
```

28.3 Query Recipes

```sql
-- ═══ FIND FUNCTIONS ═══

-- Find by name
SELECT * FROM Function WHERE name = 'wifi_connect';

-- Find by name pattern
SELECT * FROM Function WHERE name LIKE 'wifi_%';

-- Find most complex functions (top 10)
SELECT f.name, f.complexity, fi.path
FROM Function f
JOIN contains_func cf ON f.id = cf.func_id
JOIN File fi ON cf.file_id = fi.id
ORDER BY f.complexity DESC
LIMIT 10;


-- ═══ CALL HIERARCHY ═══

-- What does function X call? (direct callees)
SELECT f2.name AS callee, c.call_site_line
FROM calls c
JOIN Function f2 ON c.callee_id = f2.id
WHERE c.caller_id = (SELECT id FROM Function WHERE name = 'wifi_connect');

-- What calls function X? (direct callers)
SELECT f1.name AS caller, c.call_site_line
FROM calls c
JOIN Function f1 ON c.caller_id = f1.id
WHERE c.callee_id = (SELECT id FROM Function WHERE name = 'wifi_connect');

-- Full transitive call chain (recursive CTE)
WITH RECURSIVE call_chain AS (
    SELECT id, name, 0 AS depth
    FROM Function WHERE name = 'main'

    UNION ALL

    SELECT f.id, f.name, cc.depth + 1
    FROM call_chain cc
    JOIN calls c ON cc.id = c.caller_id
    JOIN Function f ON c.callee_id = f.id
    WHERE cc.depth < 10
)
SELECT DISTINCT name, MIN(depth) AS min_depth
FROM call_chain
GROUP BY name
ORDER BY min_depth;


-- ═══ FILE ANALYSIS ═══

-- Functions per file (most populated files)
SELECT fi.path, COUNT(cf.func_id) AS func_count, SUM(f.complexity) AS
total_complexity
FROM File fi
```

```sql
JOIN contains_func cf ON fi.id = cf.file_id
JOIN Function f ON cf.func_id = f.id
GROUP BY fi.path
ORDER BY func_count DESC;

-- Files that include a specific header
SELECT fi.path
FROM includes_file inc
JOIN File fi ON inc.source_id = fi.id
WHERE inc.target_id = (SELECT id FROM File WHERE filename = 'wifi_hal.h');


-- ═══ STATISTICS ═══

-- Index overview
SELECT
    (SELECT COUNT(*) FROM File) AS total_files,
    (SELECT COUNT(*) FROM Function) AS total_functions,
    (SELECT COUNT(*) FROM calls) AS total_call_edges,
    (SELECT COUNT(*) FROM includes_file) AS total_includes,
    (SELECT SUM(loc) FROM File) AS total_lines_of_code;

-- Language distribution
SELECT language, COUNT(*) AS file_count, SUM(loc) AS total_loc
FROM File
GROUP BY language
ORDER BY total_loc DESC;

-- Complexity distribution
SELECT
    CASE
        WHEN complexity <= 2 THEN 'simple (1-2)'
        WHEN complexity <= 5 THEN 'moderate (3-5)'
        WHEN complexity <= 10 THEN 'complex (6-10)'
        ELSE 'very complex (11+)'
    END AS category,
    COUNT(*) AS function_count
FROM Function
GROUP BY category
ORDER BY MIN(complexity);


-- ═══ IMPACT ANALYSIS ═══

-- "If I change wifi_connect, what functions are affected?"
-- (Reverse transitive closure — who ultimately depends on this function)
WITH RECURSIVE impact AS (
    SELECT id, name, 0 AS depth
    FROM Function WHERE name = 'wifi_connect'

    UNION ALL

    SELECT f.id, f.name, i.depth + 1
    FROM impact i
    JOIN calls c ON i.id = c.callee_id  -- note: callee_id, reverse direction!
    JOIN Function f ON c.caller_id = f.id
```

```
    WHERE i.depth < 8
)
SELECT DISTINCT name, MIN(depth) AS distance
FROM impact
WHERE name != 'wifi_connect'
GROUP BY name
ORDER BY distance;


-- ═══ DEAD CODE DETECTION ═══

-- Functions with zero callers (potentially dead code)
SELECT f.name, fi.path
FROM Function f
JOIN contains_func cf ON f.id = cf.func_id
JOIN File fi ON cf.file_id = fi.id
WHERE f.id NOT IN (SELECT callee_id FROM calls)
  AND f.name != 'main'
ORDER BY fi.path, f.name;
```

---

29. Yocto Project Integration Deep Dive

29.1 Why Yocto Is Special

A Yocto project can have **hundreds of thousands** of source files. Most of them are upstream packages (busybox, systemd, glibc) that you never modify. Without smart tiering, you'd waste time indexing code you don't care about.

Code Crawler's Yocto analyzer understands the project structure:

```
my-yocto/
├── meta-poky/              ← OpenEmbedded core (i0: ignore)
├── meta-openembedded/      ← Community recipes (i0: ignore)
├── meta-raspberrypi/       ← BSP layer (i1: stub)
├── meta-my-product/        ← YOUR CUSTOM LAYER (i3: full index!)
│   ├── recipes-wifi/
│   │   └── wifi-daemon/
│   │       ├── wifi-daemon_1.0.bb    ← Recipe
│   │       └── files/
│   │           ├── src/main.c        ← YOUR CODE
│   │           ├── src/wifi_hal.c    ← YOUR CODE
│   │           └── src/config.c      ← YOUR CODE
│   └── recipes-core/
│       └── images/
│           └── my-image.bb
├── build/
│   ├── conf/
│   │   ├── bblayers.conf             ← Which layers are active
│   │   └── local.conf                ← Build configuration
│   └── tmp/
│       └── work/
```

```
│           └── *.c                    ← Build artifacts (i0: ignore)
└── downloads/                          ← Cached tarballs (i0: ignore)
```

29.2 How We Detect Yocto

```
YOCTO_SIGNATURES = {
    # Glob patterns to look for
    "meta-*/conf/layer.conf": "glob",      # Every Yocto layer has this file
    "build/conf/bblayers.conf": "exact",   # Build configuration
    "oe-init-build-env": "exact",          # Setup script
}

# Score: each found file adds points
# If total score > threshold → this is a Yocto project
```

29.3 What the Yocto Analyzer Extracts

```
# From bblayers.conf:
layers = [
    {"name": "meta-poky",         "path": "/yocto/meta-poky",
"priority": 5},
    {"name": "meta-oe",           "path": "/yocto/meta-oe",
"priority": 6},
    {"name": "meta-my-product",   "path": "/yocto/meta-my-product",
"priority": 10},
]
# Custom layers have higher priority → higher tier

# From local.conf:
machine = "raspberrypi4"
distro = "poky"
extra_image_features = ["debug-tweaks", "ssh-server-openssh"]

# From recipes:
recipes = [
    {"name": "wifi-daemon", "version": "1.0", "layer": "meta-my-product",
     "src_uri": "file://src/main.c file://src/wifi_hal.c"},
]
```

29.4 Tier Assignment for Yocto

```
Layer                    | Priority | Is Custom? | Tier
————————————————————————|————————|————————————|———————
meta-poky                | 5        | No         | i0 (ignore)
meta-openembedded        | 6        | No         | i0 (ignore)
meta-raspberrypi         | 7        | No         | i1 (stub)
meta-my-product          | 10       | Yes        | i3 (full)
build/                   | —        | No         | i0 (ignore)
downloads/               | —        | No         | i0 (ignore)
```

This means for a 500K-file Yocto project, we might only deeply index 200 files in your custom layer — saving 99.96% of processing time!

---

30. Incremental Indexing — How We Avoid Re-Parsing

30.1 The Problem

Indexing a large codebase takes time. If you change one file and re-run `codecrawler index`, you don't want to parse all 10,000 files again.

30.2 The Solution: Content Hashing

```
# During indexing:
for file_info in discovered_files:
    # 1. Compute SHA-256 hash of file content
    current_hash = hashlib.sha256(file.read_bytes()).hexdigest()

    # 2. Check if this file is already in the database with the same hash
    stored = db.execute(
        "SELECT hash, last_indexed FROM File WHERE path = ?",
        [str(file_info.path)]
    ).fetchone()

    if stored and stored[0] == current_hash:
        # File hasn't changed! Skip re-parsing.
        logger.debug("Unchanged: %s", file_info.path)
        continue

    # 3. File is new or changed → parse it
    result = crawler.parse(file_info)

    # 4. Update the database
    if stored:
        # File exists but changed → update
        db.execute("DELETE FROM Function WHERE id IN (SELECT func_id FROM
contains_func WHERE file_id = ?)", [stored_id])
        db.execute("DELETE FROM contains_func WHERE file_id = ?", [stored_id])
        db.execute("DELETE FROM calls WHERE caller_id IN (...)", [stored_id])
        # ... then re-insert with new data
    else:
        # New file → insert
        db.execute("INSERT INTO File (...) VALUES (...)")

    # 5. Update the stored hash
    db.execute("UPDATE File SET hash = ?, last_indexed = current_timestamp
WHERE path = ?",
               [current_hash, str(file_info.path)])
```

30.3 Handling Deletions

```
# After parsing all files, check for files that exist in the DB but not on disk
stored_files = db.execute("SELECT path FROM File").fetchall()
discovered_paths = {str(f.path) for f in discovered_files}

for (stored_path,) in stored_files:
    if stored_path not in discovered_paths:
        # File was deleted! Remove from database
        logger.info("Removed: %s", stored_path)
        file_id = db.execute("SELECT id FROM File WHERE path = ?",
[stored_path]).fetchone()[0]
        db.execute("DELETE FROM calls WHERE caller_id IN (SELECT func_id FROM
contains_func WHERE file_id = ?)", [file_id])
        db.execute("DELETE FROM contains_func WHERE file_id = ?", [file_id])
        db.execute("DELETE FROM File WHERE id = ?", [file_id])
```

30.4 The `--force` Flag

```
# Normal: only re-parse changed files
codecrawler index --root ./my-project

# Force: re-parse everything (useful after upgrading Code Crawler)
codecrawler index --root ./my-project --force
```

---

31. Error Handling Philosophy

31.1 Never Crash the Pipeline

The #1 rule: **one bad file should never crash the entire indexing run.**

```
# BAD: One exception kills everything
for file_info in files:
    result = crawler.parse(file_info)  # If this throws, game over

# GOOD: Isolate each file's processing
for file_info in files:
    try:
        result = crawler.parse(file_info)
        results.append(result)
    except Exception:
        logger.exception("Failed to parse %s, skipping", file_info.path)
        # Continue with the next file
```

31.2 Never Crash Event Handlers

Same principle for the event bus:

```
# BAD: One handler crash kills all handlers
for handler in self._handlers[topic]:
    handler(data)  # If handler[0] throws, handler[1] never runs

# GOOD: Isolate each handler
for handler in self._handlers[topic]:
    try:
        handler(data)
    except Exception:
        logger.exception("Event handler failed for topic '%s'", topic)
        # Continue with the next handler
```

31.3 Graceful Degradation

If Tree-sitter fails, fall back to regex. If regex fails, return an empty ParseResult (file exists but no functions extracted). If the database can't create HNSW indexes, skip vector search (keyword search still works).

```
# Example: Tree-sitter → Regex → Empty
def parse(self, file_info):
    try:
        return self._parse_tree_sitter(file_info)
    except Exception:
        logger.warning("Tree-sitter failed for %s, trying regex",
file_info.path)

    try:
        return self._parse_regex(file_info)
    except Exception:
        logger.warning("Regex failed for %s, returning empty result",
file_info.path)

    return ParseResult(file_info=file_info)  # Empty but valid
```

32. Performance Considerations

32.1 Memory

Component	Memory Usage	Why
Tree-sitter parser	~10MB	Compiled C parser + grammar tables
One file's AST	~2-5× file size	Tree nodes have metadata
DuckDB connection	~50MB base	In-process database engine
Embedding model	~90MB	Neural network weights
100K embeddings	~146MB	384 floats × 4 bytes × 100K
Total (indexing)	~300-500MB	Peaks during embedding generation

Component	Memory Usage	Why
Total (MCP serving)	~200-300MB	No new embeddings generated

32.2 Speed

Operation	Time (1K files)	Time (100K files)
File discovery	~100ms	~5s
Tree-sitter parsing	~2s	~3min
Database insertion	~500ms	~30s
Embedding generation	~30s	~50min
Priority scoring	~100ms	~10s
Manifest building	~200ms	~20s
Total (no embeddings)	**~3s**	**~4min**
Total (with embeddings)	**~33s**	**~55min**

32.3 Optimization Techniques

Batch insertion: Insert 1000 rows at once instead of 1000 individual INSERTs:

```
# SLOW: one insert per function
for func in functions:
    conn.execute("INSERT INTO Function VALUES (?,...)", [func.name, ...])

# FAST: batch insert
conn.executemany("INSERT INTO Function VALUES (?,...)",
                 [(f.name, ...) for f in functions])
```

Batch embedding: Generate embeddings for 32 functions at once instead of one at a time:

```
# SLOW: one embedding per function
for func in functions:
    embedding = model.encode(func.signature)

# FAST: batch embedding (GPU-friendly)
texts = [f"{func.name}: {func.signature}" for func in functions]
embeddings = model.encode(texts, batch_size=32)
```

Lazy loading: Don't load the embedding model until someone actually needs embeddings:

```
class VectorSearch:
    def __init__(self):
        self._model = None  # Not loaded yet
```

```
    @property
    def model(self):
        if self._model is None:
            self._model = SentenceTransformer(...)  # Load on first use
        return self._model
```

---

33. Common Pitfalls and How to Avoid Them

33.1 PITFALL: Circular Includes

```
// a.h
#include "b.h"

// b.h
#include "a.h"
```

Problem: Infinite loop during include resolution. **Solution**: Track visited files during traversal:

```
def resolve_includes(file_id, visited=None):
    if visited is None:
        visited = set()
    if file_id in visited:
        return  # Already visited — break the cycle!
    visited.add(file_id)

    for included_id in get_includes(file_id):
        resolve_includes(included_id, visited)
```

33.2 PITFALL: Function Pointer Calls

```
typedef int (*handler_fn)(int data);
handler_fn handlers[] = {wifi_handler, bt_handler, log_handler};
handlers[msg_type](data);  // Which function is being called?
```

Problem: Tree-sitter sees `handlers[msg_type](data)` as a call expression, but the callee is a subscript expression, not an identifier. **Solution**: Mark as indirect call and record what we can:

```
CallEdge(caller="process_message", callee="handlers[msg_type]",
         call_site_line=42, is_indirect=True)
```

33.3 PITFALL: Preprocessor Conditionals

```
#ifdef CONFIG_WIFI
int wifi_connect(const char *ssid) { ... }
#else
int wifi_connect(const char *ssid) { return -ENOTSUP; }
#endif
```

Problem: Two definitions of the same function. Which one is active? **Solution**: Check the BuildConfig table:

```
if config.get("CONFIG_WIFI", False):
    # Only index the #ifdef branch
else:
    # Only index the #else branch
```

Currently (v4), we index BOTH branches and tag them with their guard. v5 will do proper conditional compilation awareness.

33.4 PITFALL: Macro-Generated Functions

```
#define DEFINE_HANDLER(name) \
    int name##_handler(int data) { return process_##name(data); }

DEFINE_HANDLER(wifi)    // Generates: int wifi_handler(int data) { ... }
DEFINE_HANDLER(bt)      // Generates: int bt_handler(int data) { ... }
```

Problem: Tree-sitter sees the macro call, not the generated function. **Solution**: v4 records the macro definitions. v5 will attempt macro expansion for common patterns.

33.5 PITFALL: Very Long Files

```
// kernel/sched/core.c — over 11,000 lines with 100+ functions!
```

Problem: Tree-sitter handles it fine, but the resulting ParseResult is huge. **Solution**: Process functions in batches and limit manifest size:

```
# If file has > 50 functions, split the manifest into chunks
if len(parse_result.functions) > 50:
    chunks = []
    for i in range(0, len(parse_result.functions), 50):
        chunk = parse_result.functions[i:i+50]
        chunks.append(build_manifest_chunk(chunk))
```

---

34. What Changes in v5

34.1 Planned v5 Features

Feature	v4 (current)	v5 (planned)
LLM Integration	Placeholder	Local LLM for summarization and classification
Self-Tuning Weights	Fixed weights	Adjust weights based on which functions users actually query
Swarm Indexing	Single-process	Distribute indexing across multiple machines
Git-Aware Graphs	Snapshot only	Track how the call graph evolves over time
IPC Edge Detection	Not implemented	Detect D-Bus, Netlink, socket, pipe IPC between processes
Proactive AI	Basic pattern match	LLM-powered vulnerability and quality analysis
Macro Expansion	Record only	Expand common macro patterns to find hidden functions
Multi-Language Linking	Separate crawlers	Cross-language call detection (C → Python via FFI)
Live Index	File watcher only	IDE integration for instant updates on save

34.2 Self-Tuning Weight Algorithm (v5)

```
# Track which functions users actually query
query_log = [
    {"query": "wifi connection", "clicked_function": "wifi_connect",
"timestamp": "..."},
    {"query": "scan results",   "clicked_function": "wifi_scan",
"timestamp": "..."},
    ...
]

# Analyze: which dimension most correlates with user interest?
# If users mostly click functions with high centrality → increase centrality
weight
# If users mostly click recently modified functions → increase recency weight

# Gradient-based adjustment:
for dimension in ["tier", "usage", "centrality", "build", "runtime",
"recency"]:
    correlation = correlate(
        scores=[func.dimension_score for func in queried_functions],
        clicked=[1 if func.was_clicked else 0 for func in queried_functions]
    )
    if correlation > 0.5:
        weights[dimension] *= 1.05  # Increase by 5%
    elif correlation < -0.2:
        weights[dimension] *= 0.95  # Decrease by 5%

# Normalize so weights still sum to 1.0
total = sum(weights.values())
weights = {k: v/total for k, v in weights.items()}
```

Final Notes

How Long Should Indexing Take?

```
Small project (100 files):     5-10 seconds
Medium project (1K files):     30-60 seconds
Large project (10K files):     5-10 minutes
Yocto project (100K+ files):   15-30 minutes (with tiering: 2-5 minutes)
```

How Big Is the Database?

```
100 files:    ~2 MB
1K files:     ~20 MB
10K files:    ~200 MB
100K files:   ~2 GB (mostly embeddings)
Without embeddings: ~200 MB for 100K files
```

Where to Get Help

- **This study guide**: Everything you need to understand the system
- **Source code**: Well-commented, every file has a module docstring
- `codecrawler status`: Shows current index statistics
- **DuckDB CLI**: `duckdb .codecrawler/index.duckdb` for direct queries

End of Code Crawler v4 Complete Technical Study Guide

24 chapters in Part I (System Overview) Chapters 14–24 in Part II (Deep-Dive Fundamentals) Chapters 25–34 in Part III (Source Code Walkthroughs & Advanced Topics)

Written for embedded Linux C developers who want to understand the full system.

 Code Crawler v4 — *Index Smart, Query Fast, Ship Confident*

PART IV — HANDS-ON EXERCISES, REAL-WORLD SCENARIOS & ARCHITECTURE DECISIONS

This part provides practical exercises to cement your understanding, real-world debugging scenarios, architecture decision records explaining *why* we made each design choice, and exhaustive data flow trace tables.

---

35. Hands-On Exercises

35.1 Exercise 1: Trace a Function Through the Pipeline

Task: For the following C code, manually walk through every pipeline stage and write down what each stage produces.

```c
// file: src/led_controller.c

#include "led_hal.h"
#include <stdbool.h>

#define MAX_LEDS 8
#define LED_PIN_BASE 16

static int led_states[MAX_LEDS];

int led_init(void) {
    for (int i = 0; i < MAX_LEDS; i++) {
        led_states[i] = 0;
        hal_gpio_init(LED_PIN_BASE + i, GPIO_OUTPUT);
    }
    return 0;
}

int led_set(int id, bool on) {
    if (id < 0 || id >= MAX_LEDS)
        return -EINVAL;
    led_states[id] = on ? 1 : 0;
    return hal_gpio_write(LED_PIN_BASE + id, led_states[id]);
}

int led_get(int id) {
    if (id < 0 || id >= MAX_LEDS)
        return -EINVAL;
    return led_states[id];
}
```

Expected Answers:

Stage 1: File Discovery → FileInfo

```
FileInfo(
    path="src/led_controller.c",
    language="c",       # from LANGUAGE_MAP: ".c" → "c"
    size=487,           # file size in bytes
    hash="a1b2c3...",   # SHA-256 of file content
    last_modified=None  # set later from git/mtime
)
```

Stage 3: Tier Classification → TierClassification

```
TierClassification(
    path="src/",
    tier=3,               # i3: full index (it's in src/, looks like custom
code)
    confidence=0.8,
    reason="Custom source directory"
)
```

Stage 4: Parsing → ParseResult

```
ParseResult(
    file_info=<FileInfo above>,
    functions=[
        FunctionDef(
            name="led_init",
            signature="int led_init(void)",
            start_line=11, end_line=17,
            complexity=2,          # 1 base + 1 for the 'for' loop
            is_static=False,
            return_type="int"
        ),
        FunctionDef(
            name="led_set",
            signature="int led_set(int id, bool on)",
            start_line=19, end_line=24,
            complexity=4,          # 1 base + 1 if + 1 || + 1 ternary
            is_static=False,
            return_type="int"
        ),
        FunctionDef(
            name="led_get",
            signature="int led_get(int id)",
            start_line=26, end_line=30,
            complexity=3,          # 1 base + 1 if + 1 ||
            is_static=False,
            return_type="int"
        ),
    ],
    structs=[],
    variables=[
        VariableDef(
            name="led_states",
            var_type="int[MAX_LEDS]",
            is_global=False,  # static file scope
            is_static=True,
            is_const=False,
            line=9
        ),
    ],
    calls=[
        CallEdge(caller="led_init", callee="hal_gpio_init", call_site_line=14),
        CallEdge(caller="led_set", callee="hal_gpio_write", call_site_line=23),
    ],
```

```
    includes=[
        IncludeEdge(source_path="src/led_controller.c",
target_path="led_hal.h", is_system=False),
        IncludeEdge(source_path="src/led_controller.c",
target_path="stdbool.h", is_system=True),
    ],
    macros=[
        MacroDef(name="MAX_LEDS", value="8", is_config_guard=False, line=4),
        MacroDef(name="LED_PIN_BASE", value="16", is_config_guard=False,
line=5),
    ],
)
```

Stage 5: Priority Scoring → PriorityScoreResult

For `led_set` (assuming it's the most-called function):

```
PriorityScoreResult(
    func_id=2,              # auto-generated ID
    tier_score=1.0,          # i3 → 1.0
    usage_score=0.75,        # Called by 3 functions out of max 4
    centrality_score=0.3,    # Some centrality but not a bridge node
    build_score=1.0,         # No #ifdef guard, always compiled
    runtime_score=0.0,       # No telemetry data yet
    recency_score=0.9,       # Modified recently
    composite_score=0.71,    # Weighted sum
)
```

Stage 6: IndexManifest

```
{
  "file": {"path": "src/led_controller.c", "language": "c", "loc": 30},
  "functions": [
    {"name": "led_init", "sig": "int led_init(void)", "cx": 2},
    {"name": "led_set", "sig": "int led_set(int id, bool on)", "cx": 4},
    {"name": "led_get", "sig": "int led_get(int id)", "cx": 3}
  ],
  "calls": [
    {"from": "led_init", "to": "hal_gpio_init"},
    {"from": "led_set", "to": "hal_gpio_write"}
  ],
  "includes": ["led_hal.h"],
  "globals": [{"name": "led_states", "type": "int[8]", "static": true}]
}
```

Token estimate: ~180 tokens (vs ~500 for raw source). 2.8× compression.

35.2 Exercise 2: Write a SQL Query

Task: Write a SQL query to find all functions that:

1. Have complexity > 5
2. Are in files with the `.c` extension
3. Have at least 3 callers
4. Are NOT static

Answer:

```
SELECT
    f.name,
    f.complexity,
    fi.path,
    COUNT(c.caller_id) AS caller_count
FROM Function f
JOIN contains_func cf ON f.id = cf.func_id
JOIN File fi ON cf.file_id = fi.id
LEFT JOIN calls c ON f.id = c.callee_id
WHERE f.complexity > 5
  AND fi.extension = '.c'
  AND f.is_static = FALSE
GROUP BY f.id, f.name, f.complexity, fi.path
HAVING COUNT(c.caller_id) >= 3
ORDER BY f.complexity DESC;
```

Explanation for C devs: Think of this as a multi-step filter:

1. `JOIN` chains: Follow pointers from Function → File (through contains_func)
2. `LEFT JOIN calls`: Count incoming edges (callers)
3. `WHERE`: Filter by complexity, extension, and static keyword
4. `GROUP BY`: Since we're counting callers, SQL needs to know how to group
5. `HAVING`: Like WHERE, but works on aggregated results (COUNT)
6. `ORDER BY`: Sort output

35.3 Exercise 3: Implement a Simple Crawler

Task: Implement a `MakefileCrawler` that extracts targets from Makefiles.

```
"""Makefile parser — extracts build targets as 'functions'."""

import re
from codecrawler.crawlers.base import BaseCrawler
from codecrawler.core.types import FileInfo, ParseResult, FunctionDef, CallEdge

# Matches Makefile targets:  target: prerequisites
#   wifi-daemon: main.o wifi.o config.o
TARGET_RE = re.compile(r'^(\S+)\s*:\s*(.*)$', re.MULTILINE)

# Matches $(call ...) or $(shell ...)
CALL_RE = re.compile(r'\$\((?:call|shell)\s+(\S+)', re.MULTILINE)
```

```python
class MakefileCrawler(BaseCrawler):
    """Parse Makefiles to extract build targets and their dependencies."""

    @property
    def supported_languages(self) -> list[str]:
        return ["makefile"]

    def can_parse(self, file_info: FileInfo) -> bool:
        return file_info.language == "makefile"

    def parse(self, file_info: FileInfo) -> ParseResult:
        source = file_info.path.read_text()

        functions = []  # We model targets as "functions"
        calls = []      # Dependencies become "call edges"

        for i, match in enumerate(TARGET_RE.finditer(source)):
            target_name = match.group(1)
            prerequisites = match.group(2).strip()

            # Skip pattern rules (%.o: %.c) — they're templates, not concrete
targets
            if '%' in target_name:
                continue

            # Skip internal targets starting with .
            if target_name.startswith('.'):
                continue

            line_num = source[:match.start()].count('\n') + 1

            functions.append(FunctionDef(
                name=target_name,
                signature=f"{target_name}: {prerequisites}",
                start_line=line_num,
                end_line=line_num,
                complexity=1 + len(prerequisites.split()),  # More deps = more
complex
            ))

            # Each prerequisite is a "call" to another target
            for prereq in prerequisites.split():
                if prereq and not prereq.startswith('$'):
                    calls.append(CallEdge(
                        caller=target_name,
                        callee=prereq,
                        call_site_line=line_num,
                    ))

        return ParseResult(
            file_info=file_info,
            functions=functions,
            calls=calls,
        )
```

What this teaches: The universal DTO pattern. Even Makefiles can be modeled as functions (targets) and calls (dependencies). The core engine doesn't know or care that these are Makefile targets — it stores them in the same tables and computes priority scores on them just like C functions.

35.4 Exercise 4: Calculate Betweenness Centrality

Task: Given this call graph, calculate the betweenness centrality for each function by hand.

```
main → dispatch → handler_wifi → hal_connect
main → dispatch → handler_bt → hal_connect
main → dispatch → handler_log
main → init
```

Step-by-step:

1. **List all pairs of nodes** (that have a path between them): There are 7 functions. All pairs where a path exists: C(7,2) = 21 pairs.
2. **For each pair, find all shortest paths**:
   - main↔hal_connect: 2 shortest paths (via handler_wifi, via handler_bt) Both go through `dispatch`. Neither goes through `init` or `handler_log`.
   - main↔handler_wifi: 1 path (main→dispatch→handler_wifi) Goes through `dispatch`.
   - etc.
3. **Count how many shortest paths go through each node**:

Node	Paths through it	Total shortest paths	Betweenness
main	0 (endpoint, not counted)	—	0.0
dispatch	12	15	12/15 = 0.80
handler_wifi	3	15	3/15 = 0.20
handler_bt	3	15	3/15 = 0.20
handler_log	0	15	0/15 = 0.00
hal_connect	0 (usually endpoint)	—	0.00
init	0	—	0.00

Interpretation: `dispatch` has the highest betweenness centrality (0.80). It's the bottleneck — if it breaks, most communication paths break. This matches intuition: `dispatch` is the router that connects `main` to all handlers.

35.5 Exercise 5: Design a DTO for a New Language

Task: You need to add Rust support. Design the additional DTOs needed for Rust-specific constructs that don't exist in C.

```
@dataclass(frozen=True)
class TraitDef:
    """A Rust trait definition (similar to an interface or abstract base
class).

    trait Connectable {
        fn connect(&self, address: &str) -> Result<(), Error>;
        fn disconnect(&self) -> Result<(), Error>;
    }
    """
    name: str                      # "Connectable"
    methods: list[str] = field(default_factory=list)  # ["fn connect(&self,
...)", ...]
    supertraits: list[str] = field(default_factory=list)  # ["Send", "Sync"]
    start_line: int = 0
    end_line: int = 0

@dataclass(frozen=True)
class ImplBlock:
    """A Rust impl block — associates methods with a type.

    impl Connectable for WifiDriver {
        fn connect(&self, address: &str) -> Result<(), Error> { ... }
    }
    """
    type_name: str = ""            # "WifiDriver"
    trait_name: str = ""           # "Connectable" (empty for inherent impl)
    methods: list[str] = field(default_factory=list)  # ["connect",
"disconnect"]
    start_line: int = 0
    end_line: int = 0

@dataclass(frozen=True)
class LifetimeAnnotation:
    """Tracks lifetime parameters for Rust borrow checker analysis.

    fn process<'a>(data: &'a [u8]) -> &'a str
    """
    function_name: str = ""
    lifetimes: list[str] = field(default_factory=list)  # ["'a", "'b"]

@dataclass(frozen=True)
class UnsafeBlock:
    """Tracks unsafe blocks — critical for security auditing.

    unsafe {
        ptr::write(addr, value);
    }
    """
    function_name: str = ""      # Which function contains this unsafe block
    start_line: int = 0
    end_line: int = 0
    operations: list[str] = field(default_factory=list)  # ["ptr::write",
"transmute"]
```

Key insight: The DTO design stays the same — frozen dataclasses with no behavior. Language-specific constructs are modeled as new DTOs, but they all flow through the same pipeline (EventBus, Storage, ManifestBuilder). The core engine never needs to change for a new language.

---

36. Real-World Debugging Scenarios

36.1 Scenario: "WiFi Stopped Working After the Last Commit"

Setup: A team member pushed a change to `wifi_config.c` and now WiFi fails to connect on the device. You need to figure out what changed and what's affected.

Using Code Crawler:

```
# Step 1: Find what functions are in wifi_config.c
codecrawler mcp  # (or use your AI assistant)
```

```
AI: "What functions are in wifi_config.c?"
MCP Response:
  - wifi_load_config() at line 45 (complexity: 6)
  - wifi_save_config() at line 78 (complexity: 4)
  - wifi_validate_ssid() at line 92 (complexity: 3)
  - wifi_get_default_config() at line 105 (complexity: 2)
```

```
AI: "Who calls wifi_load_config?"
```

```
-- What the MCP server runs internally:
WITH RECURSIVE callers AS (
    SELECT id, name, 0 AS depth
    FROM Function WHERE name = 'wifi_load_config'
    UNION ALL
    SELECT f.id, f.name, c.depth + 1
    FROM callers c
    JOIN calls ca ON c.id = ca.callee_id
    JOIN Function f ON ca.caller_id = f.id
    WHERE c.depth < 5
)
SELECT name, MIN(depth) FROM callers GROUP BY name ORDER BY MIN(depth);
```

```
Result:
  Depth 0: wifi_load_config
  Depth 1: wifi_init, wifi_reconnect
  Depth 2: main, wifi_monitor_thread
  Depth 3: (none)
```

```
BLAST RADIUS: 4 functions across 3 files
```

```
AI: "Show me the git diff for wifi_config.c"
```

```
-    config->timeout = atoi(value);
+    config->timeout = atoi(value) * 1000;  // Convert to milliseconds
```

Diagnosis: The developer changed the timeout from seconds to milliseconds but didn't update the callers. `wifi_connect` was passing a 30-second timeout, which became 30,000 milliseconds = 30,000 seconds = 8.3 hours. The connection attempt was just taking extremely long.

What Code Crawler provided: Instant blast radius analysis, so you knew exactly which functions to check. Without it, you'd manually grep through the codebase.

36.2 Scenario: "Serial Console Crash Log"

Setup: You see this on the serial console:

```
[  345.123456] wifi_daemon: FATAL: null pointer dereference at wifi_hal.c:167
[  345.123457] wifi_daemon: Call trace:
[  345.123458]   wifi_scan_callback+0x4c/0x120
[  345.123459]   nl80211_scan_done+0x38/0x80
[  345.123460]   genl_rcv_msg+0x1a4/0x200
```

Using Code Crawler:

```
AI: "Correlate these crash log lines to source code"
```

```
# What the MCP server does:
@mcp_tool("correlate_serial_log")
async def correlate(log_lines):
    results = []
    for line in log_lines:
        # Extract function name: "wifi_scan_callback+0x4c/0x120"
        match = re.search(r'(\w+)\+0x', line)
        if match:
            func_name = match.group(1)

            # Look up in database
            func = db.execute("""
                SELECT f.name, f.signature, f.start_line, f.end_line, fi.path
                FROM Function f
                JOIN contains_func cf ON f.id = cf.func_id
```

```
                JOIN File fi ON cf.file_id = fi.id
                WHERE f.name = ?
            """, [func_name]).fetchone()

            if func:
                results.append({
                    "log_line": line,
                    "function": func[0],
                    "signature": func[1],
                    "location": f"{func[4]}:{func[2]}-{func[3]}",
                    "summary": func[5] if len(func) > 5 else "",
                })

    return results
```

```
Response:
  wifi_scan_callback → wifi_hal.c:145-180 (complexity: 8)
    "Handles scan results from netlink. Allocates result buffer."

  nl80211_scan_done → nl80211_ops.c:230-255 (complexity: 3)
    (i1 tier — stub only, this is a kernel/upstream function)

  Likely crash point: wifi_hal.c:167 (line 167 is inside wifi_scan_callback)
```

Diagnosis: The crash is at line 167, which is inside `wifi_scan_callback`. The function summary says "Allocates result buffer" — likely a NULL check was missing after allocation.

36.3 Scenario: "Adding a New Feature — Where Do I Start?"

Setup: You need to add Bluetooth LE (BLE) support to the firmware. You've never touched this codebase before.

Using Code Crawler:

```
AI: "Search for bluetooth-related code"
```

```
MCP search_code("bluetooth connection handling"):
  1. bt_handler.c (score: 0.82)
     - bt_connect() — "Initiates classic Bluetooth connection via HCI"
     - bt_disconnect() — "Tears down active BT connection"

  2. bt_scan.c (score: 0.71)
     - bt_scan_start() — "Starts BT device discovery"
     - bt_scan_stop() — "Stops discovery and returns results"

  3. connectivity_manager.c (score: 0.65)
     - dispatch_connect() — "Routes connection requests to WiFi or BT handler"
```

```
AI: "Show me the architecture — what calls what around bt_connect?"
```

```
Call hierarchy (direction: both, depth: 3):

  Callers of bt_connect:
    dispatch_connect → bt_connect
    main → init_radios → bt_connect (for initial pairing)

  Callees of bt_connect:
    bt_connect → hci_open_dev
    bt_connect → hci_create_connection
    bt_connect → bt_set_state
```

What you learn: BLE support should follow the same pattern as classic BT — implement `ble_connect`, `ble_scan_start`, etc., and register them with `dispatch_connect`. The architecture is already designed for multiple transport types.

---

37. Architecture Decision Records (ADRs)

ADR-001: Why DuckDB Instead of SQLite or PostgreSQL

Context: We need a database for storing the code index. Options:

- SQLite: Small, embedded, universally available
- PostgreSQL: Powerful, feature-rich, client-server
- DuckDB: Analytical, embedded, column-oriented

Decision: DuckDB

Rationale:

Criterion	SQLite	PostgreSQL	DuckDB
Deployment	Embedded (single file)	Server (requires install)	Embedded (single file)
Analytics	Slow on aggregations	Fast	Very fast
Array columns	No	Yes	Yes (FLOAT[], JSON)
Graph queries	No	Via extension	Yes (DuckPGQ)
Vector search	No	Via pgvector	Yes (VSS extension)
Write speed	Very fast	Fast	Moderate
Read speed	Moderate	Fast	Very fast
Memory	Tiny	Large	Moderate
Multi-process	Limited (write lock)	Yes	Limited

Key factors:

1. **Embedded**: No server install required. Just `pip install duckdb`.
2. **Array columns**: We store 384-dim embeddings as `FLOAT[384]`. SQLite would need a separate table or blob encoding.
3. **DuckPGQ**: Graph queries (call chains, impact analysis) are first-class.
4. **VSS**: Vector similarity search without an external service.

Trade-off: DuckDB is slower than SQLite for individual row inserts. We mitigate this with batch inserts.

ADR-002: Why Tree-sitter Instead of libclang or ANTLR

Context: We need a C parser. Options:

- libclang: Official LLVM C parser, most accurate
- ANTLR: Parser generator framework
- Tree-sitter: Incremental parser generator

Decision: Tree-sitter

Rationale:

Criterion	libclang	ANTLR	Tree-sitter
Language support	C/C++/ObjC only	Many (need grammars)	Many (community grammars)
Error tolerance	Poor (fails on incomplete code)	Moderate	Excellent
Speed	Slow (full semantic analysis)	Fast	Very fast
Install complexity	Needs LLVM libraries	Java dependency	Pure C library
Incremental	No	No	Yes
Python bindings	Yes (clang.cindex)	Yes	Yes (tree-sitter)

Key factors:

1. **Error tolerance**: Source files being indexed may have syntax errors (incomplete edits, platform-specific extensions). Tree-sitter continues parsing around errors.
2. **Speed**: We parse thousands of files. Tree-sitter is ~10× faster than libclang for our use case.
3. **Multi-language**: Adding Python, Shell, or Rust support is just installing another Tree-sitter grammar.
4. **No LLVM**: libclang requires LLVM libraries (~200MB). Tree-sitter's C grammar is ~500KB.

Trade-off: Tree-sitter produces a concrete syntax tree, not an AST. We lose some semantic information (type resolution, overload resolution). For our use case (structural extraction), this is acceptable.

ADR-003: Why Frozen Dataclasses Instead of Dictionaries

Context: DTOs could be plain Python dicts or dataclasses.

Decision: Frozen dataclasses

Rationale:

```
# Dictionary approach:
result = {
    "name": "wifi_connect",
    "signature": "int wifi_connect(...)",
    "start_line": 14,
}
# Problems:
result["naem"]  # Typo → KeyError at runtime, not caught by IDE
result["name"] = "changed"  # Mutation → hard-to-find bugs
result["new_field"] = "surprise"  # No schema enforcement

# Dataclass approach:
@dataclass(frozen=True)
class FunctionDef:
    name: str
    signature: str
    start_line: int

result = FunctionDef(name="wifi_connect", signature="...", start_line=14)
# Benefits:
result.naem      # IDE shows error immediately (red squiggle)
result.name = "x"  # FrozenInstanceError → caught immediately
result.new_field  # AttributeError → caught immediately
```

For C developers: `frozen=True` is like declaring a `const struct`. Once created, it can never be modified. This eliminates an entire category of bugs.

ADR-004: Why an Event Bus Instead of Direct Function Calls

Context: Components need to communicate (e.g., "file parsed" → store in DB, update stats, build manifest).

Option A: Direct calls

```
result = crawler.parse(file)
database.store(result)
stats.update(result)
manifest.build(result)
```

Option B: Event bus

```
result = crawler.parse(file)
bus.publish("file.parsed", result)  # All interested components react
```

Decision: Event Bus

Rationale:

1. **Decoupling**: Adding a new subscriber (e.g., a code quality analyzer) doesn't require changing the pipeline.
2. **Testability**: Components can be tested in isolation by publishing events directly.

3. **Error isolation**: If stats.update() crashes, manifest.build() still runs.
4. **Extensibility**: Plugins can subscribe to any event without modifying core code.

Trade-off: Slightly harder to debug (event flow is implicit, not explicit call chain). We mitigate this with event logging.

C analogy: This is exactly like D-Bus on Linux — services publish signals, other services subscribe. Or like Netlink — the kernel publishes events, userspace listens.

ADR-005: Why a Plugin Architecture for Crawlers

Context: We need to support multiple languages. Options:

- Hard-code all parsers in the pipeline
- Use a plugin/strategy pattern

Decision: Plugin pattern (crawlers as plugins)

Rationale:

```
# Hard-coded approach:
class Pipeline:
    def parse(self, file):
        if file.language == "c":
            return self._parse_c(file)
        elif file.language == "python":
            return self._parse_python(file)
        # ... every new language requires modifying this file

# Plugin approach:
class Pipeline:
    def parse(self, file):
        crawler = self._registry.get_by_language(file.language)
        return crawler.parse(file)
        # New languages: just register a new crawler. Pipeline unchanged.
```

C analogy: This is like Linux kernel modules. Want WiFi support? Load the WiFi module. Want BT? Load the BT module. The kernel core doesn't change.

---

38. Data Flow Trace Tables

These tables show the exact data transformations at each stage for a complete indexing run of 3 files.

38.1 Input Files

File	Size	Lines	Functions	Includes
main.c	250B	15	1 (main)	wifi.h
wifi.c	800B	45	3 (wifi_connect, wifi_scan, wifi_get_status)	wifi.h, stdio.h

File	Size	Lines	Functions	Includes
wifi.h	150B	10	0 (declarations only)	—

38.2 Stage 1: Discovery

#	Event	Data Created
1	Walk filesystem	Find 3 matching files (*.c, *.h)
2	Filter extensions	All 3 pass (`.c` and `.h` in LANGUAGE_MAP)
3	Filter size	All 3 pass (all < 1MB)
4	Compute hashes	SHA-256 for each file
5	Create DTOs	3 × FileInfo objects
6	Publish events	3 × "file.discovered" events

38.3 Stage 2: Build Detection

#	Check	Result
1	Look for meta-*/conf/layer.conf	Not found
2	Look for CMakeLists.txt	Not found
3	Look for Makefile	Found!
4	Classify as "generic + Makefile"	BuildConfig(system="generic", variant="makefile")

38.4 Stage 3: Tier Classification

Directory	Heuristic	Evidence	Assigned Tier	Reason
src/	Custom code	Has .c files, no known upstream patterns	i3 (Full)	"Custom source directory"

38.5 Stage 4: Parsing

File	Crawler	Functions Extracted	Calls Extracted	Structs	Globals	Includes
main.c	CCrawler	main (cx:2)	wifi_connect (from main)	0	0	wifi.h
wifi.c	CCrawler	wifi_connect (cx:4), wifi_scan (cx:3), wifi_get_status (cx:1)	printf (from wifi_connect), hal_wifi_connect (from wifi_connect), hal_scan (from wifi_scan)	0	g_wifi_state (static int)	wifi.h, stdio.h
wifi.h	CCrawler	0 (declarations only)	0	wifi_config (struct)	0	—

38.6 Stage 5: Database State After Insertion

File table:

id	path	language	hash	loc
1	main.c	c	abc123	15
2	wifi.c	c	def456	45
3	wifi.h	c	ghi789	10

Function table:

id	name	signature	start_line	end_line	complexity
1	main	int main(void)	3	15	2
2	wifi_connect	int wifi_connect(const char *ssid, int timeout)	5	25	4
3	wifi_scan	int wifi_scan(wifi_config *cfg)	27	38	3
4	wifi_get_status	int wifi_get_status(void)	40	45	1

contains_func table:

file_id	func_id
1	1
2	2
2	3
2	4

calls table:

caller_id	callee_id	call_site_line	is_indirect
1	2	10	false
2	—	18	false
2	—	20	false
3	—	30	false

(Note: callee_id for external functions like `printf`, `hal_wifi_connect` may be NULL or point to stub entries)

includes_file table:

source_id	target_id	is_system
1	3	false
2	3	false
2	—	true

38.7 Stage 5: Priority Scoring

func_id	name	tier	usage	centrality	build	runtime	recency	composite
1	main	1.0	0.0	0.1	1.0	0.0	0.9	**0.53**
2	wifi_connect	1.0	1.0	0.8	1.0	0.0	0.9	**0.77**
3	wifi_scan	1.0	0.0	0.2	1.0	0.0	0.9	**0.52**
4	wifi_get_status	1.0	0.0	0.0	1.0	0.0	0.9	**0.48**

wifi_connect ranks highest because it has the most callers (usage=1.0) and highest centrality (0.8 — it bridges main to the HAL layer).

---

39. Understanding the Embedding Pipeline in Detail

39.1 What Exactly Gets Embedded

Not all text is worth embedding. We carefully construct the embedding input:

```
def build_embedding_text(func: FunctionDef, file_path: str) -> str:
    """Build the text that gets converted to a vector.

    Why this format?
    - Function name is most important (weighted first)
    - Signature provides parameter context
    - File path provides component context
    - Doc comment provides intent context

    We DON'T include the function body because:
    1. Bodies can be very long (blow up token limits)
    2. Semantic models work best on natural-language-like text
    3. The signature + name captures WHAT, not HOW
    """
    parts = [
        func.name.replace('_', ' '),  # "wifi_connect" → "wifi connect"
        func.signature,               # Full type information
    ]

    if func.doc_comment:
        parts.append(func.doc_comment)

    # Add file path context
    path_parts = file_path.replace('/', ' ').replace('_', ' ')
    parts.append(path_parts)

    return " | ".join(parts)

# Example output:
# "wifi connect | int wifi_connect(const char *ssid, int timeout) | src wifi
hal c"
```

39.2 The Embedding Model Internals

```
Input text: "wifi connect | int wifi_connect(const char *ssid, int timeout)"
    ↓
Tokenizer (WordPiece):
    ["wifi", "connect", "|", "int", "wifi", "_", "connect", "(", "const", ...]
    ↓
Token IDs:
    [8721, 4682, 1064, 3420, 8721, 1035, 4682, 1006, 9530, ...]
    ↓
Transformer (6 layers, 384 hidden dim):
    Layer 1: Self-attention → each token attends to all other tokens
    Layer 2: Feed-forward → non-linear transformation
    ... (6 layers total)
    ↓
Token embeddings: shape [14 tokens, 384 dimensions]
    ↓
Mean pooling: Average all token embeddings
    ↓
Final embedding: shape [384] = one vector
    [0.0234, -0.1567, 0.3421, 0.0891, ..., -0.0543]
```

39.3 How Similarity Search Works End-to-End

```
User query: "how to connect to wifi"
    ↓
1. Embed query: model.encode("how to connect to wifi") → query_vector [384
floats]
    ↓
2. HNSW index search: Find approximate nearest neighbors
    - Start at top layer of the HNSW graph
    - Navigate to the region closest to query_vector
    - Return top 10 closest vectors
    ↓
3. Compute exact cosine distances for the 10 candidates:
    cos_dist(query, func_1_embedding) = 0.12  ← closest! (lower = more similar)
    cos_dist(query, func_2_embedding) = 0.23
    cos_dist(query, func_3_embedding) = 0.31
    ...
    ↓
4. Join with metadata:
    func_1 = wifi_connect (distance: 0.12, priority: 0.77, file: wifi.c)
    func_2 = wifi_scan    (distance: 0.23, priority: 0.52, file: wifi.c)
    ↓
5. Build IndexManifests for matched files
    ↓
6. Return to AI agent as JSON
```

39.4 Why 384 Dimensions?

Model	Dimensions	Quality	Speed	Memory

Model	Dimensions	Quality	Speed	Memory
all-MiniLM-L6-v2	384	Good	Fast	Low
all-MiniLM-L12-v2	384	Better	Medium	Low
all-mpnet-base-v2	768	Best	Slow	High
text-embedding-3-small	1536	Best (API)	N/A	Very High

We chose 384 because:

- **Memory**: 384 × 4 bytes × 100K functions = 146 MB (vs 585 MB for 1536-dim)
- **Speed**: Embedding generation is 2× faster than 768-dim models
- **Quality**: For code search, 384-dim captures enough semantic nuance
- **Offline**: No API calls needed, runs entirely on your machine

---

40. How the Property Graph Works Under the Hood

40.1 What DuckPGQ Does

DuckPGQ lets you define a "graph view" on top of existing tables. No data is copied — it's just a logical overlay.

```
-- Define the graph
CREATE PROPERTY GRAPH code_graph
    VERTEX TABLES (
        Function PROPERTIES (name, signature, complexity),
        File     PROPERTIES (path, language, loc)
    )
    EDGE TABLES (
        calls
            SOURCE KEY (caller_id) REFERENCES Function (id)
            DESTINATION KEY (callee_id) REFERENCES Function (id)
            PROPERTIES (call_site_line, is_indirect),
        contains_func
            SOURCE KEY (file_id) REFERENCES File (id)
            DESTINATION KEY (func_id) REFERENCES Function (id)
    );
```

40.2 Graph Query Examples

```
-- Find all functions reachable from main within 3 hops
FROM GRAPH_TABLE (code_graph
    MATCH (a:Function WHERE a.name = 'main')
          -[e:calls]->{1,3}
          (b:Function)
    COLUMNS (a.name AS caller, b.name AS callee,
             element_id(e) AS edge_id)
);

-- Find the shortest path between two functions
```

```
FROM GRAPH_TABLE (code_graph
    MATCH p = SHORTEST (a:Function WHERE a.name = 'main')
              -[e:calls]->*
              (b:Function WHERE b.name = 'hal_wifi_connect')
    COLUMNS (path_length(p) AS distance, nodes(p) AS path_nodes)
);

-- Find which files are most connected (most include edges)
FROM GRAPH_TABLE (code_graph
    MATCH (a:File)-[e:contains_func]->(f:Function)-[c:calls]->(g:Function)<-
[cf:contains_func]-(b:File)
    WHERE a.path != b.path
    COLUMNS (a.path AS file_a, b.path AS file_b, COUNT(*) AS connection_count)
)
ORDER BY connection_count DESC;
```

40.3 When to Use Graph Queries vs Regular SQL

Query Type	Use Regular SQL	Use DuckPGQ
Find function by name	□ Simple WHERE	× Overkill
Direct callers/callees	□ Single JOIN	× Overkill
Transitive call chain	□ Recursive CTE works	□ Cleaner syntax
Shortest path	× Complex CTE	□ Built-in
Pattern matching	× Very hard in SQL	□ Built-in
Betweenness centrality	× Very hard in SQL	□ Built-in

Rule of thumb: If you need to follow edges more than 2 hops, use DuckPGQ. For 1-2 hops, regular JOIN is simpler and faster.

---

41. Security Considerations

41.1 What We Trust

Input	Trust Level	Mitigation
Source files on disk	High	Files are on the developer's machine
User config (TOML)	High	User explicitly created it
AI agent requests (MCP)	Medium	Validate all input schema
Git data	Medium	Could have malicious commits
Downloaded packages	Low	Never execute, only read

41.2 MCP Input Validation

```
# Every tool validates its input against the schema:
@mcp_tool("search_code")
async def search_code(query: str, language: str = None, limit: int = 10):
    # Validate limit
    if limit < 1 or limit > 100:
        raise ValueError("limit must be between 1 and 100")

    # Validate language
    VALID_LANGUAGES = {"c", "cpp", "python", "shell", "makefile"}
    if language and language not in VALID_LANGUAGES:
        raise ValueError(f"Invalid language: {language}")

    # Parameterized queries prevent SQL injection
    db.execute("SELECT * FROM Function WHERE name = ?", [query])
    #                                                ^ parameterized, NOT string
concatenation!

    # BAD (SQL injection):
    # db.execute(f"SELECT * FROM Function WHERE name = '{query}'")
    # If query = "'; DROP TABLE Function; --" → database destroyed!
```

41.3 Read-Only MCP Database Access

```
# The MCP server opens the database in READ-ONLY mode:
conn = duckdb.connect(db_path, read_only=True)
# This prevents the AI agent from modifying the index
# Even if a tool has a bug, it can't corrupt the database
```

---

42. Comparison with Other Code Intelligence Tools

Feature	Code Crawler v4	ctags	cscope	Sourcetrail	GitHub Code Search
Language support	C, Python, Shell (+extensible)	40+	C/C++/Java	C/C++/Java/Python	Many
Semantic search	□ Vector embeddings	×	×	×	□ (cloud)
Call graph	□ Full + graph queries	×	□	□	Partial
Build awareness	□ Yocto, Buildroot, Kernel	×	×	×	×
Priority scoring	□ 6-dimension	×	×	×	□ (proprietary)
AI integration	□ MCP protocol	×	×	×	×

Feature	Code Crawler v4	ctags	cscope	Sourcetrail	GitHub Code Search
Incremental	□ Hash-based	×	×	□	□
Offline	□ Fully	□	□	□	× (cloud)
Team sharing	□ Git-tracked DB	□ (file)	□ (file)	×	□
Token efficiency	□ Manifests (~500 tok)	×	×	×	×
Telemetry correlation	□ Crash log → source	×	×	×	×
Database	DuckDB (SQL + graph + vector)	Flat file	Binary	SQLite	Proprietary
Install	`pip install codecrawler`	Package manager	Package manager	GUI installer	Browser

End of Part IV

Part I: Chapters 1–13 (System Overview) Part II: Chapters 14–24 (Deep-Dive Fundamentals) Part III: Chapters 25–34 (Source Code Walkthroughs & Advanced Topics) Part IV: Chapters 35–42 (Hands-On Exercises, Real-World Scenarios & Architecture Decisions)

Total: 42 chapters

Written for embedded Linux C developers who want to understand, study, and implement the Code Crawler system.

□ **Code Crawler v4** — *Index Smart, Query Fast, Ship Confident*

PART V — APPENDICES & REFERENCE

Reference material for daily use. Print these pages and pin them next to your monitor.

Appendix A: Complete Regex Reference for C Parsing

Every regex used in Code Crawler's fallback parser, dissected character by character.

A.1 Function Definition Regex

```
Pattern:  ^([\w\s\*]+?)\s+(\w+)\s*\(([^)]*)\)\s*\{
          ↑               ↑       ↑              ↑
          |               |       |              └── Opening brace
          |               |       └── Parameters (anything not a closing paren)
          |               └── Function name (word characters)
          └── Return type (word chars, spaces, asterisks — lazy match)
```

Detailed breakdown:

Character(s)	Meaning	C Example
`^`	Start of line	(anchors to line start)
`[\w\s\*]+?`	Return type: word chars, whitespace, `*` (lazy)	`static int`, `void`, `struct wifi_config *`
`\s+`	At least one whitespace	Space between type and name
`(\w+)`	Function name (captured)	`wifi_connect`
`\s*`	Optional whitespace	
`\(`	Opening parenthesis (literal)	`(`
`([^)]*)`	Parameters: anything except `)` (captured)	`const char *ssid, int timeout`
`\)`	Closing parenthesis (literal)	`)`
`\s*`	Optional whitespace	
`\{`	Opening brace (literal)	`{`

What it matches:

```
int wifi_connect(const char *ssid, int timeout) {    □
static void hal_init(void) {                          □
struct foo *create_foo(int count) {                    □
```

What it DOESN'T match (limits of regex):

```
int
wifi_connect(const char *ssid,                        × (multiline signature)
             int timeout) {
int wifi_connect(void(*callback)(int)) {              × (nested parens in
params)
#define FUNC(name) int name(void) {                   × (macro-generated)
```

A.2 Include Regex

```
Pattern:  ^\s*#\s*include\s+([<"])([^>"]+)[>"]
                              ↑      ↑
                              |      └── Header name (anything not > or ")
                              └── Opening delimiter (< or ")
```

Match	`([<"])`	`([^>"]+)`	System?

Match	([<"])	([^>"]+)	System?
`#include "wifi.h"`	`"`	`wifi.h`	No
`#include <stdio.h>`	`<`	`stdio.h`	Yes
`# include "hal.h"`	`"`	`hal.h`	No (handles extra spaces)

A.3 Call Expression Regex

```
Pattern:  \b(?!if|while|for|switch|return|sizeof|typeof)\b(\w+)\s*\(
          ↑                                                     ↑
          └── Negative lookahead: NOT these keywords            └── Function name
```

Why the negative lookahead?

Without it, control-flow keywords would match as "function calls":

```
if (x > 0) { ... }      // "if" is NOT a function call
while (running) { ... }  // "while" is NOT a function call
wifi_connect(ssid);      // "wifi_connect" IS a function call
```

A.4 Macro Definition Regex

```
Pattern:  ^\s*#\s*define\s+(\w+)(?:\s+(.+))?$
                            ↑        ↑
                            |        └── Optional value (everything after name)
                            └── Macro name
```

Source	Name	Value
`#define MAX_LEDS 8`	`MAX_LEDS`	`8`
`#define CONFIG_WIFI`	`CONFIG_WIFI`	(empty)
`#define PIN(x) (16 + (x))`	`PIN`	`(x) (16 + (x))`

A.5 Struct Definition Regex

```
Pattern:  \bstruct\s+(\w+)\s*\{([^}]*)\};
                      ↑        ↑
                      |        └── Members (everything inside braces)
                      └── Struct name
```

Source	Name	Members

Source	Name	Members
`struct wifi_config { char ssid[32]; int channel; };`	`wifi_config`	`char ssid[32]; int channel;`

Limitation: Nested structs (`struct a { struct b { int x; }; };`) break this regex because `[^}]*` stops at the first `}`.

Appendix B: Python Standard Library Functions Used in Code Crawler

A reference for C developers who aren't familiar with Python's standard library.

B.1 `pathlib.Path` — Like a Smart `char *`

```
from pathlib import Path

# Creating paths (like constructing a filepath string)
p = Path("/home/dev/rdk/ccsp/wifi/wifi_hal.c")

# Properties (like parsing the string)
p.name        # "wifi_hal.c"       — basename
p.stem        # "wifi_hal"         — basename without extension
p.suffix      # ".c"               — extension
p.parent      # Path("/home/dev/rdk/ccsp/wifi")
p.parts       # ("/", "home", "dev", "rdk", "ccsp", "wifi", "wifi_hal.c")

# Operations
p.exists()      # True/False — like access(path, F_OK)
p.is_file()     # True/False — like S_ISREG(stat.st_mode)
p.is_dir()      # True/False — like S_ISDIR(stat.st_mode)
p.stat()        # os.stat result — like stat(path, &sb)
p.read_text()   # Read entire file as string — like malloc+read+close
p.read_bytes()  # Read entire file as bytes
p.write_text("hello")  # Write string to file — like open+write+close

# Path math
Path("/home") / "dev" / "rdk"   # Path("/home/dev/rdk")  — like snprintf(buf,
"%s/%s/%s", ...)
p.relative_to("/home/dev")      # Path("rdk/ccsp/wifi/wifi_hal.c")

# Glob (like find + fnmatch)
Path("/project").glob("**/*.c")    # All .c files recursively
Path("/project").glob("*.h")       # .h files in current directory only
```

B.2 `hashlib` — Like OpenSSL Digest Functions

```
import hashlib
```

```
# Create a SHA-256 hash (like SHA256_Init/Update/Final)
h = hashlib.sha256()
h.update(b"hello")          # Like SHA256_Update(&ctx, data, len)
h.update(b" world")         # Can call multiple times
digest = h.hexdigest()      # Like SHA256_Final + hex encoding
# digest = "b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9"

# One-liner for simple cases
hashlib.sha256(b"hello world").hexdigest()
```

B.3 `collections.defaultdict` — Like a Hash Map with Default Values

```
from collections import defaultdict

# Regular dict
d = {}
d["key"]              # KeyError! (like accessing uninitialized memory)

# defaultdict
d = defaultdict(list)
d["key"]              # Returns [] (empty list — auto-created)
d["key"].append(42)   # Now d["key"] == [42]

# Used in EventBus:
handlers = defaultdict(list)
handlers["file.parsed"].append(callback)
# No need to check if "file.parsed" key exists first
```

B.4 `logging` — Like syslog or printf-based Debug Logging

```
import logging

# Create a logger (like getting a log handle)
logger = logging.getLogger(__name__)
# __name__ = module name, e.g., "codecrawler.crawlers.c_crawler"

# Log levels (like syslog priorities)
logger.debug("Parsing %s", filepath)     # LOG_DEBUG   — verbose detail
logger.info("Indexed %d files", count)   # LOG_INFO    — normal operation
logger.warning("Tree-sitter unavailable") # LOG_WARNING — degraded operation
logger.error("Failed to parse %s", path)  # LOG_ERR     — error, continuing
logger.exception("Crash in handler")      # LOG_ERR + stack trace

# C equivalent:
# syslog(LOG_INFO, "Indexed %d files", count);
# fprintf(stderr, "[INFO] Indexed %d files\n", count);
```

B.5 `json` — Like cJSON or jansson

```python
import json

# Serialize (struct → JSON string)
data = {"name": "wifi_connect", "complexity": 4, "calls": ["hal_connect"]}
json_str = json.dumps(data, indent=2)
# '{\n  "name": "wifi_connect",\n  "complexity": 4,\n  "calls":
["hal_connect"]\n}'

# Deserialize (JSON string → dict)
parsed = json.loads(json_str)
parsed["name"]  # "wifi_connect"

# File I/O
with open("manifest.json", "w") as f:
    json.dump(data, f, indent=2)     # Write to file

with open("manifest.json") as f:
    loaded = json.load(f)            # Read from file
```

B.6 `os.walk` — Like `nftw()` or `opendir()`/`readdir()` Recursion

```python
import os

# Walk a directory tree (like nftw or manual recursive opendir)
for dirpath, dirnames, filenames in os.walk("/project"):
    # dirpath:   current directory being visited
    # dirnames:  list of subdirectory names (modifiable to control traversal!)
    # filenames: list of file names in current directory

    # Skip .git directory (modify dirnames IN PLACE)
    dirnames[:] = [d for d in dirnames if d != ".git"]

    for filename in filenames:
        full_path = os.path.join(dirpath, filename)
        print(full_path)
```

C equivalent pseudocode:

```c
void walk(const char *dir) {
    DIR *dp = opendir(dir);
    struct dirent *entry;
    while ((entry = readdir(dp)) != NULL) {
        if (entry->d_type == DT_DIR && strcmp(entry->d_name, ".git") != 0) {
            char subdir[PATH_MAX];
            snprintf(subdir, sizeof(subdir), "%s/%s", dir, entry->d_name);
            walk(subdir);  // recursive
        } else if (entry->d_type == DT_REG) {
            printf("%s/%s\n", dir, entry->d_name);
        }
    }
```

```
    closedir(dp);
}
```

Appendix C: DuckDB CLI Quick Reference

When you need to poke around the database directly.

C.1 Starting the CLI

```
# Open the Code Crawler database
duckdb .codecrawler/index.duckdb

# Read-only mode (recommended for exploration)
duckdb -readonly .codecrawler/index.duckdb
```

C.2 Essential Commands

```
-- List all tables
SHOW TABLES;

-- Describe a table schema
DESCRIBE Function;

-- Quick row count
SELECT COUNT(*) FROM Function;

-- First 5 rows
SELECT * FROM Function LIMIT 5;

-- Pretty-print output
.mode box
SELECT name, complexity FROM Function ORDER BY complexity DESC LIMIT 10;

-- Export results to CSV
.output results.csv
SELECT * FROM Function;
.output  -- reset to terminal

-- Export to JSON
COPY (SELECT * FROM Function LIMIT 10) TO 'functions.json' (FORMAT JSON);

-- Time a query
.timer on
SELECT COUNT(*) FROM calls;
.timer off

-- Exit
.quit
```

C.3 Common Exploration Queries

```sql
-- What's in the database?
SELECT
    (SELECT COUNT(*) FROM File) as files,
    (SELECT COUNT(*) FROM Function) as functions,
    (SELECT COUNT(*) FROM calls) as call_edges,
    (SELECT COUNT(*) FROM includes_file) as includes;

-- Largest files
SELECT path, loc FROM File ORDER BY loc DESC LIMIT 10;

-- Most called functions
SELECT f.name, COUNT(*) as call_count
FROM Function f
JOIN calls c ON f.id = c.callee_id
GROUP BY f.name
ORDER BY call_count DESC
LIMIT 20;

-- Functions with no callers (dead code candidates)
SELECT f.name, fi.path
FROM Function f
JOIN contains_func cf ON f.id = cf.func_id
JOIN File fi ON cf.file_id = fi.id
WHERE f.id NOT IN (SELECT callee_id FROM calls)
  AND f.name NOT IN ('main', '__init__', 'setup', 'teardown')
ORDER BY fi.path;

-- Which directories have the most functions?
SELECT
    regexp_extract(fi.path, '^([^/]+)') AS top_dir,
    COUNT(DISTINCT fi.id) AS file_count,
    COUNT(cf.func_id) AS func_count,
    SUM(f.complexity) AS total_complexity
FROM File fi
JOIN contains_func cf ON fi.id = cf.file_id
JOIN Function f ON cf.func_id = f.id
GROUP BY top_dir
ORDER BY func_count DESC;
```

Appendix D: Troubleshooting Guide

D.1 "Tree-sitter is not installed"

```
WARNING: Tree-sitter unavailable, using regex fallback: No module named
'tree_sitter_c'
```

Fix:

```
pip install tree-sitter tree-sitter-c
```

If you get build errors:

```
# Install build tools (Ubuntu)
sudo apt install build-essential python3-dev

# Then retry
pip install tree-sitter tree-sitter-c
```

D.2 "DuckDB database is locked"

```
Error: IO Error: Could not set lock on file ".codecrawler/index.duckdb"
```

Cause: Another process is using the database. **Fix**: Close the other process, or if you're just reading:

```
# Open read-only
duckdb -readonly .codecrawler/index.duckdb
```

D.3 "Embedding model download fails"

```
Error: Connection refused while downloading sentence-transformers model
```

Fix: Download the model manually for offline use:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
model.save("/path/to/local/model")
```

Then configure Code Crawler to use the local path:

```
[embeddings]
model = "/path/to/local/model"
```

D.4 "Index seems incomplete — missing functions"

Possible causes:

1. File was excluded by patterns → check `[index].exclude_patterns`
2. File is too large → check `[index].max_file_size`

3. File has syntax errors → Tree-sitter still extracts what it can, regex may miss more
4. Macro-generated functions → v4 can't expand macros (v5 will)

Debug:

```
# Check what files were discovered
codecrawler status

# Check directly in the database
duckdb .codecrawler/index.duckdb -c "SELECT path FROM File ORDER BY path;"

# Check for a specific file
duckdb .codecrawler/index.duckdb -c "
    SELECT f.name, f.start_line
    FROM Function f
    JOIN contains_func cf ON f.id = cf.func_id
    JOIN File fi ON cf.file_id = fi.id
    WHERE fi.path LIKE '%wifi_hal%';
"
```

D.5 "Priority scores are all 0"

Cause: Priority scoring runs AFTER parsing. If parsing produced 0 functions, there's nothing to score. **Also**: Runtime score requires telemetry data. Without `codecrawler ingest-logs`, runtime scores are 0.

D.6 "MCP server doesn't start"

```
# Check if the port is in use
ss -tlnp | grep 8080

# Try a different port
codecrawler mcp --port 9090

# Check for Python errors
codecrawler mcp --verbose 2>&1 | head -50
```

Appendix E: Quick Reference Cheat Sheet

E.1 CLI Commands

```
codecrawler index --root ./project     # Index a project
codecrawler index --root ./project --force  # Re-index everything
codecrawler mcp                        # Start MCP server
codecrawler mcp --port 9090           # MCP on custom port
codecrawler status                     # Show index statistics
codecrawler watch --root ./project     # Watch for file changes
codecrawler ingest-logs crash.log      # Import crash/serial logs
```

```
codecrawler ui                          # Start web UI
codecrawler sync                        # Sync with team
```

E.2 Config File Location

```
# Project-specific
.codecrawler.toml

# Global
~/.config/codecrawler/config.toml

# Create default config
codecrawler init
```

E.3 Database Location

```
# Default
.codecrawler/index.duckdb

# Custom (via config)
[database]
path = "/custom/path/index.duckdb"
```

E.4 Important Tables

```
File           → Source files on disk
Function       → Functions extracted from files
calls          → Function call edges (caller → callee)
contains_func  → File ↔ Function associations
includes_file  → #include edges (source → target)
Directory      → Directory tree with tier assignments
PriorityScore  → 6-dimension priority scores
IndexManifest  → Compressed file representations for AI
TelemetryEvent → Imported crash/log data
BuildConfig    → Detected build system configuration
```

E.5 Key DTOs

```
FileInfo       → One file on disk (path, language, hash)
FunctionDef    → One function (name, signature, complexity)
StructDef      → One struct/class (name, members)
CallEdge       → Function A calls Function B
IncludeEdge    → File A includes File B
ParseResult    → Everything extracted from one file
TierClassification → Directory/file tier (0-3)
```

```
PriorityScoreResult → 6-dimension score for a function
IndexManifestBundle → Compressed file for AI consumption
```

E.6 Pipeline Stages (in order)

```
1. Discover files          → FileInfo
2. Detect build system     → BuildConfig
3. Classify tiers          → TierClassification
4. Parse files             → ParseResult
5. Score priorities        → PriorityScoreResult
6. Build manifests         → IndexManifestBundle
7. Generate embeddings     → vectors in DB
8. Run intelligence        → PatchSuggestion
```

E.7 Priority Score Formula

```
composite = 0.25 × tier          tier_map = {0:0, 1:0.33, 2:0.66, 3:1.0}
          + 0.20 × usage         = call_count / max_call_count
          + 0.15 × centrality    = betweenness_centrality (normalized)
          + 0.10 × build         = 1.0 if guard active, else 0.0
          + 0.15 × runtime       = runtime_hits / max_runtime_hits
          + 0.15 × recency       = 1 / (1 + days/7)
```

E.8 MCP Tools

```
search_code          → Semantic + keyword search
get_call_hierarchy   → Callers/callees tree
get_build_context    → CONFIG_* status
trace_ipc_flow       → Inter-process edges
correlate_serial_log → Crash log → source mapping
analyze_impact       → Blast radius analysis
sync_team            → Pull team annotations
```

End of Code Crawler v4 Complete Technical Study Guide

```
Parts:     5 (System Overview, Deep-Dive Fundamentals, Source Walkthroughs,
           Hands-On Exercises, Appendices & Reference)
Chapters:  42 + 5 Appendices
```

This guide is your complete reference for understanding, studying, and implementing Code Crawler. Whether you're debugging a WiFi driver crash at 3 AM or designing the v5 architecture, this document has you covered.

Written for embedded Linux C developers. No prior Python, SQL, or database experience assumed.

 Code Crawler v4 — *Index Smart, Query Fast, Ship Confident*